

AC700N_SDK_介绍

珠海市杰理科技股份有限公司

Zhuhai Jieli Technology Co.,LTD

版权所有，未经许可，禁止外传

修改记录

版本	更新日期	描述
V1.0	2020/04/28	AC700N 用户手册
更新:	● 建立初始版本 ● 定义文档格	
V1.1	2022/04/15	
更新:	● 合并音频开发快速指引，完善音频配置说明 ● 增加离线语音识别配置说明	
V1.2	2022/07/09	
更新:	● 增加辅听耳机说明 ● 增加混响 K 歌说明	
V1.2.1	2022/08/11	
更新:	● 增加辅听音量调节说明	
V1.2.2	2022/10/27	
更新	● 增加 linein 功能章节 ● 增加语音识别语言配置说明	
V1.2.3	2023/06/28	
更新	● 增加音效配置章节，添加虚拟低音说明	
V1.2.4	2023/07/05	
	● 增加第三方算法接入说明及对应的 FAQ	

目录

Chapter 1	SDK 开发包快速使用说明	7
1.1	编写目的	7
1.2	安装包管理工具	7
1.3	配置工具使用	10
1.4	UI 操作说明	15
1.5	MIC 配置	17
1.6	两个 IO 口推两个灯配置	19
1.7	是否使用配置文件的参数配置	20
1.8	充电仓使用注意点	21
1.9	长按复位设置	21
1.10	内置触摸按键配置	22
1.11	AC700N 输出 EVDD 电源说明	28
Chapter 2	VM 接口的使用	31
2.1	编写目的	31
2.2	详细说明	31
Chapter 3	通话效果调试	33
3.1	编写目的	33
3.2	工具配置说明	33
3.3	调试说明	33
Chapter 4	Audio 模块使用	34
4.1	编写目的	34
4.2	详细说明	34
4.3	在线 EQ 调试	35
4.4	DAC 配置	51
4.4.1	IDAC 高低压模式选择	51
4.4.2	DAC 输出幅度	51
4.4.3	DAC 输出通道配置	52
4.4.2	DAC 使用范例	53
4.5	ADC 配置	53
4.5.1	ADC 使用范例	53
4.5.2	通话过程 mic 数据获取	53

4.5.3 省电容 mic 使用	53
4.6 音量配置	53
4.6.1 音量调节方式	53
4.7 编解码配置	54
4.7.1 解码声道配置	54
4.7.2 编解码格式配置	55
4.8 音频同步配置	55
4.9 通话算法开发配置	56
4.9.1 算法离线开发	56
4.9.2 第三方算法开发	56
4.9.3 通话参数调试	57
4.9.4 双 mic 降噪 (ENC)	57
4.9.5 双 mic 风噪检测模块	58
4.9.6 双麦克风异常检测	59
4.9.7 神经网络降噪 DNS	61
4.9.8 双 MIC 话务耳机	61
4.9.9 双 MIC 话务耳机单双麦切换	62
4.10 离线语音识别配置	63
4.11 辅听功能配置	66
4.11.1 声学结构规范	72
4.11.2 使能配置	66
4.11.3 功能模块说明	69
4.11.4 辅听音量	72
4.11.5 听力验配	74
4.11.6 独立辅听+低功耗模式	80
4.12 混响 K 歌配置	81
4.13 LINEIN 功能配置	82
4.14 音效配置	84
4.14.1 虚拟低音	84
4.15 第三方音效算法接入	90
4.15.1 音乐播放音效算法	90
4.15.2 算法移植 FAQ	92
Chapter 5 TWS 功能使用说明	89

5.1 编写目的	89
5.2 TWS 的开关	89
5.3 TWS 配对方式选择	90
5.4 TWS 消息同步	92
5.5 TWS 互发数据接口	92
5.6 siri 状态的获取	92
5.7 三方通话接口	93
Chapter 6 常用设备使用介绍	89
6.1 编写目的	89
6.2 IOKEY 使用	89
6.3 ADKEY 使用	92
6.4 按键扫描配置和按键消息处理	93
6.5 关于按键一些特殊需求的修改说明	94
6.6 串口使用	96
6.7 IIC 接口使用	98
6.8 充电仓和测试盒通信接口配置	102
6.9 充电相关配置说明	103
Chapter 7 功耗说明	89
7.1 编写目的	89
7.2 功耗说明	89
Chapter 8 透传说明	89
8.1 编写目的	89
8.2 文件目录说明	89
8.3 功能开关	89
8.4 SPP 数据通信	89
8.5 BLE 数据通信	91
Chapter 9 TWS+透传流程说明	89
9.1 编写目的	89
9.2 文件目录说明	89
9.3 适配接口说明	89
9.4 函数接口介绍	89
Chapter 10 杰理之家 APP 介绍	88
10.1 编写目的	88

10.2 APP 介绍	88
10.3 文件目录说明	89
10.4 功能开关、配置	90
10.5 命令、数据 API 介绍	91
10.6 功能配置存储和读取	93
10.7 TWS 数据传输	94
Chapter 11 客户常见问题反馈解答	89
Q1:	89
Q2:	89

Chapter 1 SDK 开发包快速使用说明

1.1 编写目的

该文档主要描述 AC700N_SDK 开发包的使用方法及开发中注意的一些问题，为用户进行二次开发提供参考，其中包括：

安装包管理工具、配置工具使用、配置宏的选择以及充电仓使用注意事项

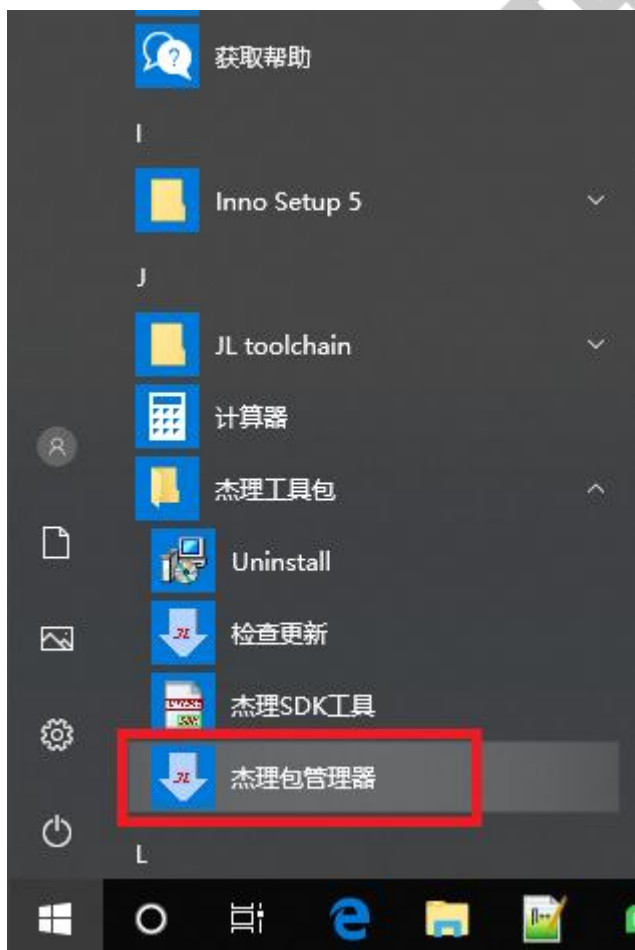
1.2 安装包管理工具

❖ 包管理工具安装

双击“杰理包管理器-setup-1.0.15.exe”安装，按照提示安装完成即可。

详细的安装及使用步骤如下：

1. 安装 杰理软件包 工具之后，开始菜单栏会出现对应的菜单



使用方式：杰理包管理器界面如下图所示：



* 并不是说有包都有执行按钮。这是因为有一些包里面没有可执行程序。

安装软件包：



也可以单独选择特定的包安装。

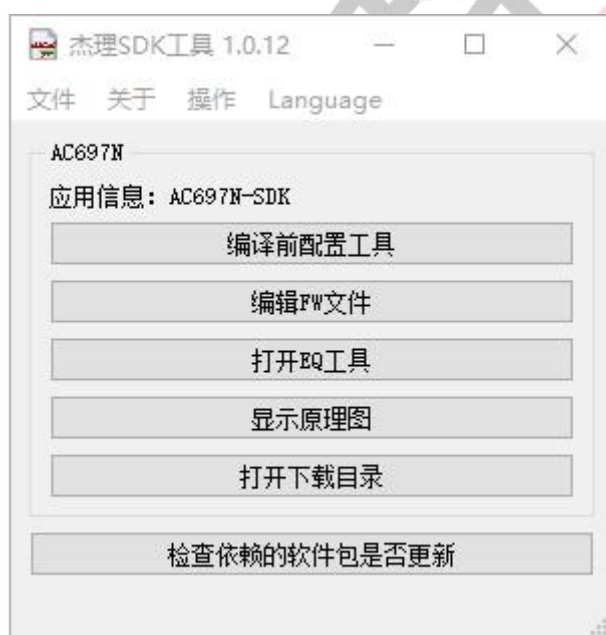
如果下载安装过的软件被破坏了、或者出现了缺失。可以点击“校验”来校验是否出现缺失或者破损，并从服务器中下载安装对应的正确文件。

1.3 配置工具使用

1、打开 sdk 工程目录，进入 cbp_out\cpu\br36\tools\AC700N_config_tool 目录；

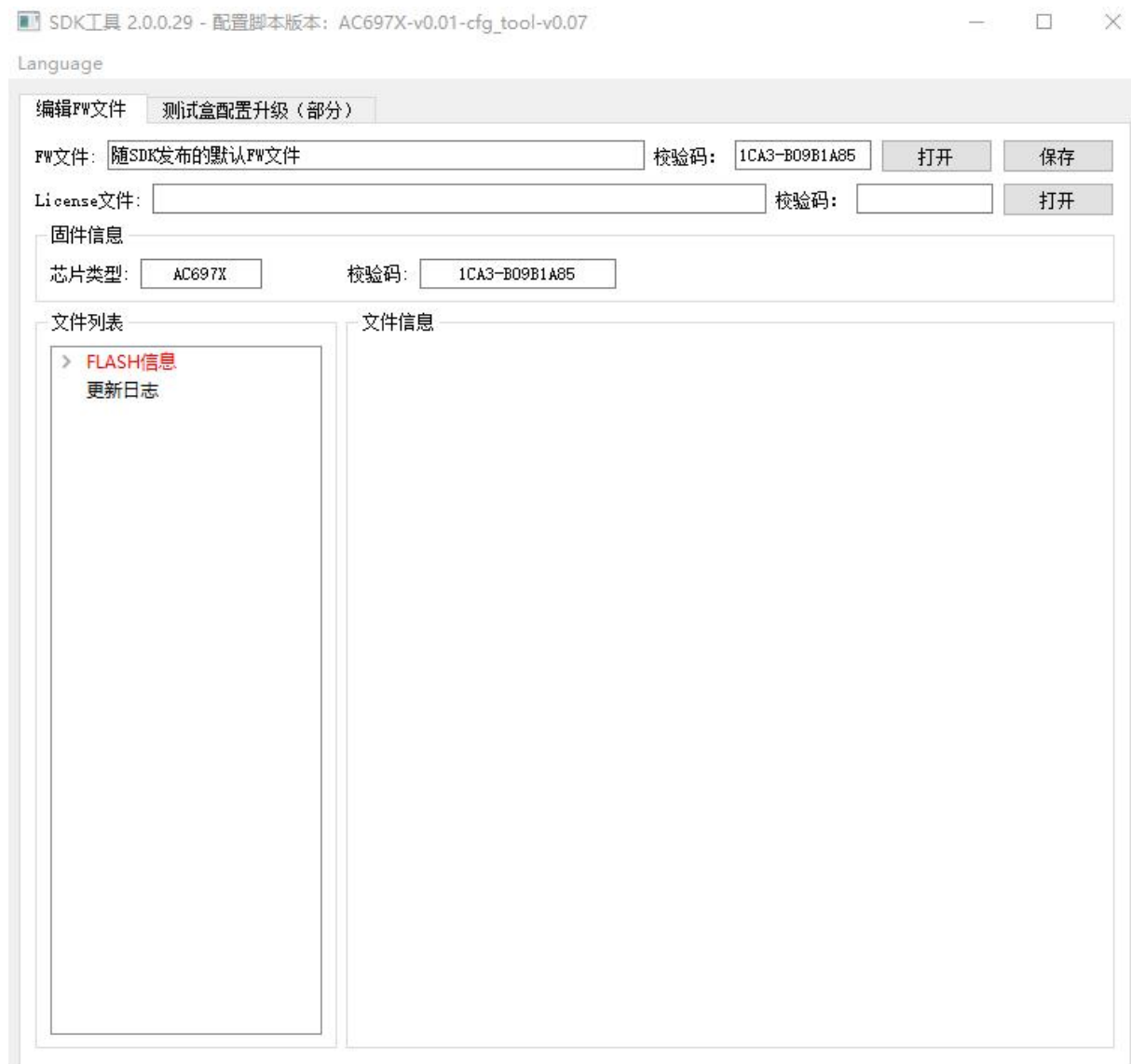


2、双击【AC700N-配置工具入口.jlxproj】，打开【杰理 SDK 工具】

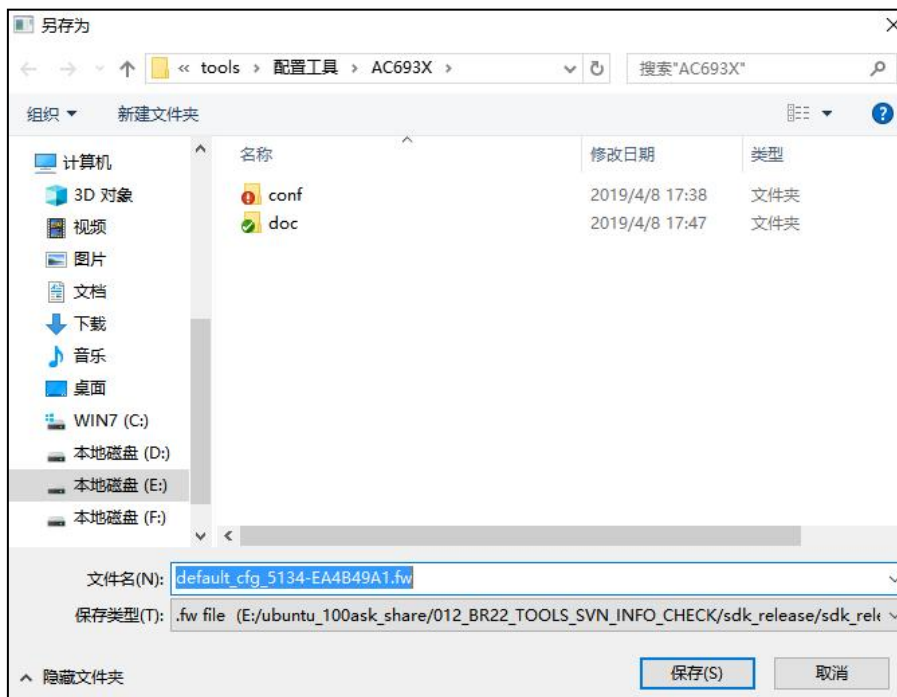


【杰理 SDK 配置工具】界面说明：

1) 编辑 FW 文件：点击【编辑 FW 文件】，可对一个 FW 文件的进行编辑，修改 FW 文件的板级配置、蓝牙配置，状态配置和提示音配置，界面如下：



点击【恢复默认值】可以恢复 sdk 发布时的原始配置，配置完成后，点击【保存】，可选择保存路径保存修改过后的 fw 和 ufw 文件；



编辑 fw 文件有以下几点需要注意：

(1) 板级配置的可配选项取决 cfg_tool.bin 文件，也就是说编译前配置选了什么，编辑 fw 文件就有什么配置，比如说编译前配置是选了 3 个 key，则编辑 fw 文件的时候也是只有 3 个 key 的选项。

(2) fw 版本号控制，只有工具和 fw 的版本号对得上，才能编辑对应的 fw 文件，版本号在 AC697N_config_tool\conf\entry 目录下的 user_cfg.lua 文件下修改，如下图：

```

1  设置版本信息
2  cfg:addKeyInfo("script_version", "AC693X-v0.0.3");
3
4  设置应用名称
5  product_name = "AC693X"; --此名称将显示于配置工具入口界面
6
7  设置配置工具开发状态
8  -- develop: 开发状态, 用于开发SDK使用
9  -- release: 发布状态, 用于发布上传使用

```

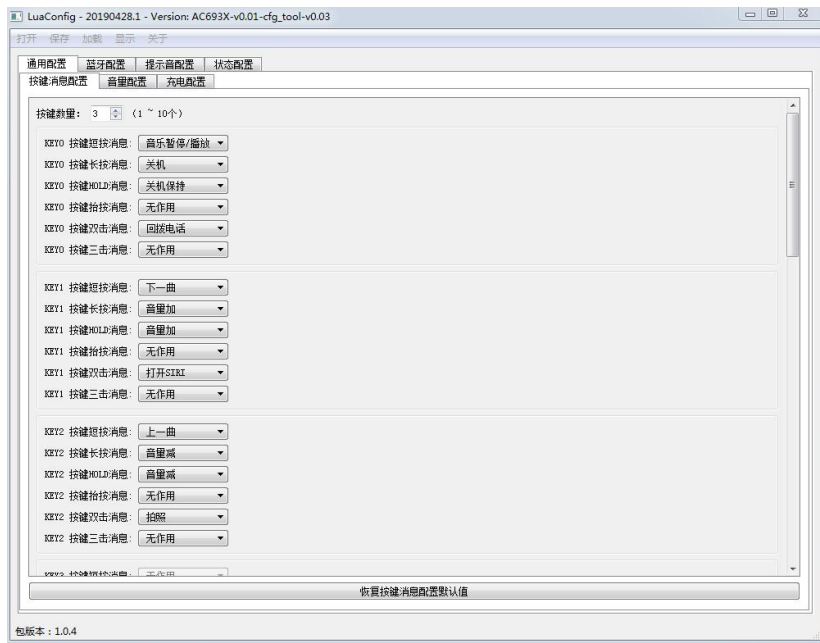
如果版本号提示不对，请确认版本号。

(3) fw 文件制作：当 fw 文件测试没有问题后，想发布一个可编辑的 fw 文件时，把生成的 fw 文件替换 AC697N_config_tool\conf\output\default 目录下 default_cfg.fw 文件即可

(4) 默认值的制作：用编译前选项配置好后，点击保存后会在 AC697N_config_tool\conf\output 生成一个 default_cfg.lua 文件，把该文件覆盖 AC697N_config_tool\conf\output\default 目录下的 default_cfg.lua 文件，到时在编辑 fw 文件的时候点击恢复默认设置，就会恢复之前设置的值。

2) 显示原理图：点击【显示原理图】，可以打开一个 doc 的文件夹，可以在该文件夹下存放原理图等相关文件；

3) 编译前配置：点击【编译前配置】，可以在 sdk 代码编译前进行相关配置项配置，打开界面如下：



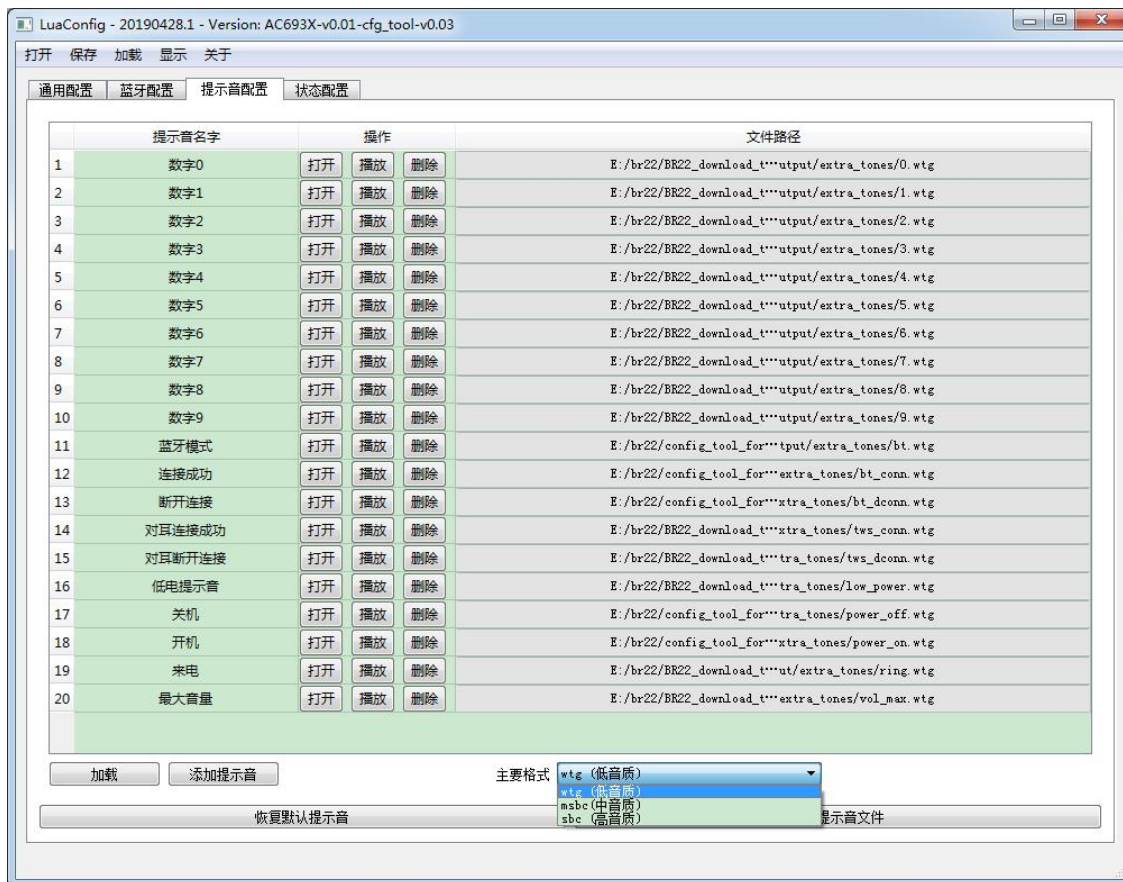
可以在编译前配置工具中进行通用配置、蓝牙配置、提示音配置和状态配置，点击【恢复默认值】可以恢复 sdk 发布时的原始配置，配置完成后，点击【保存】保存配置，将会输出 `cfg_tools.bin` 到下载目录中，**重新编译** sdk 代码可应用该最新配置；

4) 打开下载目录：点击【打开下载目录】，将会打开 tools 下载目录，`cfg_tools.bin` 将会输出到该目录下；

5) 检查依赖的软件包是否更新：点击【检查依赖的软件包是否更新】，将会检查相关配置工具的版本更新情况，如有更新，则下载相应更新即可；

3、提示音配置

提示音设置在工具的“提示音配置”选项里面，点击后如下图所示：



默认是加载 conf\output\extra_tones 下的提示音，如果有一个新的 tone.cfg 文件，点击加载按钮可以把该提示音文件加载进来。如果想更换提示音，直接把对应的提示音文件拉到对应选项即可。提示音工具提供 3 种音质提示音格式，每个提示音均可以单独指定成不同格式，主要是看 flash 的空间。比如“蓝牙模式”提示音想音质好点，可以先选 sbc，然后把对应提示音文件拉过去即可，其他格式也是可以这样操作，每个提示音生成是什么格式取决于拉提示音文件过去的时候选的是什么格式。

注意：报号数字提示音格式一定要选同一种，原始音频数据 sbc 要用 32k 的采样率转，msbc 要用 16k 的采样率转才能保证音质。

1.4 UI 操作说明

开机	耳机在盒子外，长按 2s 按键，耳机开机。
	开盖
关机	耳机在盒子外，长按 2s 按键，耳机关机。
	关盖
首次开机 TWS 配对规则	1) TWS 配对失败 6S 超时，耳机在和手机配对模式与 TWS 配对之间定时切换，如果和手机连接成功，则关掉 TWS 配对模式 2) TWS 配对成功，主耳进入和手机配对模式
非首次开机 (TWS 已配对)	TWS 回连，回连超时后： 1) 若已和手机有配对记录，主耳直接回连手机，回连超时，在和手机配对状态与 TWS 配对之间定时切换； 2) 若和手机无配对记录，主耳在和手机配对状态与 TWS 配对之间定时切换；
TWS 已连接，主耳和手机已连接。主耳走远和对耳超距断开连接，同时主耳和手机超距断开连接：	主耳回连附耳及手机，附耳只回连主耳机。 在 3 分钟内若连上了手机，没有连上对耳，耳机不会关机；若 3 分钟内没有连上手机，只连上了对耳，耳机关机。
耳机有 1 个按键	1) 来电响铃中-->单击左耳按键或右耳按键，接听来电 2) 来电响铃中-->长按左耳按键或右耳按键，拒接来电 3) 通话中-->单击左耳按键或右耳按键，结束通话 4) 播放音乐中-->单击左耳按键或右耳按键，音乐播放/暂停 5) 开机状态-->长按左耳按键或右耳按键，关机 6) 关机状态-->长按左耳按键或右耳按键，开机

7) 长按左耳按键或右耳按键 8 秒, 复位

耳机有 2 个 LED:

- 蓝色
- 红色

详细定义, 请参考以下 LED 灯效和提示音配置表:

不同状态下 LED 灯显示效果和提示音设置		
开始充电:	红灯慢闪	无提示音
充电完成:	蓝灯亮	无提示音
开机:	蓝灯亮	开机
关机:	红灯闪烁三下	关机
低电:	红灯五秒内闪烁一下	低电量
最大音量:	无LED灯显示效果	最大音量
来电:	无LED灯显示效果	来电
去电:	无LED灯显示效果	无提示音
通话中:	无LED灯显示效果	无提示音
蓝牙初始化完成:	红灯和蓝灯交替闪烁(慢闪)	无提示音
蓝牙连接成功:	蓝灯和红灯全灭	连接成功
蓝牙断开连接:	红灯和蓝灯交替闪烁(快闪)	断开连接
对耳连接成功:	红灯和蓝灯交替闪烁(快闪)	对耳连接成功
对耳断开连接:	红灯和蓝灯交替闪烁(慢闪)	对耳断开连接

1.5 MIC 配置

在每个 board.c 文件里都有配置 mic 参数的结构体，如下图所示：

```
struct adc_platform_data adc_data = {
    /*MIC 是否省隔直电容: 0: 不省电容 1: 省电容 */
    .mic_capless = 1,
    /*MIC LDO电流档位设置: 0:5 1:10ua 2:15ua 3:20ua*/
    .mic_ldo_isel = TCFG_AUDIO_ADC_LDO_SEL,
    /*MIC LDO电压档位设置, 也会影响MIC的偏置电压0:1.5v 1:8v 2:2.1v 3:2.4v 4:2.7v 5:3.0 */
    .mic_ldo_vsel = 3,
    /*MIC免电容方案需要设置, 影响MIC的偏置电压
    21:1.18K 20:1.42K 19:1.55K 18:1.99K 17:2.2K 16:2.4K 15:2.6K 14:2.91K 13:3.05K 12:3.5K 11:3.73K
    10:3.91K 9:4.41K 8:5.0K 7:5.6K 6:6K 5:6.5K 4:7K 3:7.6K 2:8.0K 1:8.5K */
    .mic_bias_res = 21,
    /*MIC电容隔直模式使用内部mic偏置(PA2)*/
    .mic_bias_inside = 0,
    /*保持内部mic偏置(PA2)输出*/
    .mic_bias_keep = 0,

    /*MIC1 是否省隔直电容: 0: 不省电容 1: 省电容 */
    .mic1_capless = 1,
    /*MIC1 LDO电流档位设置: 0:5 1:10ua 2:15ua 3:20ua*/
    .mic1_ldo_isel = TCFG_AUDIO_ADC_LDO_SEL,
    /*MIC1 LDO电压档位设置, 也会影响MIC的偏置电压0:1.5v 1:8v 2:2.1v 3:2.4v 4:2.7v 5:3.0 */
    .mic1_ldo_vsel = 3,
    /*MIC1免电容方案需要设置, 影响MIC的偏置电压
    21:1.18K 20:1.42K 19:1.55K 18:1.99K 17:2.2K 16:2.4K 15:2.6K 14:2.91K 13:3.05K 12:3.5K 11:3.73K
    10:3.91K 9:4.41K 8:5.0K 7:5.6K 6:6K 5:6.5K 4:7K 3:7.6K 2:8.0K 1:8.5K */
    .mic1_bias_res = 18,
    /*MIC1电容隔直模式使用内部mic偏置(PB7)*/
    .mic1_bias_inside = 0,
    /*保持内部mic偏置(PB7)输出*/
    .mic1_bias_keep = 0,

    // ladc 通道
    .ladc_num = ARRAY_SIZE(ladc_list),
    .ladc = ladc_list,
};
#endif
```

AC697N 系列有两个 mic_in，默认使用 mic0_in

Mic 的偏置方式选择：

(1) 省电容模式

内部提供偏置，配合内部偏置电阻，将 mic 的偏置电压调整到 1.4 到 1.5 之间，正常工作状态，mic 的电压会收敛稳定在 1.5v 左右。

(2) 隔直电容模式

(a) MIC_LDO 提供偏置：配合内部偏置电阻 mic_bias_res，调整 mic 的偏置电压

(b) DACVDD 或者其他电源提供偏置

1.主要关注以下变量：

1) mic_capless: 0: 选用不省电容模式 1: 选用省电容模式

2) mic_bias_res: mic_ldo 内部上拉偏置电阻，使用内部偏置时有效，选择范围为：

电阻档	1	2	3	4	5	6	7
电阻值	8.5k	8.0K	7.6K	7K	6.5K	6K	5.6K
电阻档	8	9	10	11	12	13	14
电阻值	5.0K	4.41K	3.91K	3.73K	3.5k	3.05K	2.91K
电阻档	15	16	17	18	19	20	21
电阻值	2.6K	2.4K	2.2K	1.99K	1.55K	1.42K	1.18K

电阻档和电阻值对应表

3) mic_ldo_vsel: mic_ldo 的偏置电压，与偏置电阻共同决定 mic 的偏置，选择范围为：

偏置档位	0	1	2	3	4	5
偏置电压	1.5v	1.8v	2.1v	2.4v	2.7v	3.0v

偏置档位和偏置电压对应表

4) mic_bias_inside: mic 外部电容隔直，芯片内部提供偏置电压，当 mic_bias_inside=1，可以正常使用 mic_bias_res 和 mic_ldo_vsel

注意：如果用的是硅 mic 的话要配置成不省电容模式，并且要 io 口独立给硅 mic 供电。如果供电的时候硅 mic 的偏置在 0.9v 以上，则此款硅 mic 不适用于 dac 跑低压的方案。

1.6 两个 IO 口推两个灯配置

在每个 board.c 文件里都有配置 pwmlled 参数的结构体，如下图所示：

```
237 /***** PWM_LED *****/
238 #if TCFG_PWMLED_ENABLE
239 LED_PLATFORM_DATA_BEGIN(pwm_led_data)
240     .led_pin = TCFG_PWMLED_PORT,
241     .led_cfg.led0_bright = TCFG_BLUELED_LIGHT,
242     .led_cfg.led1_bright = TCFG_REDLED_LIGHT,
243     .led_cfg.single_led_slow_freq = TCFG_SINGLE_SLOW_FLASH_FREQ,
244     .led_cfg.single_led_fast_freq = TCFG_SINGLE_FAST_FLASH_FREQ,
245     .led_cfg.double_led_slow_freq = TCFG_DOUBLE_SLOW_FLASH_FREQ,
246     .led_cfg.double_led_fast_freq = TCFG_DOUBLE_FAST_FLASH_FREQ,
247 //两个IO推两个灯配置
248     .two_io.two_io_mode_enable = 1,
249     .two_io.led0_pin = IO_PORTB_07,
250     .two_io.led1_pin = IO_PORTB_06,
251 LED_PLATFORM_DATA_END()
252 #endif
```

如果需要两个 IO 推灯应用，需要增加 two_io 结构配置，如上图红框内增加的配置；不需要两个 IO 推灯的应用，不需要增加 two_io 结构配置，板级配置不需要更改。

1.7 是否使用配置文件的参数配置

默认 sdk 是从配置文件读取配置来设置灯状态和提示音，如果不想从配置文件读取配置，只需修改

```
#define USE_CONFIG_BIN_FILE 0

#define USE_CONFIG_STATUS_SETTING 1
#define USE_CONFIG_AUDIO_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_CHARGE_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_KEY_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_PWMLED_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_MIC_TYPE_SETTING USE_CONFIG_BIN_FILE
```

成

```
#define USE_CONFIG_BIN_FILE 0

#define USE_CONFIG_STATUS_SETTING 0
#define USE_CONFIG_AUDIO_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_CHARGE_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_KEY_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_PWMLED_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_MIC_TYPE_SETTING USE_CONFIG_BIN_FILE
```

即可，然后默认配置在每个 board.c 里面设置

```
STATUS_CONFIG status_config = {
    //灯状态设置
    .led = {
        .charge_start = PWM_LED1_ON,
        .charge_full = PWM_LED0_ON,
        .power_on = PWM_LED0_ON,
        .power_off = PWM_LED1_FLASH_THREE,
        .lowpower = PWM_LED1_SLOW_FLASH,
        .max_vol = PWM_LED_NULL,
        .phone_in = PWM_LED_NULL,
        .phone_out = PWM_LED_NULL,
        .phone_activ = PWM_LED_NULL,
        .bt_init_ok = PWM_LED0_LED1_FAST_FLASH,
        .bt_connect_ok = PWM_LED0_SLOW_FLASH,
        .bt_disconnect = PWM_LED0_LED1_FAST_FLASH,
    },
    //提示音设置
    .tone = {
        .charge_start = IDEX_TONE_NONE,
        .charge_full = IDEX_TONE_NONE,
        .power_on = IDEX_TONE_POWER_ON,
        .power_off = IDEX_TONE_POWER_OFF,
        .lowpower = IDEX_TONE_LOW_POWER,
        .max_vol = IDEX_TONE_MAX_VOL,
        .phone_in = IDEX_TONE_NONE,
        .phone_out = IDEX_TONE_NONE,
        .phone_activ = IDEX_TONE_NONE,
        .bt_init_ok = IDEX_TONE_BT_MODE,
        .bt_connect_ok = IDEX_TONE_CONN,
        .bt_disconnect = IDEX_TONE_DISCONN,
    },
};
```

注意：如果想生成的 fw 文件能单独配置某些选项，要把对应的宏打开，如果想全部打开则设置

```
#define USE_CONFIG_BIN_FILE 1
```

即可

1.8 充电仓使用注意点

目前市面上充电仓芯片主要分为三种：

- 1、5v 输出常开
- 2、充满 5v 关闭
- 3、充满 5v 掉到 3v （如 Sy7628）

第一种，样机充满后进入 power off ，禁能下拉电阻整体功耗大概是 5uA

第二种，可以正常使用，但是这种充满电就断了 5v ，样机就不能做拔出开机快速连接

第三种，充满后，5v 变弱驱 3v ，充满后进入 power off，**功耗和维持电压及下拉电阻设置相关**

1.9 长按复位设置

新发出的 V0.0.3 以上版本的设置在如下路径...\\cpu\\br30\\tools 的 isd_config.ini 文件设置

硬件复位原理都是 IO 口检测对应的电平及持续的时间进行判断，从而复位芯片。方式一般有以下两种：

- 1，按键长按复位，默认配置为 **RESET=PB01_08_0**；即 PB1 检测到低电平持续 8s，就会复位
- 2，LDOIN 入仓复位，默认配置为 **RESET=LDOIN_02_1**；即耳机入仓之后 LDOIN 引脚检测到高电平持续 2s,就会复位。（正常运行的耳机入仓会将复位检测电平重新设置为低电平，出仓之后才设置为 高电平，所以不用担心正常样机入仓也会复位的情况，死机的样机入仓不会重新设置检测电平所以会复位）

```
*isd_config.ini - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
##### UBOOT配置项, 请勿随意调整顺序 #####
[SYS_CFG_PARAM]
#data_width[0 1 2 3 4] 3的时候uboot自动识别2或者4线
#clk [0-255]
#mode:
#
# 0 RD_OUTPUT,          1 cmd          1 addr
# 1 RD_I/O,             1 cmd          x addr
# 2 RD_I/O_CONTINUE] no_send_cmd x add
#port:
#
# 0 优先选A端口 CS:PD3 CLK:PD0 D0:PD1 D1:PD2 D2:PB7 D3:PD5
# 1 优先选B端口 CS:PA13 CLK:PD0 D0:PD1 D1:PA14 D2:PA15 D3:PD5
SPI=2_3_0_0; #data_clk_mode_port;
#OSC=btosc;
#OSC_FREQ=12MHz; #[24MHz 12MHz]
#SYS_CLK=24MHz; #[48MHz 24MHz]
UTTX=PA05; //uboot串口tx
UTBD=1000000; //uboot串口波特率
#UTRX=PB01; 串口升级[PB00 PB05 PA05]
#硬件复位可选择port口或LDOIN复位, LDOIN复位: 死机进仓就可以复位
RESET=PB01_08_0; //port口_长按时间_有效电平 (长按时间有00、01、02、04、08、16六个值可选, 单位为秒, 当长按时间为00时, 则关闭长按复位功能。)
#RESET=LDOIN_02_1; //LDOIN口_入仓时间_有效电平 (入仓时间有00、01、02、04、08、16六个值可选, 单位为秒, 当入仓时间为00时, 则关闭长按复位功能。)

#0:disable
#1:PA9 PA10
#2:USB
#3:PB1 PB2
#4:PB6 PB7

#sdtap=2;
<
```

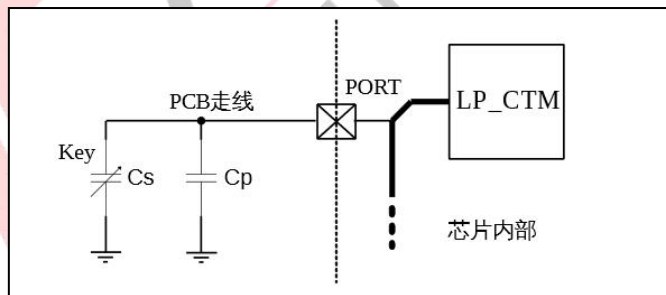
复位类型只能选一种

长按时间有 00、04、08、16 可选, 其他值默认选 08

1.10 内置触摸按键配置

1、关于内置触摸按键感应原理

内置触摸按键检测模块是利用人体分布电容对触摸按键电容影响来进行按键检测的, 如下图:



当人体触摸外部电容按键时, IO 口外部电容增加, 芯片内部的触摸感应模块可以检测到该电容变化, 从而检测到按键是否被按下。在没有触摸的时候, PCB 板走线和其他寄生电容组成了触摸按键固有电容 C_p , 当按键被触摸时, 电容增加了 C_s , 当 C_s/C_p 越大时, 触摸按键检测越灵敏, PCB layout 建议尽可能减少 C_p , 增加 C_s , 以增加触摸按键灵敏度。

2、关于触摸按键硬件布板的建议

1) 按键离芯片的距离。离芯片越近的按键, 其触摸效果越好, 反之则越差。因此用户在 PCB 布局的时候, 尽量将芯片放置在距离芯片最近的位置。

2) 按键至芯片的连线线宽。按键至芯片走线越细, 触摸效果越好, 反之则越差。因此尽量使按键至芯片之间连线更细。

3) 按键至芯片的连线和其它信号线（包括地线）的距离。距离越远，则其它信号线对触摸按键的影响越小，建议触摸按键至芯片的连线尽量远离其它信号线。

4) 触摸按键和面板的接触面积。面积越大、接触越紧密，触摸效果越好，反之越差。

5) 触摸面板的材质和厚度。面板越薄，触摸效果越好，反之越差。用玻璃、微晶板等材质做成的面板，其触摸效果要比用塑料、有机玻璃等材质做成的面板好。而金属材质的面板无法检测触摸按键。

（上面所说的触摸效果/灵敏度，本质是外部电容）

3、关于触摸按键软件配置介绍

1) 触摸按键配置结构体

在每个板级配置文件（board_ac697x_tws.c 和 board_ac697x_tws_cfg.h）里面配置触摸模块相关配置

```
board_ac697x_tws_cfg.h
//***** lp touch key 配置 *****
//***** lp touch key 配置 *****
//***** lp touch key 配置 *****
#define TCFG_LP_TOUCH_KEY_ENABLE ENABLE_THIS_MODULE //是否使能触摸按键
//电容检测灵敏度级数配置(范围: 0 ~ 9)
//该参数配置与触摸时电容变化量有关，触摸时电容变化量跟模具厚度，触摸片材质，面积等有关，
//触摸时电容变化量越小，推荐选择灵敏度级数越大，
//触摸时电容变化量越大，推荐选择灵敏度级数越小，
//用户可以从灵敏度级数为0开始调试，级数逐渐增大，直到选择一个合适的灵敏度配置值。
#define TCFG_LP_TOUCH_KEY_SENSITIVITY 5 //触摸按键电容检测灵敏度配置(级数0 ~ 9)
```

board_ac697x_tws.c

```
/****** LP TOUCH KEY ******/
#if TCFG_LP_TOUCH_KEY_ENABLE
const struct lp_touch_key_platform_data lp_touch_key_config = {
    //CH0: POWER KEY
    .ch0.enable = 1,
    .ch0.port = IO_PORTB_01,
    .ch0.sensitivity = TCFG_LP_TOUCH_KEY_SENSITIVITY, //cap检测灵敏度级数5
    .ch0.key_value = 0,
};
#endif /* #if TCFG_LP_TOUCH_KEY_ENABLE */
```

结构体配置说明：

enable	触摸模块开关，enable = 1 和宏 TCFG_LP_TOUCH_KEY_ENABLE 同时打开使能该模块；
port	触摸模块检测 IO 固定为 PB01，不能更改；
sensitivity	触摸按键模块电容灵敏度级数配置，该参数可以通过配置工具配置；
key_value	触摸按键键值，该值会对应 key_table 按键消息表，但按键被检测到按下时，会响应对

	应消息 <pre> u8 key_table[KEY_NUM_MAX][KEY_EVENT_MAX] = { // SHORT LONG HOLD UP DOUBLE TRIPLE {KEY_MUSIC_PP, KEY_POWEROFF, KEY_POWEROFF_HOLD, KEY_NULL, KEY_CALL_LAST_NO, KEY_LOW_LATENCY}, // {KEY_MUSIC_NEXT, KEY_VOL_UP, KEY_VOL_UP, KEY_NULL, KEY_OPEN_SIRI, KEY_NULL}, //KEY_1 {KEY_MUSIC_PREV, KEY_VOL_DOWN, KEY_VOL_DOWN, KEY_NULL, KEY_HID_CONTROL, KEY_NULL}, //KEY_2 {KEY_EAR_IN, KEY_EAR_OUT, KEY_NULL, KEY_NULL, KEY_NULL, KEY_NULL}, //KEY_3 }; </pre> 触摸按键默认键值为 0，可以检测单击，长按，HOLD，长按释放，双击和三击按键操作类型，用户可以修改不同按键操作类型对应的响应事件；
--	--

4、关于灵敏度参数配置

灵敏度参数，即对应结构体的 sensitivity 变量，该参数表示的是触摸模块检测的电容变化量灵敏度，范围为 0~9 级，级数数值越大，电容变化很小时也可以检测到，级数数值越小，电容变化很大时才可以检测到，值得注意的是对于某一个固定的硬件（包含触摸片和触摸面板），触摸时电容变化量一般固定，因此用户调试时可以从级数最小值 0 开始调试，级数配置值逐渐增加，直到得到满足需求的灵敏度级数。

该参数配置方式有两个：

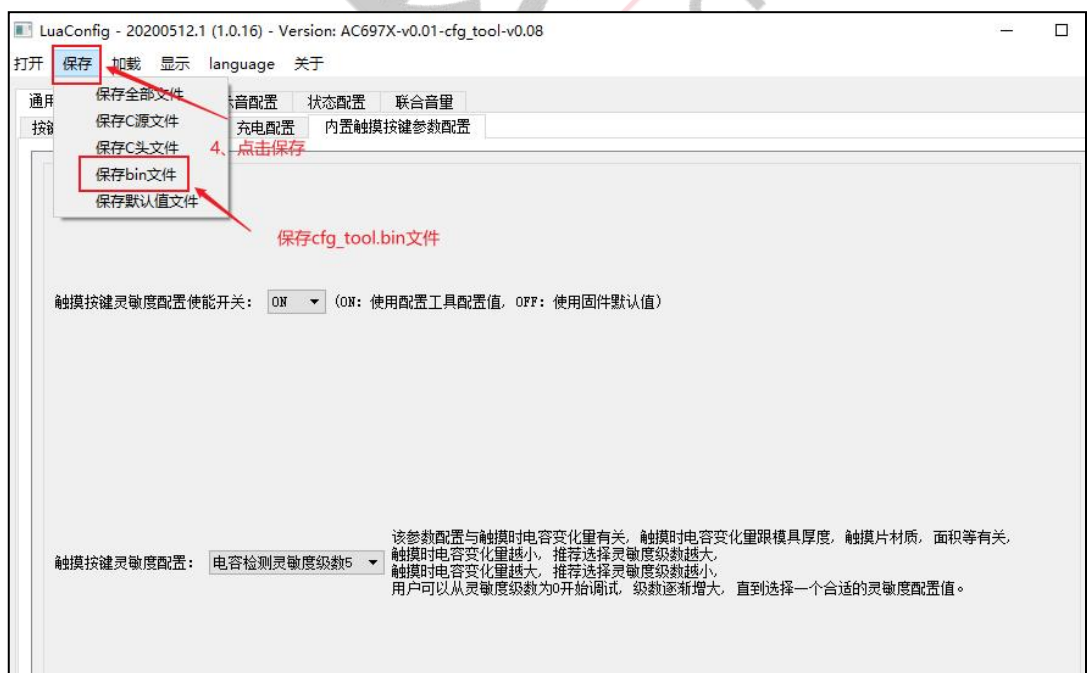
1) 通过在结构体中配置（文件 board_ac697x_tws_cfg.h）

```

//*****
//                                lp touch key 配置                                //
//*****
#define TCFG_LP_TOUCH_KEY_ENABLE          ENABLE_THIS_MODULE          //是否使能触摸按键
//电容检测灵敏度级数配置(范围: 0 ~ 9)
//该参数配置与触摸时电容变化量有关，触摸时电容变化量跟模具厚度，触摸片材质，面积等有关，
//触摸时电容变化量越小，推荐选择灵敏度级数越大，
//触摸时电容变化量越大，推荐选择灵敏度级数越小，
//用户可以从灵敏度级数为0开始调试，级数逐渐增大，直到选择一个合适的灵敏度配置值。
#define TCFG_LP_TOUCH_KEY_SENSITIVITY      5          //触摸按键电容检测灵敏度配置(级数0 ~ 9)

```

2) 通过配置工具配置

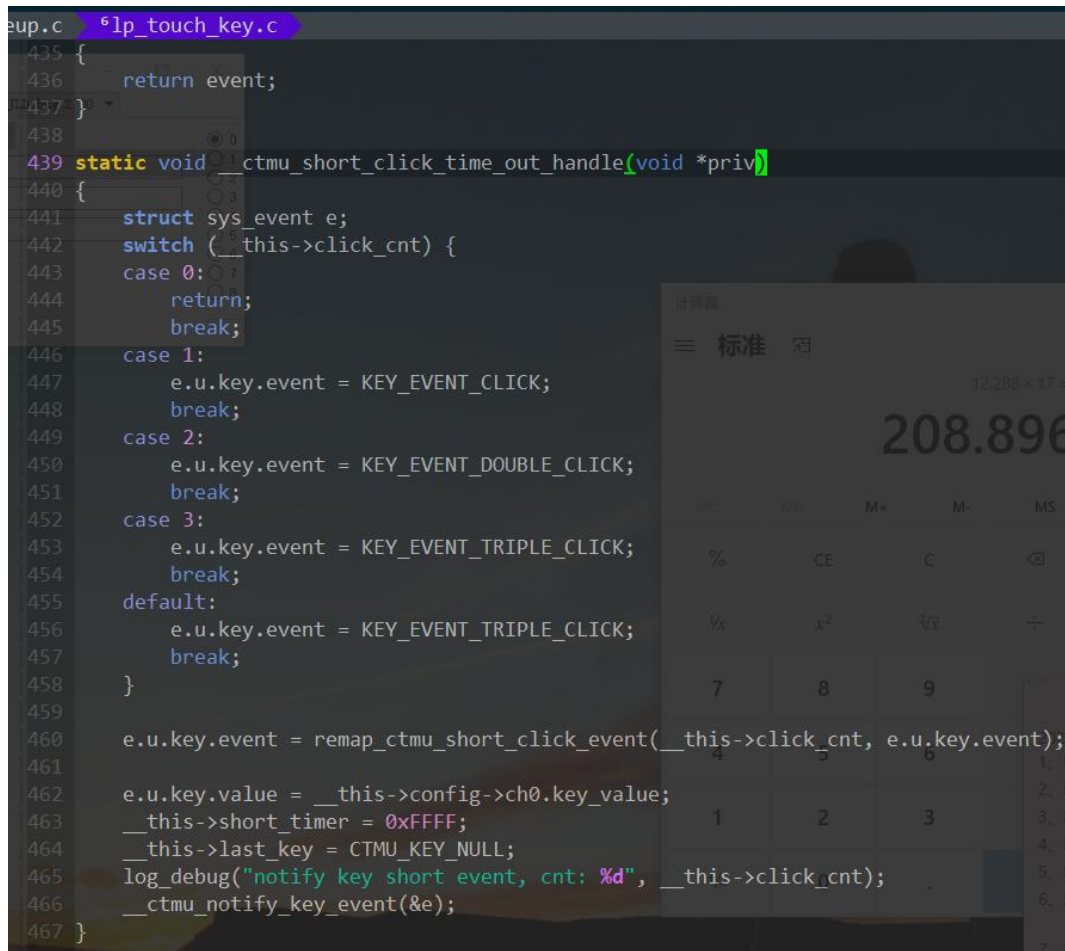


保存 cfg_tool.bin 文件后，升级固件，检测触摸按键灵敏度是否满足需求。

5、关于支持触摸按键多击的修改

内置触摸按键默认支持最多连续 3 击的事件判断（实际操作超过三击按照三击事件处理），如果用户需要支持更多击得事件处理，可以通过两种方式修改。

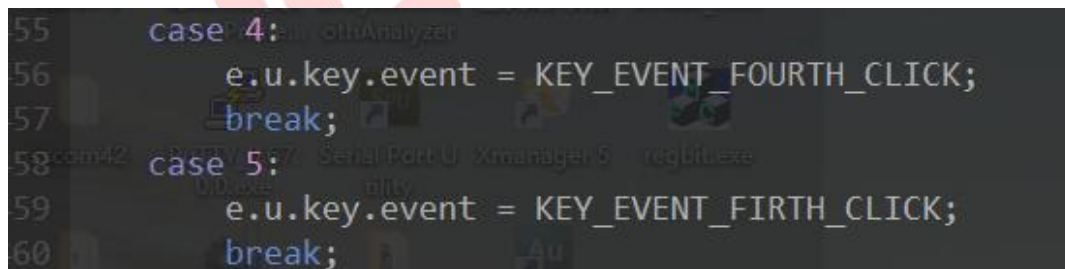
方法 1: 修改 lp_touch_key.c 文件



```

435 {
436     return event;
437 }
438
439 static void ctmu_short_click_time_out_handle(void *priv)
440 {
441     struct sys_event e;
442     switch (__this->click_cnt) {
443     case 0:
444         return;
445         break;
446     case 1:
447         e.u.key.event = KEY_EVENT_CLICK;
448         break;
449     case 2:
450         e.u.key.event = KEY_EVENT_DOUBLE_CLICK;
451         break;
452     case 3:
453         e.u.key.event = KEY_EVENT_TRIPLE_CLICK;
454         break;
455     default:
456         e.u.key.event = KEY_EVENT_TRIPLE_CLICK;
457         break;
458     }
459
460     e.u.key.event = remap_ctmu_short_click_event(__this->click_cnt, e.u.key.event);
461
462     e.u.key.value = __this->config->ch0.key_value;
463     __this->short_timer = 0xFFFF;
464     __this->last_key = CTMU_KEY_NULL;
465     log_debug("notify key short event, cnt: %d", __this->click_cnt);
466     __ctmu_notify_key_event(&e);
467 }
    
```

变量 `__this->click_cnt` 就是内置触摸模块检测到得多击次数，用户可增加 `KEY_EVENT_FOURTH_CLICK`, `KEY_EVENT_FIRTH_CLICK`（改定义在 `event.h` 文件）或者更多击事件类型；



```

55 case 4:
56     e.u.key.event = KEY_EVENT_FOURTH_CLICK;
57     break;
58 case 5:
59     e.u.key.event = KEY_EVENT_FIRTH_CLICK;
60     break;
    
```

方法 2: 在各自得板级配置文件里面给用户预留的按键事件处理函数，该方法比较适合同一 SDK 多个板级同时开发的情况，例如在板级配置文件 `board_ac8972a_demo.c` 添加自定义函数：

```

116 #if TCFG_LP_TOUCH_KEY_ENABLE do Stop Upload ReadDev disconnect GG_JLDebu
117 u8 remap_ctmu_short_click_event(u8 click_cnt, u8 event)
118 {
119     u8 key_event = event;
120     switch (click_cnt) {
121     case 0:
122         break;
123     case 1:
124         key_event = KEY_EVENT_CLICK;
125         break;
126     case 2:
127         key_event = KEY_EVENT_DOUBLE_CLICK;
128         break;
129     case 3:
130         key_event = KEY_EVENT_TRIPLE_CLICK;
131         break;
132     case 4:
133         key_event = KEY_EVENT_FOURTH_CLICK;
134         break;
135     case 5:
136         key_event = KEY_EVENT_FIRTH_CLICK;
137         break;
138     default:
139         key_event = KEY_EVENT_FIRTH_CLICK;
140         break;
141     }
142     return key_event;
143 }
144 }
145 #endif

```

通过这两种方法中的一个可以实现用户任意多击的事件发送，最后用户可以在对应板级文件，例如 board_ac8972a_demo.c 的按键事件表填写增肌对应多击的响应的事件即可：

```

71 /****** KEY MSG******/
72 /*各个按键的消息设置，如果USER_CFG中设置了USE_CONFIG_KEY_SETTING为1，则会从配置文件读取对应的配置来填充改结构体*/
73 u8 key_table[KEY_NUM_MAX][KEY_EVENT_MAX] = {
74     // SHORT LONG HOLD UP DOUBLE TRIPLE FOURTH FIRTH //KEY_0
75     {KEY_MUSIC_PP, KEY_POWEROFF, KEY_POWEROFF_HOLD, KEY_NULL, KEY_CALL_LAST_NO, KEY_LOW_LANTECY, KEY_NULL, KEY_NULL},
76 };
77

```

6、关于按键事件操作的一些时间配置

在文件 lp_touch_key.h 有如下时间配置：

```

8 #define CTMU_SHORT_CLICK_DELAY_TIME 400 //单击事件后等待下一次单击时间(ms)
9 #define CTMU_LONG_CLICK_DELAY_TIME 1000 //产生长按事件的触摸时间(ms)
10 #define CTMU_HOLD_CLICK_DELAY_TIME 200 //long事件产生后，发hold事件间隔(ms)
11
12 //长按开机时间:
13 #define CFG_M2P_CTMU_CH0_SOFTOFF_LONG_TIME 5 //(n + 6) * 100(ms)
14

```

1) CTMU_SHORT_CLICK_DELAY_TIME: 该配置是指在单击按键抬起后等待下一次单击的时间, 该配置用于任意多击统计, 单位: ms;

2) CTMU_LONG_CLICK_DELAY_TIME: 该配置是指产生长按事件的触摸时间, 单位: ms;

3) CTMU_HOLD_CLICK_DELAY_TIME: 该配置是指持续触摸按键不放超过长按触摸时间 (CTMU_LONG_CLICK_DELAY_TIME) 后每隔该配置时间会发送一个 HOLD 事件, 单位: ms;

4) CFG_M2P_CTMU_CH0_SOFTOFF_LONG_TIME: 该配置是指进关机之后需要长按开机的时间, 时间理论计算是 (配置值 + 6) X 100, 单位: ms, 值得注意的是该时间是指内置触摸模块检测到长按的时间, 系统开机时间 = 配置值时间 + 系统上电时间;

7、关于触摸模块动态关闭接口

在某些情况下, 用户希望不产生触摸按键事件, 用户可调用接口动态打开和关闭触摸模块。

```
51 void lp_touch_key_disable(void); //关闭模块
52 void lp_touch_key_enable(void); //打开模块
```

1) 关闭触摸模块, 可调用 lp_touch_key_disable()接口;

2) 打开触摸模块, 可调用 lp_touch_key_enable()接口;

1.11 AC700N 输出 EVDD 电源说明

1、EVDD 电源介绍

AC700N 内置一个可供外部 DSP 供电的 EVDD 电源, 有如下特性:

- A. 驱动能力 5mA 1.0 ~ 1.2v, 挡位可调;
- B. 固定从 PB3 IO 输出, 输出时需要将 IO 口设置为高阻状态;
- C. 在低功耗模式会默认关闭, 如果需要在低功耗模式下保持, 需要注册低功耗回调函数, 进入 LOW_POWER_MODE_LIGHT_SLEEP 模式;
- D. 暂不支持 softoff 状态保持;

2、evdd 使用例程

```
//=====//
//      关于输出 EVDD 电源到 PB3 配置      //
//=====//

//1.包含头文件
#include "asm/power/p33.h"

//2. 调用接口输出 evdd 电源

/* void evdd_output_2_pb3(u8 en, u8 vol_sel);
```

@input:

en: 1->打开 evdd 电源, 0->关闭 evdd 电源;

vol_sel: 选择 evdd 挡位;

EVDD 挡位:

EVD_VOL_SEL_100V

EVD_VOL_SEL_105V

EVD_VOL_SEL_110V

EVD_VOL_SEL_115V

@return: void

@note: evdd 属于模拟电源, 固定从 PB3 IO 口输出, 需要将 PB3 配置为高阻状态 */

void evdd_output_open()

```
{
    //把 PB3 设置为高阻状态:
    gpio_set_pull_up(IO_PORTB_03, 0);
    gpio_set_pull_down(IO_PORTB_03, 0);
    gpio_set_die(IO_PORTB_03, 0);
    gpio_set_dieh(IO_PORTB_03, 0);
    gpio_set_direction(IO_PORTB_03, 1);
    //输出 evdd 电源:
    evdd_output_2_pb3(1, EVD_VOL_SEL_110V);
}
```

//3. 把 evdd 关闭

void evdd_output_close()

```
{
    evdd_output_2_pb3(0, EVD_VOL_SEL_110V);
}
```

//4. 如果需要 evdd 电源在低功耗保持, 需要注册函数低功耗回调函数

static u8 evdd_idle_query(void)

```
{
    return 1;
}
```

static enum LOW_POWER_LEVEL evdd_level_query(void)

```
{
```



```
    return LOW_POWER_MODE_LIGHT_SLEEP;
}

REGISTER_LP_TARGET(evdd_lp_target) = {
    .name = "evdd",
    .level = evdd_level_query,
    .is_idle = evdd_idle_query,
};
```

Chapter 2 VM 接口的使用

2.1 编写目的

该文档主要描述 AC700N_SDK VM 使用方法及开发中注意的一些问题，为用户进行二次开发提供参考

2.2 详细说明

Vm 读写接口：

读接口：

```
int syscfg_read(u16 item_id, void *buf, u16 len);
```

写接口：

```
int syscfg_write(u16 item_id, void *buf, u16 len);
```

参数说明：

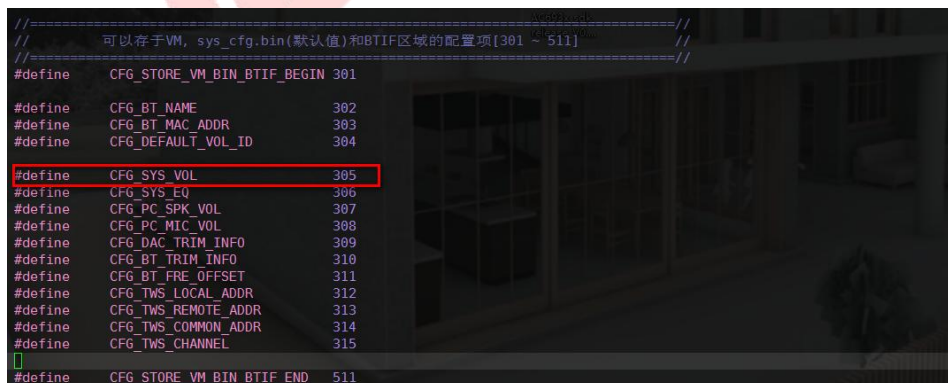
item_id: vm id 取值范围 301~511

*buf: 读写指针

Len: 长度

比如保存音量和读音量的 vm 操作：

1、确定 vm id 号



```
//-----  
// 可以存于VM, sys_cfg.bin(默认值)和BTIF区域的配置项[301 ~ 511]  
//-----  
#define CFG_STORE_VM_BIN_BTIF_BEGIN 301  
  
#define CFG_BT_NAME 302  
#define CFG_BT_MAC_ADDR 303  
#define CFG_DEFAULT_VOL_ID 304  
  
#define CFG_SYS_VOL 305  
#define CFG_SYS_EQ 306  
#define CFG_PC_SPK_VOL 307  
#define CFG_PC_MIC_VOL 308  
#define CFG_DAC_TRIM_INFO 309  
#define CFG_BT_TRIM_INFO 310  
#define CFG_BT_FRE_OFFSET 311  
#define CFG_TWS_LOCAL_ADDR 312  
#define CFG_TWS_REMOTE_ADDR 313  
#define CFG_TWS_COMMON_ADDR 314  
#define CFG_TWS_CHANNEL 315  
  
#define CFG_STORE_VM_BIN_BTIF_END 511
```

2、写音量


```
log_info(ACTION_POWEROFF_HOOK);  
if (app_var.goto_poweroff_flag) {  
    syscfg_write(CFG_SYS_VOL, &app_var.sys_vol_l, 1);  
    tone_play(TONE_POWER_OFF);  
    os_taskq_flush();  
    STATUS *p_tone = get_tone_config();  
    tone_play_index(p_tone->power_off);  
}
```

3、读音量

```
}  
#if SYS_DEFAULT_VOL  
    app_var.sys_vol_l = audio_cfg.default_vol;  
#else  
    syscfg_read(CFG_SYS_VOL, &app_var.sys_vol_l, 1);  
#endif
```

Chapter 3 通话效果调试

3.1 编写目的

该文档主要描述 AC700N_SDK 通话效果调试的使用方法及开发中注意的一些问题，为用户进行二次开发提供参考。

3.2 工具配置说明

Aec 参数配置和以前系列的配置一样。Mic 类型配置要开

```
#define USE_CONFIG_BIN_FILE 0
#define USE_CONFIG_STATUS_SETTING 1
#define USE_CONFIG_AUDIO_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_CHARGE_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_KEY_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_PWMLED_SETTING USE_CONFIG_BIN_FILE
#define USE_CONFIG_MIC_TYPE_SETTING USE_CONFIG_BIN_FILE
```

对应的宏才能在工具配置

3.3 调试说明

具体调试方法，参考配置工具内置文档



Chapter 4 Audio 模块使用

4.1 编写目的

该文档主要描述 AC700N_SDK audio 模块的使用方法及开发中注意的一些问题，为用户进行二次开发提供参考。

4.2 详细说明

DAC 硬件输出参数配置：

.output:

- 1、DAC_OUTPUT_MONO_L : DACL 单端接地
- 2、DAC_OUTPUT_MONO_R : DACR 单端接地
- 3、DAC_OUTPUT_LR : DACL/R 共地
- 4、DAC_OUTPUT_MONO_LR_DIFF : DACL DACR 差分接

注意：需要根据具体的硬件接法，配置 **TCFG_AUDIO_DAC_CONNECT_MODE** 宏，**AC697N 不支持 VCMO**

配置例子如下：

```

//***** DAC *****/
#ifdef TCFG_AUDIO_DAC_ENABLE
struct dac_platform_data dac_data = {
    .ldo_volt      = TCFG_AUDIO_DAC_LDO_VOLT,           //DACVDD等级
    .vcmo_en       = 0,                                 //是否打开VCMO
    .output        = TCFG_AUDIO_DAC_CONNECT_MODE,       //DAC输出配置，和具体硬件连接有关，需根据硬件来>
    设置
    .ldo_isel      = 0,
    .ldo_fb_isel   = 0,
    .lpf_isel      = 0x2,
    .dsm_clk       = DAC_DSM_6MHz,
};
#endif

```

DAC 延时以及起始缓冲延时的配置说明：

1) 接口函数 : int audio_dac_set_delay_time(struct audio_dac_hdl *dac, int start_ms, int max_ms);

参数 dac: dac 的结构句柄

参数 start_ms : dac 被启动的缓冲延时（毫秒）

参数 `max_ms` : dac 的最大延时缓冲 (毫秒), 通常 A2DP、普通音频仅需要 30ms 的延时, 由于通话的采样率仅有 8k/16k, 且有 AEC 等算法需求, 因此需要尽可能多的延时来满足其访问音频数据。

2) 配置示例:

```
audio_dac_set_delay_time(&dac_hdl, 20, AUDIO_DAC_DELAY_TIME);
```

4.3 在线 EQ 调试

1 软件上修改流程方法:

例如使用 AC897N 的 `board_ac8976d_tws_cfg.c` 配置。在 `board_ac8976d_tws_cfg.h` 打开如下设置。

```

//***** EQ配置 *****//
//EQ配置, 使用在线EQ时, EQ文件和EQ模式无效。有EQ文件时, 默认不用EQ模式切换功能
#define TCFG_EQ_ENABLE 1 //支持EQ功能, EQ总使能
//if TCFG_EQ_ENABLE
//define TCFG_EQ_ONLINE_ENABLE 1 //支持在线EQ调试, 如果使用蓝牙串口调试, 需要打开宏 APP_ONLINE_DEBUG, 否则, 默认使用uart调试(二选一)
//define TCFG_BT_MUSIC_EQ_ENABLE 1 //支持蓝牙音乐EQ
//define TCFG_PHONE_EQ_ENABLE 1 //支持通话近端EQ
//define TCFG_AUDIO_OUT_EQ_ENABLE 0 //mix out级, 增加eq高低音接口
//define TCFG_AEC_DCCS_EQ_ENABLE 1 //AEC DCCS
//define TCFG_AEC_UL_EQ_ENABLE 1 //AEC UL
//define TCFG_USE_EQ_FILE 0 //是否使能默认系数表 1: 不使能 0:使能
#define EQ_SECTION_MAX 10 //eq 段数, 最大20
#endif//TCFG_EQ_ENABLE

/*省电容mic通过eq模块实现去直流滤波*/
#if (TCFG_SUPPORT_MIC_CAPLESS && (TCFG_MIC_CAPLESS_ENABLE || TCFG_MIC1_CAPLESS_ENABLE))
#if ((TCFG_EQ_ENABLE == 0) || (TCFG_AEC_DCCS_EQ_ENABLE == 0))
#error "MicCapless enable, Please enable TCFG_EQ_ENABLE and TCFG_AEC_DCCS_EQ_ENABLE"
#endif
#endif
//if TCFG_EQ_ONLINE_ENABLE && TCFG_TEST_BOX_ENABLE
//error "在线调试时, 请关闭TCFG_TEST_BOX_ENABLE"
//endif

```

1.2 通讯方式

使用串口通讯, 默认使用 usb DP DM 做调试口, 波特率 115200, 可根据实际使用改用别的 IO

```

// ONLINE CONFIG
// 如果调试串口是DP DM, 使用eq调试串口时, 需关闭usb宏
#define TCFG_ONLINE_ENABLE (TCFG_EQ_ONLINE_ENABLE) //是否支持EQ在线调试功能
#define TCFG_ONLINE_TX_PORT IO_PORT_DP //EQ调试TX口选择
#define TCFG_ONLINE_RX_PORT IO_PORT_DM //EQ调试RX口选择

```

在线调试时, 请关闭 `TCFG_TEST_BOX_ENABLE`, 以免串口被占用, 导致通讯失败

```

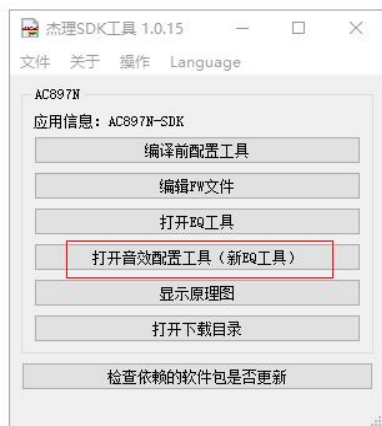
//***** 充电仓配置 *****//
//***** 充电仓配置 *****//
#define TCFG_CHARGESTORE_ENABLE ENABLE_THIS_MODULE //是否支持智能充点仓
#define TCFG_TEST_BOX_ENABLE 0 //是否支持ANC测试盒
#define TCFG_CHARGESTORE_BOX_ENABLE 0 //是否支持ANC测试盒
#define TCFG_CHARGESTORE_PORT IO_PORTP_00 //耳机和充点仓通讯的IO口
#define TCFG_CHARGESTORE_UART_ID IRQ_UART1_IDX //通讯使用的串口号

```

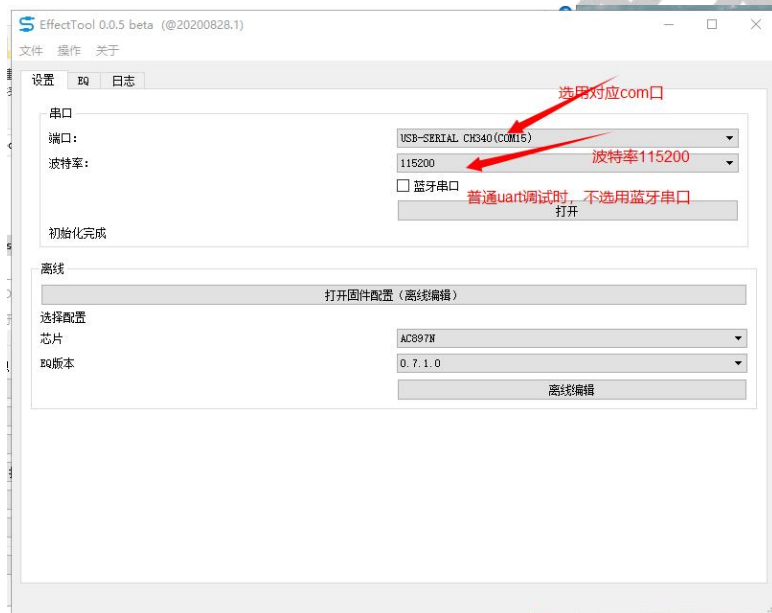
设置后编译, 下载到开发板或者样机就可以了。

2 PC 端设置流程。

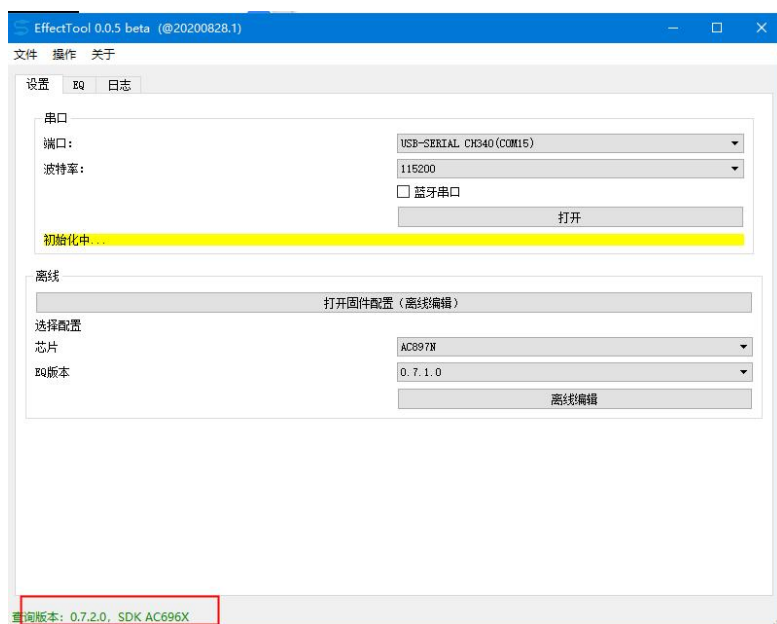
2.1 使用”杰理包管理器”软件打开，如下界面。选中新 EQ 工具



2.2 选用对应 COM 口，波特率 115200，普通 uart 调试时，不选蓝牙串口



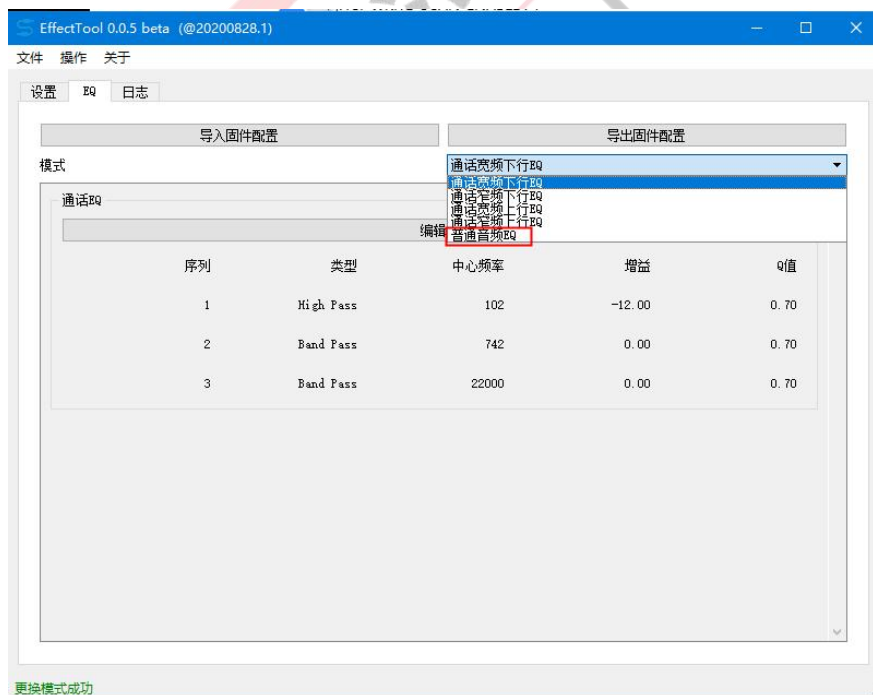
2.3 点击打开，通讯成功左下角会提示 查询版本



2.4 选中下拉列表，调试对应模式

下拉列表有：通话宽频下行 EQ、通话窄频下行 EQ、通话宽频上行 EQ、通话窄频下行 EQ、普通音频 EQ，五种模式。其中蓝牙、music、linein、fm、pc 模式播歌的 eq 对应普通音频 EQ。

例如：选中普通音频 EQ 后，点击编辑





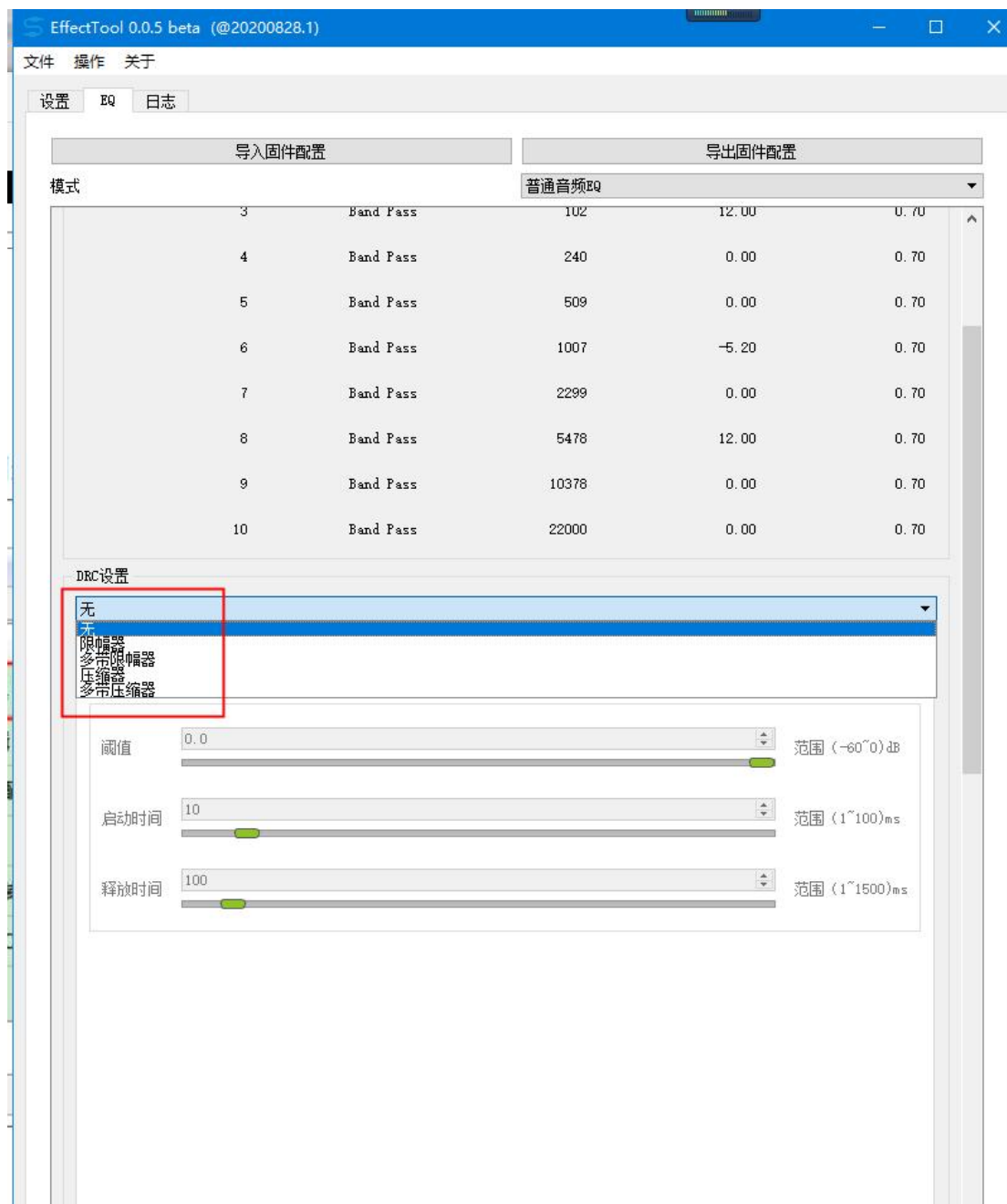


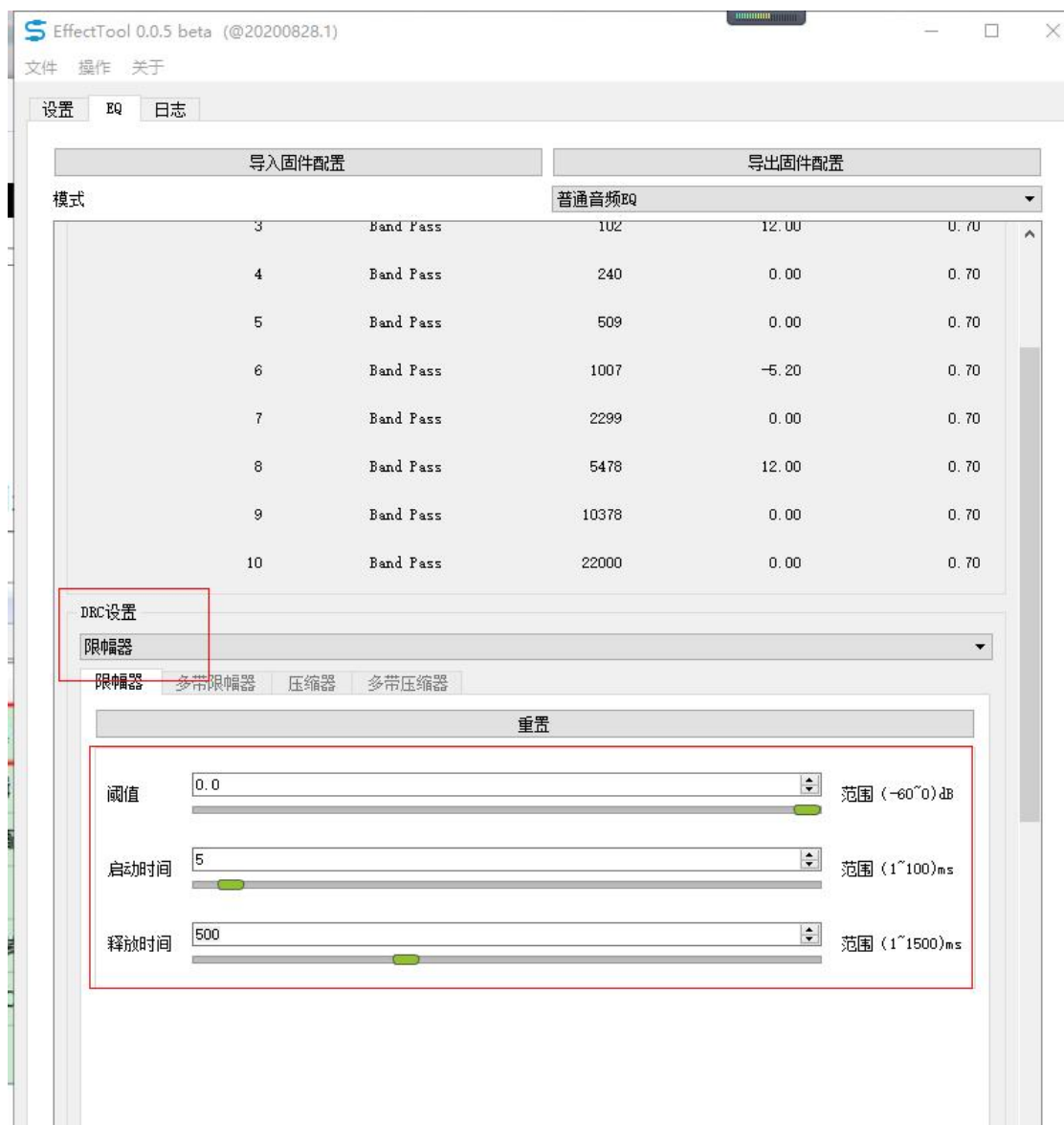
看到此界面，可以根据需对每一段 eq 的频率、增益、q 值以及滤波器类型进行调节，调节完毕点击 关闭



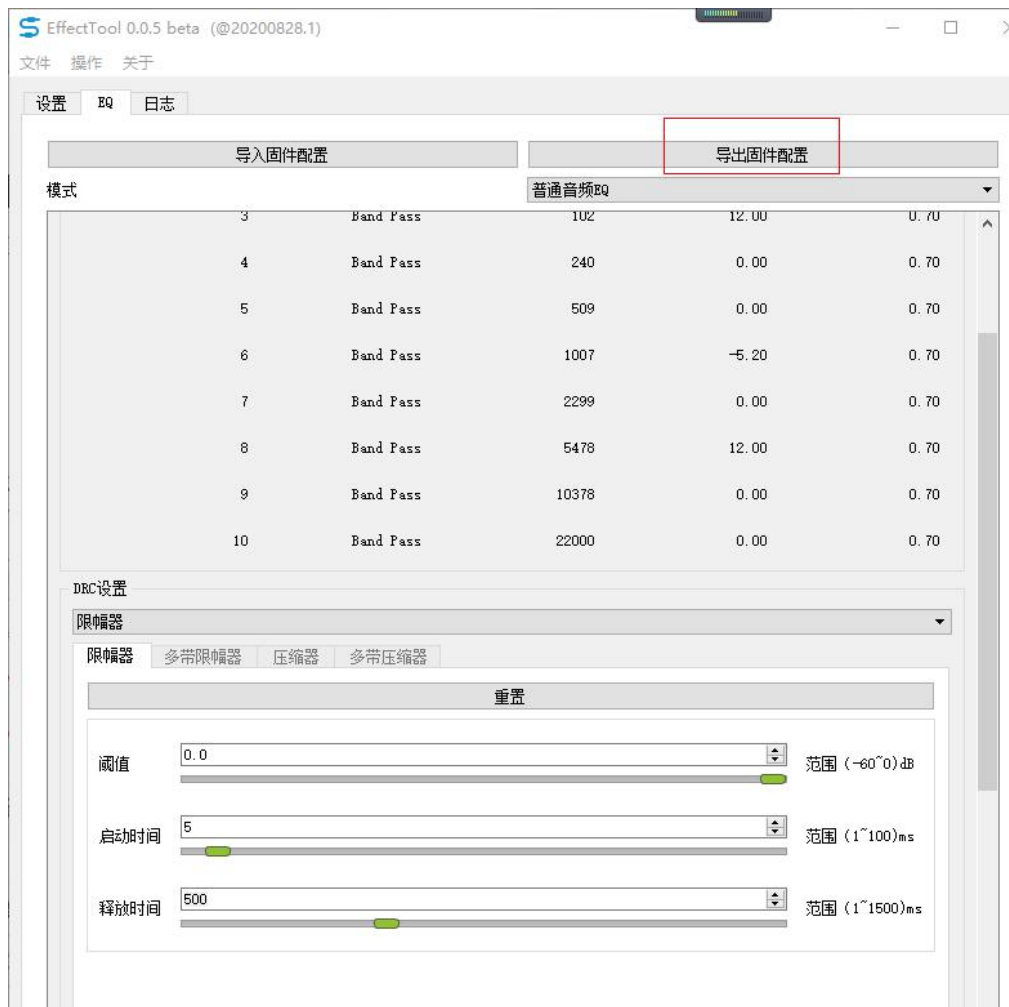
2.5 drc 限幅器、多带限幅器、压缩器、多带压缩器调节

选中 DRC 设置的下拉列表，例如选限幅器，进行设置。





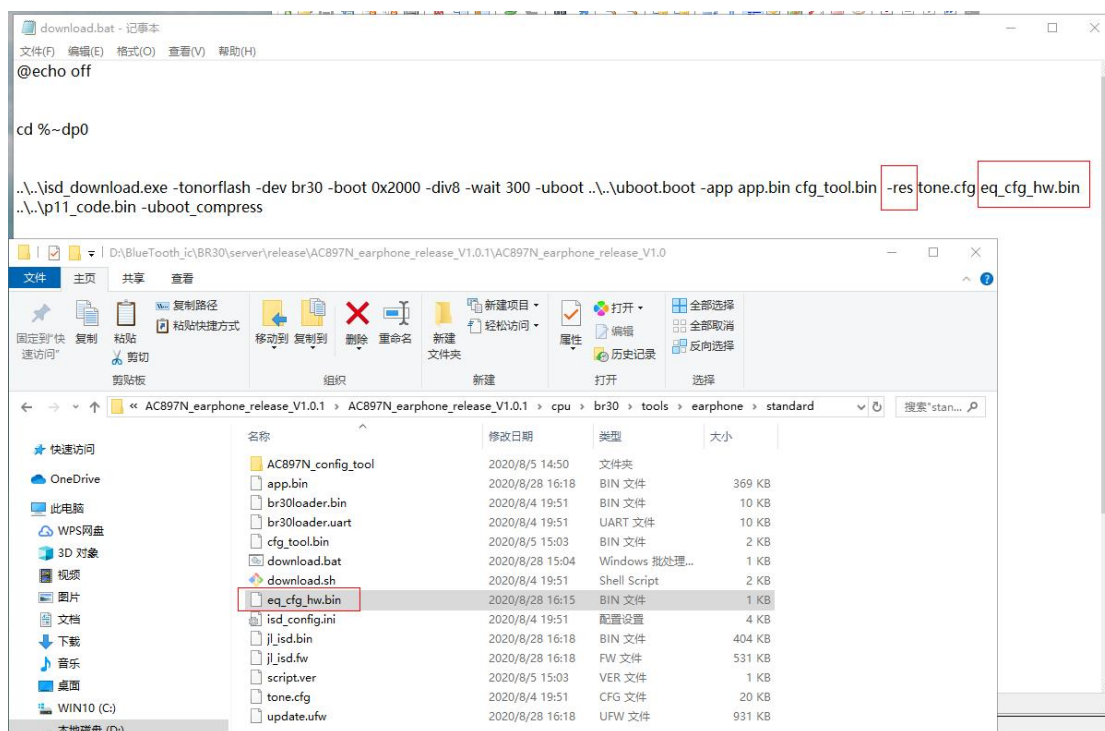
2.6 EQ 与 DRC 调试完成后，点击导出固件 eq_cfg_hw.bin 文件。



2.7 将 eq_hw_cfg.bin 文件放到下载目录，并在 download.bat 中-res 后添加 eq_hw_cfg.bin 文件。此外，调试完成需要将在线 eq 宏定义关闭,同时使能 使用 eq 文件宏定义



例如在 earphne 的 standard sdk 添加 eq_hw_cfg.bin



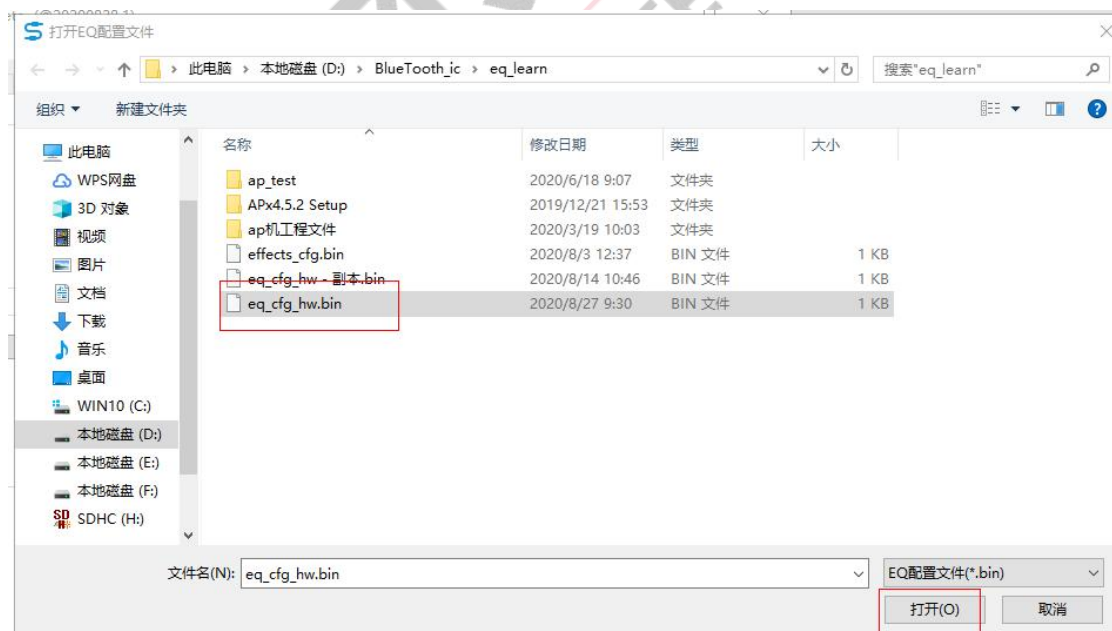
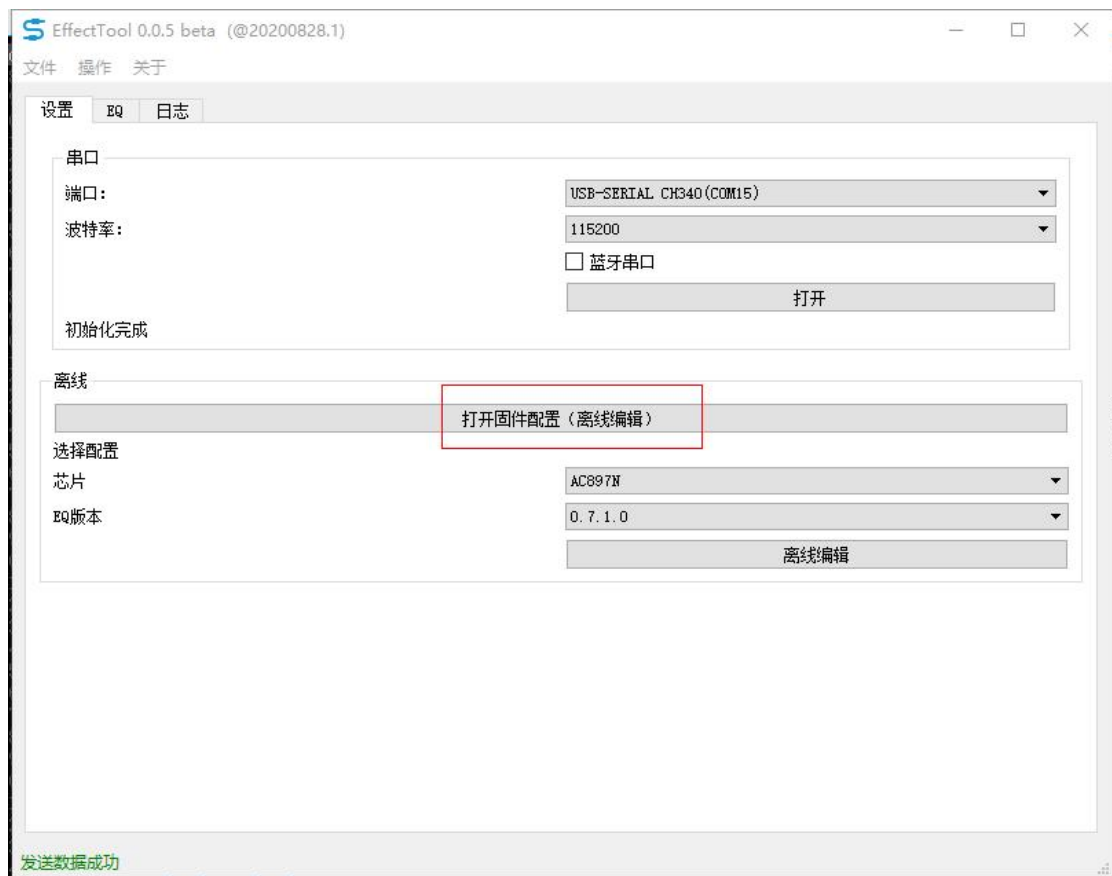
```
isd download.exe -tonorflash -dev br30 -boot 0x2000 -div8 -wait 300 -uboot uboot.boot -app app.bin cfg_tool.bin  
-res tone.cfg pll_code.bin eq_cfg_hw.bin -uboot_compress
```

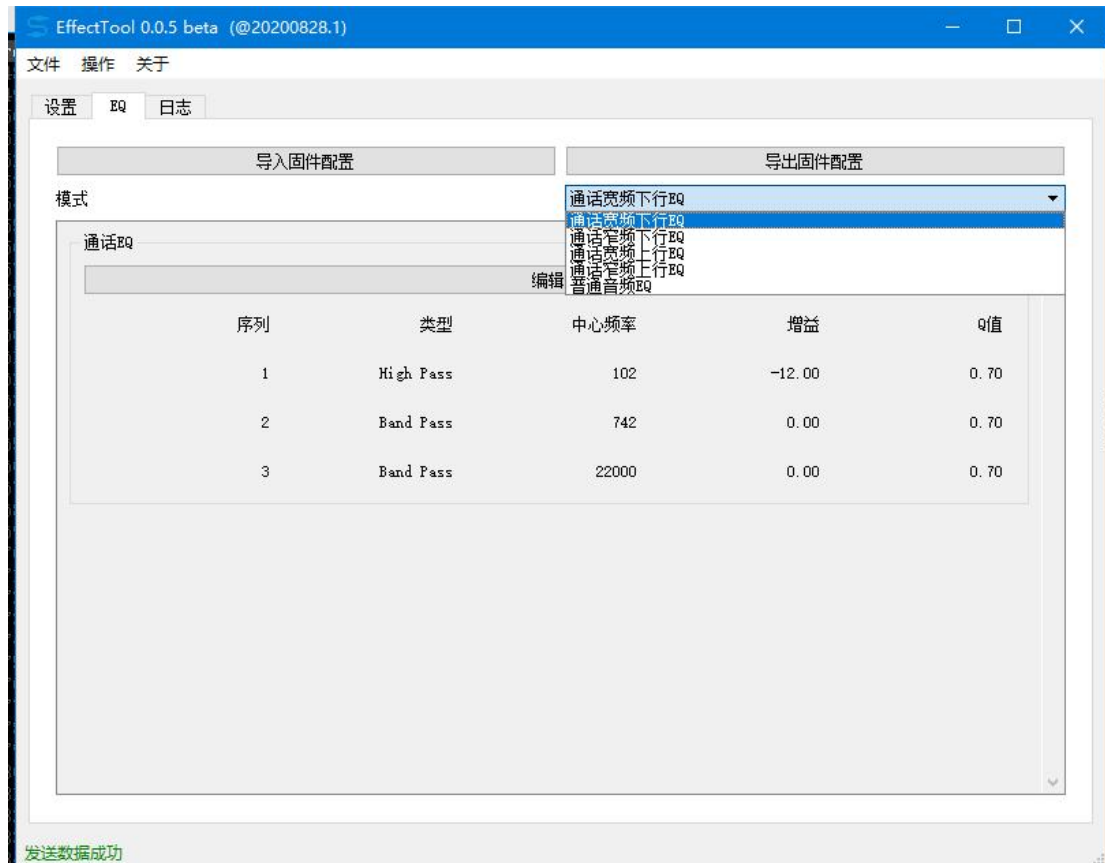
注意这里外部配置的资源一定要放在-res选项中

注意：外部的配置资源一定要放在-res 这个所属项里头，例如 eq_cfg_hw.bin 的资源，因为跟工具的识别项是同步的。

3 eq_hw_cfg.bin 文件重编辑

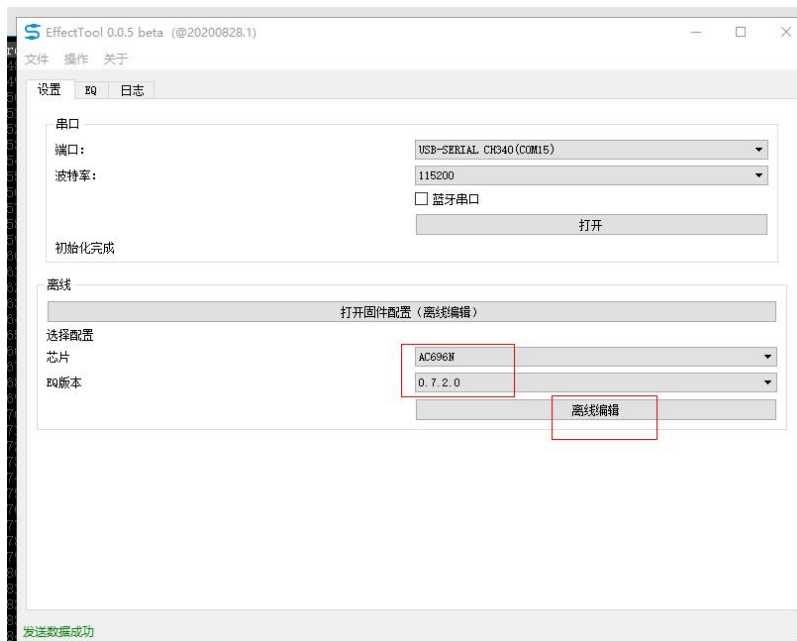
选中打开固件配置（离线编辑），选中 eq_hw_cfg.bin 文件，就会弹出与在线调试一致的界面，此时可以对已有的 eq_hw_cfg.bin 文件进行修改，修改完成，导出固件





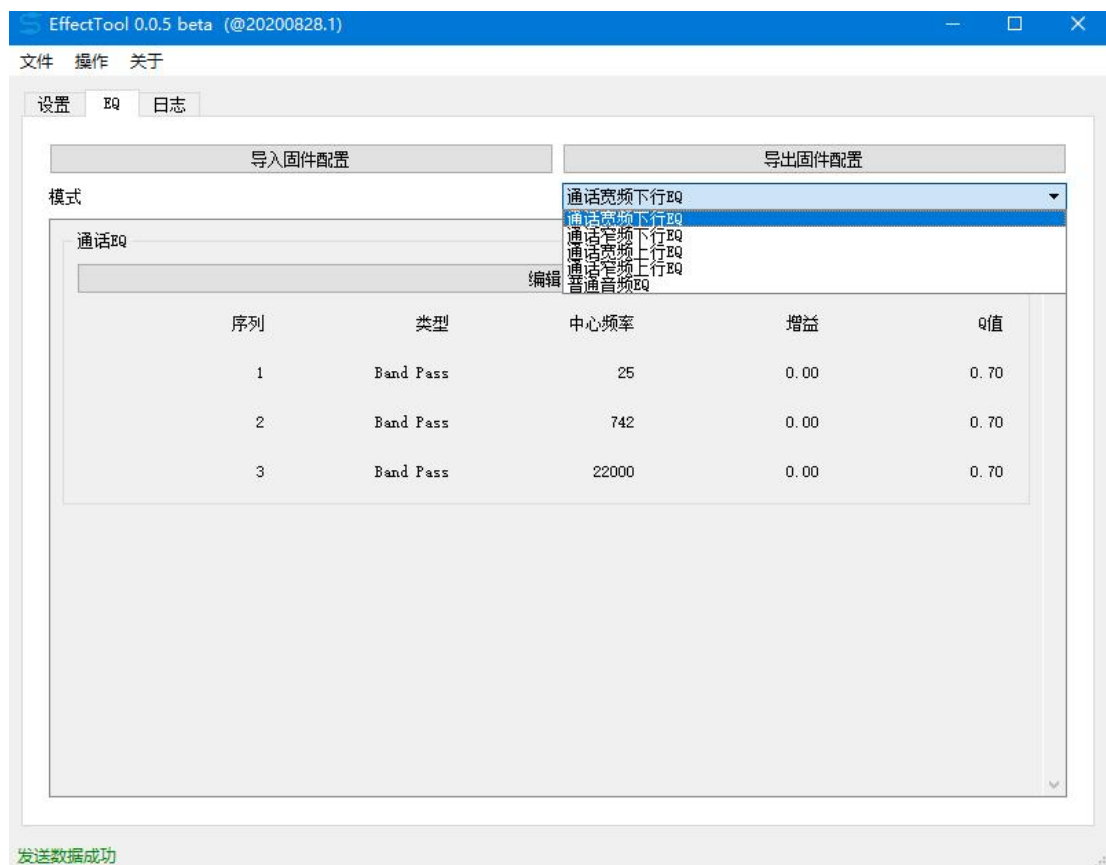
4 打开工具，eq 离线调试编辑

4.1 选中对应芯片版本、以及 eq 版本，点击离线编辑



4.2 弹出设置 eq 段数的界面，设置目标段数，点击确定，会弹出 eq 离线编辑界面。编辑完成导出固件





5 蓝牙串口调试 eq

通讯成功后，使用方式参照普通串口调试。此处说明，如何使用电脑蓝牙与小机进行连接配置。

5.1 软件修改

使用蓝牙串口调试 eq，sdk 需要使能宏定义 APP_ONLINE_DEBUG

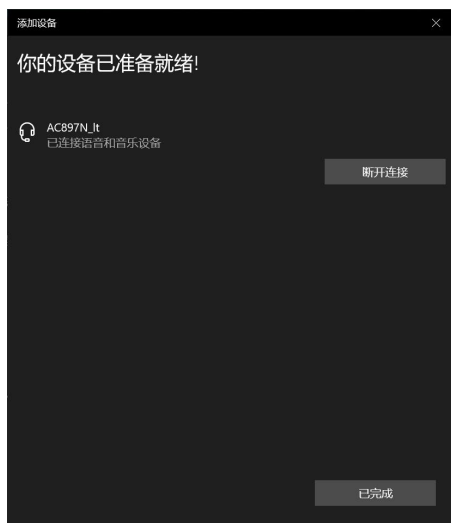
```

384
385 #define APP_ONLINE_DEBUG 1//在线APP调试, 发布默认不开
386 #if (APP_ONLINE_DEBUG && TCFG_ONLINE_ENABLE)//使能蓝牙串口在线调试时，关闭普通串口在线调试
387 #undef TCFG_ONLINE_ENABLE
388 #define TCFG_ONLINE_ENABLE 0
389 #endif
390

```

5.2 蓝牙串口连接说明

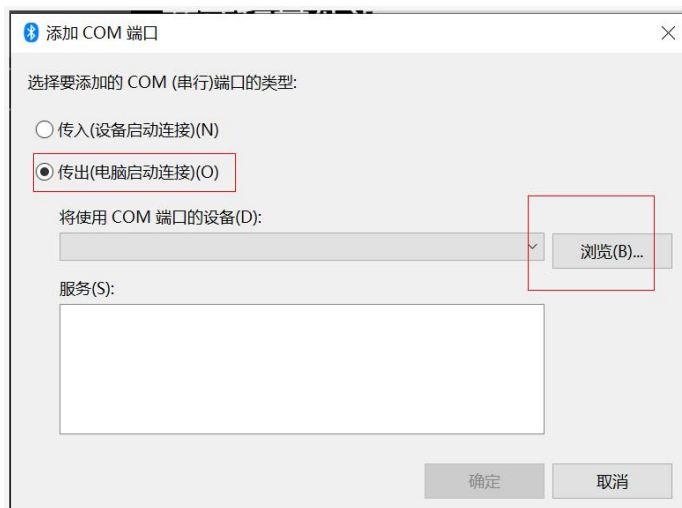
5.2.1 连上蓝牙



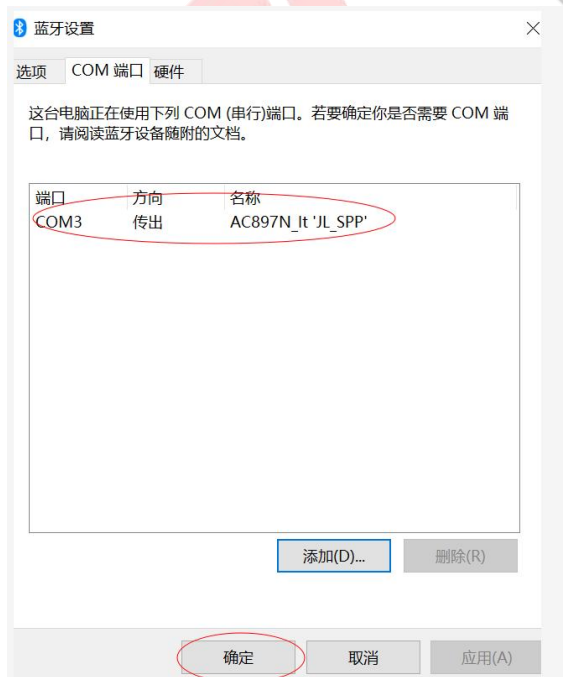
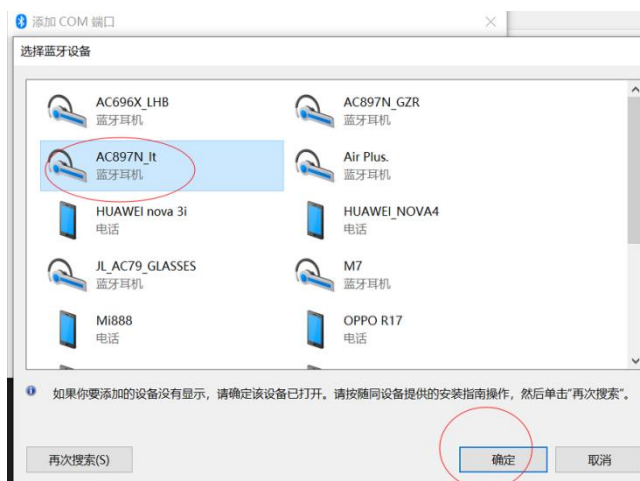
5.2.2 更多蓝牙配置



5.2.3 传出（电脑启动连接）

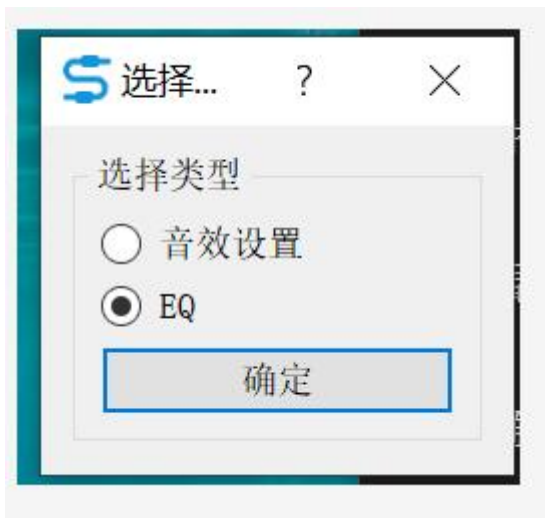


5. 2. 4 选中目标蓝牙名



5.2.5 打开 eq 工具

选 eq



5.2.6 选中蓝牙串口、选刚才传出的 com 口，波特率默认 115200



4.4 DAC 配置

4.4.1 DAC 高低压模式选择

AC700N 系列芯片**仅支持** DAC 低压模式，DACVDD 可选档位如下：

```
#define DACVDD_LDO_1_20V      0
#define DACVDD_LDO_1_25V      1
#define DACVDD_LDO_1_30V      2
#define DACVDD_LDO_1_35V      3
```

4.4.2 DAC 输出幅度

DAC 输出幅度和以下配置有关：

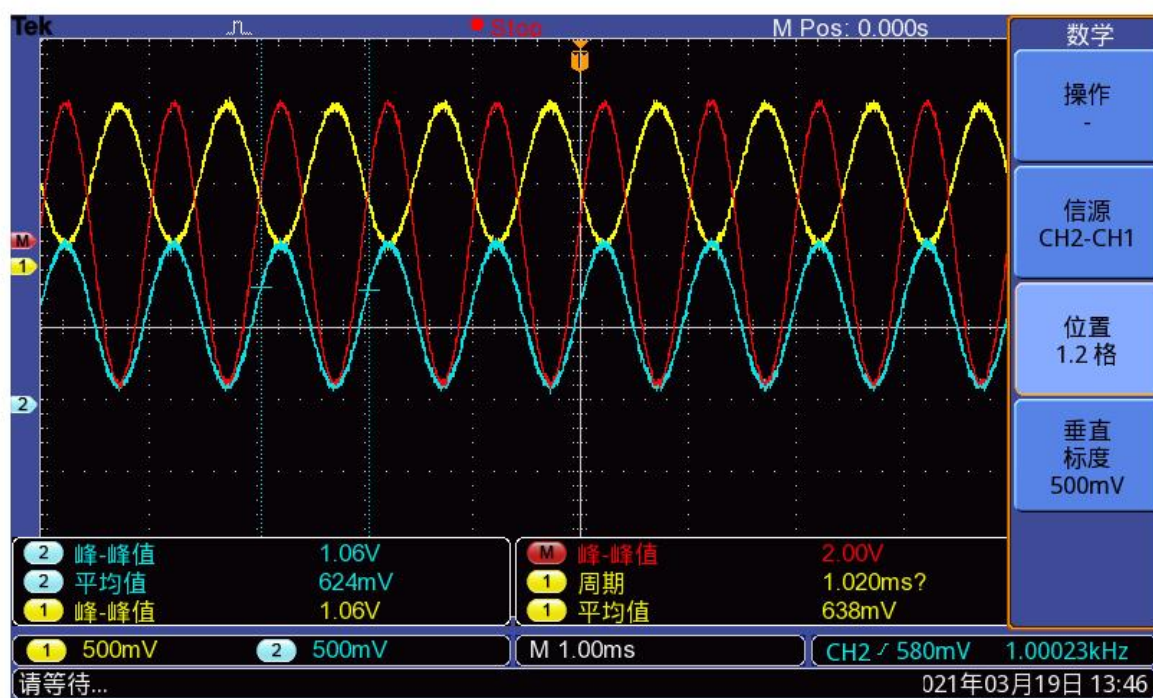
(1) DAC 电源，即 DACVDD（或者叫 DAC_LDO），DACVDD 等级越大功耗越大，同等级模拟增益下 dac 输出幅度越大。

```
#define TCFG_AUDIO_DAC_LDO_VOLT DACVDD_LDO_1_80V
```

(2) DAC 增益等级，目前 AC700N 音量调节仅支持固定模拟调节数字的方式，固定的模拟增益档位设置如下：

```
#define MAX_ANA_VOL (14) // 系统最大模拟音量,范围：0 ~ 15
```

关于差分信号幅度的测量，可以将示波器两个通道正端分别单端接输出的 P 端和 N 端，负端接地，然后使用 math 功能显示差分信号，实际测试如下：



或者板子使用电池供电，不要与示波器共地，也可以使用示波器一个通道正端接 P，负端接 N，可以直接量到差分信号。

不同的 DACVDD，在最大模拟音量等级对应的输出幅度如下（不同芯片，会有微小差异，该测试值仅供参考）：

dacvdd电压/V	dac输出幅度/mVrms
1.8	458
1.9	485
2.0	512
2.1	539
2.2	566
2.3	593
2.5	648
2.8	727

4.4.3 DAC 输出通道配置

AC700N 的 dac 支持以下三种输出方式，分别是单左差分，单右差分和双声道差分。每个声道均有一个 P 端一个 N 端，分别为差分的正负端。

DAC 硬件上的连接方式,可选的配置：

// DAC_OUTPUT_MONO_L 单左差分

```
// DAC_OUTPUT_MONO_R           单右差分
// DAC_OUTPUT_LR               双声道差分
#define TCFG_AUDIO_DAC_CONNECT_MODE    DAC_OUTPUT_LR // 选择 dac 输出通道
```

4.4.2 DAC 使用范例

范例路径: cpu/br36/audio/audio_demo/audio_dac_demo.c

使用方法: 把 void audio_dac_demo_open(void) 函数放在 app_main() 中 audio_dec_init() 后调用即可。demo 为持续输出一个正弦信号。

4.5 ADC 配置

4.5.1 ADC 使用范例

范例路径: cpu/br36/audio/audio_demo/audio_adc_demo.c

4.5.2 通话过程 mic 数据获取

文件路径: cpu/br36/audio/audio_enc.c

函数接口:

- (1) ADC_MIC: adc_mic_output_handler
- (2) PDM_MIC: audio_plnk_mic_output
- (3) IIS_MIC: audio_iis_mic_output

4.5.3 省电容 mic 使用

请查阅“省电容 mic 使用手册.docx”，了解省电容 mic 的详细用法和调试方式

4.6 音量配置

4.6.1 音量调节方式

/*

*系统音量类型选择

*软件数字音量是指纯软件对声音进行运算后得到的

*硬件数字音量是指 dac 内部数字模块对声音进行运算后输出

*/

```
#define VOL_TYPE_DIGITAL      0    //软件数字音量(调节解码输出数据的音量)
#define VOL_TYPE_ANALOG      1    //(暂未支持)硬件模拟音量
#define VOL_TYPE_AD          2    //(暂未支持)联合音量(模拟数字混合调节)
#define VOL_TYPE_DIGITAL_HW  3    //硬件数字音量(调节 DAC 模块的硬件音量)
#define SYS_VOL_TYPE         VOL_TYPE_DIGITAL
```

目前 SDK 仅支持调节数字音量，模拟音量等级为固定值，固定值由 MAX_ANA_VOL 决定，这个值也决定了最大输出信号幅度。

```
#define MAX_ANA_VOL          (13)    // 系统最大模拟音量,范围: 0 ~ 15
```

4.7 编解码配置

4.7.1 解码声道配置

解码声道配置根据 DAC 输出模式做自动适配，也可根据需要设置与 DAC 输出相匹配的声道。

AC700N 支持声道如下：

```
enum audio_channel {
    AUDIO_CH_LR = 0,           //立体声
    AUDIO_CH_L,                //左声道（单声道）
    AUDIO_CH_R,                //右声道（单声道）
    AUDIO_CH_DIFF,             //差分（单声道）
    AUDIO_CH_DUAL_L,           //双声道都为左
    AUDIO_CH_DUAL_R,           //双声道都为左右
    AUDIO_CH_DUAL_LR,          //双声道为左右混合

    AUDIO_CH_MAX = 0xff,
};
```

设置声道接口统一为：

```
int audio_decoder_set_output_channel(struct audio_decoder *dec, enum audio_channel);
```

4.7.2 编解码格式配置

```

336 //*****
337 //                                编解码配置(codec)                                //
338 //*****
339 /*解码格式使能配置*/
340 #define TCFG_DEC_G729_ENABLE          DISABLE /*format:*.wtg*/
341 #define TCFG_DEC_MTY_ENABLE          DISABLE
342 #define TCFG_DEC_WTG2_ENABLE          ENABLE /*format:*.wts*/
343 #define TCFG_DEC_WAV_ENABLE          DISABLE
344 #define TCFG_DEC_MP3_ENABLE          DISABLE
345
346 /*提示音叠加配置*/
347 #define TCFG_WAV_TONE_MIX_ENABLE      DISABLE
348 #define TCFG_MP3_TONE_MIX_ENABLE      DISABLE
349 #define TCFG_WTG_TONE_MIX_ENABLE      DISABLE
350 #define TCFG_WTS_TONE_MIX_ENABLE      ENABLE

```

1、编解码使能配置

这里需要根据具体方案需求进行配置，既能满足方案需求，又不额外使能不需要的解码格式，减少方案代码量。

2、提示音配置

提示音播放支持的格式跟普通解码一致。

【注意】提示音是否支持叠加播放是单独配置的。如果提示音播放参数 `preemption = 0`，但是对应格式的提示音叠加播放配置 `DISABLE`，则默认以打断方式播放。比如默认是支持 `wts` 格式解码，以及 `wts` 格式提示音叠加，则可以在播放 `wts` 格式提示音的时候，根据需要配置 `preemption` 参数接口。

4.8 音频同步配置

1) 蓝牙音频同步开关宏定义：

`audio_dec.c` 中，

```
#define AUDIO_CODEC_SUPPORT_SYNC 1
```

2) 蓝牙 A2DP 音频延时配置：

`lib_btctlr_config.c` 中，

普通延时模式的配置项：

```
const int CONFIG_A2DP_DELAY_TIME = 340 ;//200;
```

低延时模式的配置项：

```
const int CONFIG_A2DP_DELAY_TIME_LO = 100;
```


3) 蓝牙 ESCO 音频延时配置:

audio_dec.c 中, esco_decoder_syncts_setup()函数,

通话模式的 ESCO 延时:

```
#define ESCO_DELAY_TIME 60
```

SIRI/手机语音助手的 ESCO 延时:

```
#define ESCO_RECOGNITION_TIME 220
```

4) TWS 提示音音频同步延迟配置:

```
#define TWS_TONE_CONFIRM_TIME 250 /*TWS 提示音音频同步确认时间(也是音频解码主  
从确认超时时间)*/
```

5) 蓝牙音频同步参考时钟选择配置:

```
void bt_audio_sync_nettime_select(u8 base); //0 - 主机, 1 - tws, 2 - BLE
```

4.9 通话算法开发配置

CVP (Clear Voice Process), 即清晰语音处理。主要用在蓝牙通话中, 对上行数据 (即耳机 mic 发给远端手机的数据) 进行回音消除和降噪等处理, 使得语音清晰, 提升通话效果的算法处理。

4.9.1 算法离线开发

有些应用不用通话或者不用蓝牙, 比如只是单纯做一个降噪处理, 可以通过离线开发的方式进行开发。

实现代码: cpu/br36/audio/audio_cvp_demo.c

4.9.2 第三方算法开发

SDK 发布默认使用原厂开发的 CVP 相关算法进行处理。如有相应算法开发能力的客户, 可自行实现对语音的处理。数据流程已经写好, 只需实现算法函数函数 (audio_aec_run()) 即可。具体可以查看实现代码里面的代码注释。

- SDK 配置:

//第三方清晰语音开发使能: 使能之后, 即可在 audio_aec_demo.c 开发自研算法

```
#define TCFG_CVP_DEVELOP_ENABLE
```

```
ENABLE_THIS_MOUDLE
```

- 实现代码: earphone/aec/br36/audio_aec_demo.c
- 实现工程: AC700N_CVP.cbp

4.9.3 通话参数调试

打开杰理 SDK 工具/编译前配置工具/蓝牙配置: 通话参数配置



也可以选择在线调试参数, sdk 配置如下:

```

//*****
//
//          aec 工具在线调试
//*****
#define TCFG_AEC_TOOL_ONLINE_ENABLE          //aec 在线调试使能, 使用蓝牙串口调试, 需要打开宏 APP_ONLINE_DEBUG

```

配合安卓系统应用程序 (Audio Tools), 实现通过过程在线调试。调试完毕, 再将配置参数更新到上面 SDK 工具里面的通话参数配置即可。

4.9.4 双 mic 降噪 (ENC)

(1) 双 mic 降噪配置

双 mic 降噪功能相关配置有以下宏定义:

- 使能双 mic 降噪功能: sdk 板级配置头文件 (比如: board_ad697n_demo_cfg.h)

```

/*ENC(双mic降噪)使能*/
#define TCFG_AUDIO_DUAL_MIC_ENABLE          ENABLE_THIS_MOUDLE

```

- 数据导出调试使能: **调试使用, 正常使用需要关闭**

```

/*Audio数据导出配置:通过蓝牙spp导出或者sd写卡导出*/
#define AUDIO_DATA_EXPORT_USE_SD          1
#define AUDIO_DATA_EXPORT_USE_SPP          2
#define TCFG_AUDIO_DATA_EXPORT_ENABLE          AUDIO_DATA_EXPORT_USE_SPP

```


c.通话在线调试功能：单 mic/双 mic 均支持

```
/*通话参数在线调试*/
#define TCFG_AEC_TOOL_ONLINE_ENABLE          DISABLE_THIS_MOUDLE
```

d.指标测试功能：如果需要随时可以测试指标，则改宏定义需要常开

```
/*双mic降噪DUT测试模式，配合设备测试mic频响和降噪量*/
#define TCFG_AUDIO_DMS_DUT_ENABLE          ENABLE_THIS_MOUDLE
```

e.数据处理流程

apps/earphone/aec/br36/audio_aec_dms.c

(2) 指标测试

实现代码：cpu/br36/audio/audio_dms_tool.c

实现流程：

(1) 测试仪器通过蓝牙和耳机连接，然后通过 spp 发对应的命令给小机处理

(2) 默认的命令有：

DMS_OUTPUT_SEL_DEFAULT 输出算法处理结果，用来测试 ENC 效果

DMS_OUTPUT_SEL_MASTER 输出主 mic 的原始数据：测试 mic 的频响

DMS_OUTPUT_SEL_SLAVE 输出副 mic 的原始数据：测试 mic 的频响

(3) 通过接口 audio_aec_output_sel()控制 CVP 模块的输出，具体参数传递，请看代码注释

4.9.5 双 mic 风噪检测模块

功能介绍：检测到风噪时，降低降噪压制强度，避免把人声压制掉。

(1) 在 audio_aec_dms.c 里面把 CONST_DMS_WNC 置 1

```
181 /*
182 *风噪自适应GainFloor使能控制
183 *0:关闭，1:开启
184 */
185 const u8 CONST_DMS_WNC = 1;
186
```

(2) 在 audio_aec_dms.c 里面使能位或上 WNC_EN

```
1077
1078     if (CONST_DMS_WNC) {
1079         aec_param->EnableBit |= WNC_EN;
1080     }
1081
```

(3) 配置风噪检测参数

```
634
635     p->wn_detect_time = 0.32f; //检测时间, 单位s
636     p->wn_detect_time_ratio_thr = 0.8f; //风噪比例, 表示检测时间内有80%的帧的风噪超过风噪阈值表示有风噪需要使用设置的压制强度
637     p->wn_detect_thr = 0.5f; //风噪阈值, 范围0~1
638     p->wn_minsuppress = 0.5f; //降噪压制强度, 范围0~1, 越小压制越强
639
```

wn_detect_time: 风噪检测的时间窗大小 越大越准, 能过滤掉短时风噪, 建议值 0.32f

wn_detect_time_ratio_thr: 在时间窗内超过 wn_detect_thr 的帧数占比阈值, 越大越难触发 建议值 0.8f

wn_detect_thr: 风噪检测阈值 越大越难触发 建议值 0.5f

wn_minsuppress: 检测到风噪时, 替换 GlobalMinSuppress 为这个值

基本原理:

每一帧都会计算一个风噪的概率, 当大于 wn_detect_thr 时, 当前帧会被标记为风噪帧。当在 wn_detect_time 的时间内有 wn_detect_time_ratio_thr 比例的帧为风噪帧则认为风噪存在, 使用 wn_minsuppress 替换 GlobalMinSuppress 的值

(4) 可以通过 int cvp_dms_get_wind_detect_state(void); 获取风噪检测结果

```
222
223 /*获取风噪的检测结果, 1: 有风噪, 0: 无风噪*/
224 int cvp_dms_get_wind_detect_state(void);
225
```

4.9.6 双麦克风异常检测

功能介绍: 使用双麦时, 检测到其中一个麦克风异常 (比如麦克风堵塞、麦克风坏了等), 主动切换成单麦模式使用另一个正常的麦克风, 保证通话能正常使用。

(1) 在 audio_aec_dms.c 里面把 CONST_DMS_MALFUNCTION_DETECT 置 1

```

189 //*****
190 //          麦克风异常检测配置
191 //*****
192 /*麦克风异常检测，双mic切单mic*/
193 const u8 CONST_DMS_MALFUNCTION_DETECT = 1;
194 hui connect server successfully 18:25:31
195 /*
196 * 是否设置通话mic默认状态
197 * 需要重定义audio_dms_get_malfunc_state(void)
198 */
199 const u8 CONST_DMS_SET_MFDT_STATE = 1;
200
201 //*****麦克风异常检测配置 end*****
202

```

(2) 在 audio_aec_dms.c 里面使能位或上 ENC_EN | MFDT_EN

```

1081
1082     if (CONST_DMS_MALFUNCTION_DETECT) {
1083         aec_param->EnableBit |= ENC_EN | MFDT_EN;
1084     }
1085

```

(3) 配置故障麦克风检测参数

```

711     if (CONST_DMS_MALFUNCTION_DETECT) {
712         /*检测时间*/
713         p->detect_time = 1.0f;          // in second
714         /*0~-90 dB 两个mic能量差异持续大于此阈值超过检测时间则会检测为故障*/
715         p->detect_eng_diff_thr = 6.f;    // dB
716         /*0~-90 dB 当处于故障状态时，正常的mic能量大于此阈值才会检测能量差异，避免安静环境下误判切回正常状态*/
717         p->detect_eng_lowerbound = -55.f; // 0~-90 dB start detect when mic energy lower than this
718         p->MalfuncDet_MaxFrequency = 8000; //检测频率上限
719         p->MalfuncDet_MinFrequency = 400;  //检测频率下限
720         p->OnlyDetect = 0; // 0 -> 故障切换到单mic模式， 1-> 只检测不切换
721     }
722

```

detect_time: 检测时间，单位：秒

detect_eng_diff_thr: 麦克风能量差值，两个 mic 能量差异持续大于此阈值超过检测时间则会检测为故障，设置范围：0~90dB

detect_eng_lowerbound: 检测阈值，当有麦克风能量低于此值时，才可是做检测

MalfuncDet_MaxFrequency: 检测频率上限

MalfuncDet_MinFrequency: 检测频率下限

OnlyDetect: 是否之作检测不做单麦切换， 0 : 故障切换到单 mic 模式， 1: 只检测不切换

(4) 相关的接口

```

226 /*单双麦切换状态
227 * 0: 正常双麦 ;
228 * 1: 副麦坏了, 触发故障
229 * -1: 主麦坏了, 触发故障
230 */
231 int cvp_dms_get_malfunc_state(void);
232
233 /*
234 * 获取mic的能量值, 开了MFDT_EN才能用
235 * mic: 0 获取主麦能量
236 * mic: 1 获取副麦能量
237 * return: 返回能量值, [0~90.3], 返回-1表示错误
238 */
239 float cvp_dms_get_mic_energy(u8 mic);
240

```

4.9.7 神经网络降噪 DNS

(1) 神经网络降噪使能

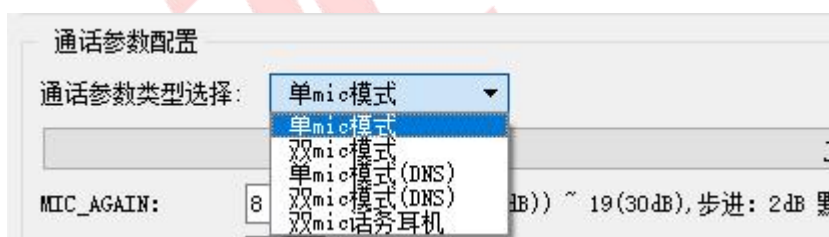
在板级头文件, 根据需要, 选择降噪模式, ANS 为传统降噪, DNS 为神经网络降噪:

```

/*通话降噪模式配置*/
#define CVP_ANS_MODE      0      /*传统降噪*/
#define CVP_DNS_MODE      1      /*神经网络降噪*/
#define TCFG_AUDIO_CVP_NS_MODE CVP_DNS_MODE

```

(2) 神经网络降噪参数配置



根据单 MIC 还是双 MIC, ANS 还是 DNS 进行模式选择, 配置完参数, 点击保存 (仅保存被选择的模式的配置)。

4.9.8 双 MIC 话务耳机

(1) 话务耳机功能使能


```
/*
 * ENC(双mic降噪) 配置
 * 双mic降噪包括DMS_NORMAL和DMS_FLEXIBLE, 在使能TCFG_AUDIO_DUAL_MIC_ENABLE
 * 的前提下, 根据具体需求, 选择对应的DMS模式
 */
/*ENC(双mic降噪) 使能*/
#define TCFG_AUDIO_DUAL_MIC_ENABLE          ENABLE_THIS_MOUDLE

/*DMS模式选择*/
#define DMS_NORMAL          1    // 普通双mic降噪(mic距离固定)
#define DMS_FLEXIBLE        2    // 适配mic距离不固定且距离比较远的情况, 比如头戴式话务耳机
#define TCFG_AUDIO_DMS_SEL    DMS_FLEXIBLE
```

❖ 先使能双 mic 降噪功能

1. `#define TCFG_AUDIO_DUAL_MIC_ENABLE ENABLE_THIS_MOUDLE`

❖ 再选择 DMS 类型: DMS_FLEXIBLE

1. `#define TCFG_AUDIO_DMS_SEL DMS_FLEXIBLE`

(2) 话务耳机参数配置



选择“双 mic 话务耳机”模式，具体调试指引，请查阅工具内置的调试手册。

(3) 双 MIC 话务耳机单双麦切换

函数头文件: `include_lib/media/media_new/media/cpu/br36/commproc_dms.h`

```
1. typedef enum {
2.     DMS_FLEXIBLE_DUAL_MIC = 0, //双 mic
3.     DMS_FLEXIBLE_USE_TALK_MIC, //通话 mic (主麦)
4.     DMS_FLEXIBLE_USE_REF_MIC,  //参考 mic (副麦)
5. } DMS_FLEXIBLE_MIC;
6.
7. /* 话务耳机切换使用的 mic
8.  * DMS_FLEXIBLE_DUAL_MIC : 正常双麦话务耳机
9.  * DMS_FLEXIBLE_USE_TALK_MIC : 使用主麦
10.  * DMS_FLEXIBLE_USE_REF_MIC : 使用副麦
11. */
```

```
12. int aec_dms_flexible_selete_mic(DMS_FLEXIBLE_MIC mic);
```

4.10 离线语音识别配置

1、离线语音支持的关键词说明

目前 AC700N 系列支持选择**中文**或者**印度英语**离线语言识别

1.1、中文关键词

关键词	模式
小杰小杰/小杰同学	唤醒手机语音助手/Siri
播放音乐	多媒体（通话不可用）
停止播放	
暂停播放	
增大音量	
减小音量	
上一首	
下一首	
打开降噪	ANC 模式（通话不可用）
关闭降噪	
打开通透	
接听电话	通话来电模式
挂断电话	

1.2、印度英语关键词

关键词	模式
play music	多媒体（通话不可用）
stop music	
volume up	
volume down	
previous song	
next song	
answer call	通话来电模式
reject call	

2、离线语音使能配置

2.1、在对应的板级配置文件中，使能 TCFG_SMART_VOICE_ENABLE;

2.2、在对应的板级配置文件中，配置宏 TCFG_AUDIO_KWS_LANGUAGE_SEL 选择支持的离线识别语言，目前支持中文离线语音识别和印度英语离线语音识别。

```
1.  /*******//
2.  //                                Audio Smart Voice                                //
3.  /*******//
4.  #define TCFG_SMART_VOICE_ENABLE          ENABLE_THIS_MOUDLE          //ENABLE_THIS_MOUDLE
5.  #define TCFG_KWS_VOICE_EVENT_HANDLE_ENABLE TCFG_SMART_VOICE_ENABLE //语音事件处理流程开关
6.  #if TCFG_KWS_VOICE_RECOGNITION_ENABLE
7.  #define TCFG_CALL_KWS_SWITCH_ENABLE          DISABLE_THIS_MOUDLE
8.  #else
9.  #define TCFG_CALL_KWS_SWITCH_ENABLE          TCFG_SMART_VOICE_ENABLE
10. #endif
11.
12. /*识别语言选择*/
13. #define KWS_CH          1          /*中文*/
14. #define KWS_INDIA_EN    2          /*印度英语*/
15. #define TCFG_AUDIO_KWS_LANGUAGE_SEL    KWS_CH
```

3、播歌模式离线语音关键词识别配置

在 lib_media_config.c 中，根据需求进行关键词的识别阈值配置，普通模式的为 CONFIG_KWS_MULTIO_CONFIGDENCE[]，通话模式未 CONFIG_KWS_CALL_CONFIDENCE[]

```
1.  /*杰理 KWS 关键词识别*/
2.  #if TCFG_SMART_VOICE_ENABLE
3.  const int CONFIG_KWS_WAKE_WORD_MODEL_ENABLE = 0;
4.  const int CONFIG_KWS_MULTI_CMD_MODEL_ENABLE = 1;
```



```
5.  #if TCFG_CALL_KWS_SWITCH_ENABLE
6.  const int CONFIG_KWS_CALL_CMD_MODEL_ENABLE = 1;
7.  #else
8.  const int CONFIG_KWS_CALL_CMD_MODEL_ENABLE = 0;
9.  #endif
10. #ifdef CONFIG_CPU_BR28
11. const int CONFIG_KWS_RAM_USE_ENABLE = 1;
12. #else
13. const int CONFIG_KWS_RAM_USE_ENABLE = 2;
14. #endif
15. const int CONFIG_KWS_ONLINE_MODEL = 1;
16.
17. #if (TCFG_AUDIO_KWS_LANGUAGE_SEL == KWS_CH)
18. const float CONFIG_KWS_MULTI_CONFIDENCE[] = {
19.     0.4, 0.4, 0.4, 0.4, //小杰小杰, 小杰同学, 播放音乐, 停止播放
20.     0.4, 0.4, 0.4, 0.4, //暂停播放, 增大音量, 减小音量, 上一首
21.     0.4, 0.4, 0.4, 0.4, //下一首, 打开降噪, 关闭降噪, 打开通透
22. };
23.
24. const float CONFIG_KWS_CALL_CONFIDENCE[] = {
25.     0.4, 0.4, //接听电话, 挂断电话
26. };
27. #else /*TCFG_AUDIO_KWS_LANGUAGE_SEL == KWS_INDIA_EN*/
28. const float CONFIG_KWS_MULTI_CONFIDENCE[] = {
29.     0.4, 0.4, 0.3, 0.4, //小杰小杰, 小杰同学, 播放音乐, 停止播放
30.     0.4, 0.4, 0.3, 0.4, //暂停播放, 增大音量, 减小音量, 上一首
31.     0.3, 0.4, 0.4, 0.4, //下一首, 打开降噪, 关闭降噪, 打开通透
32. };
33.
34. const float CONFIG_KWS_CALL_CONFIDENCE[] = {
35.     0.4, 0.3, //接听电话, 挂断电话
36. };
37. #endif /*TCFG_AUDIO_KWS_LANGUAGE_SEL*/
```

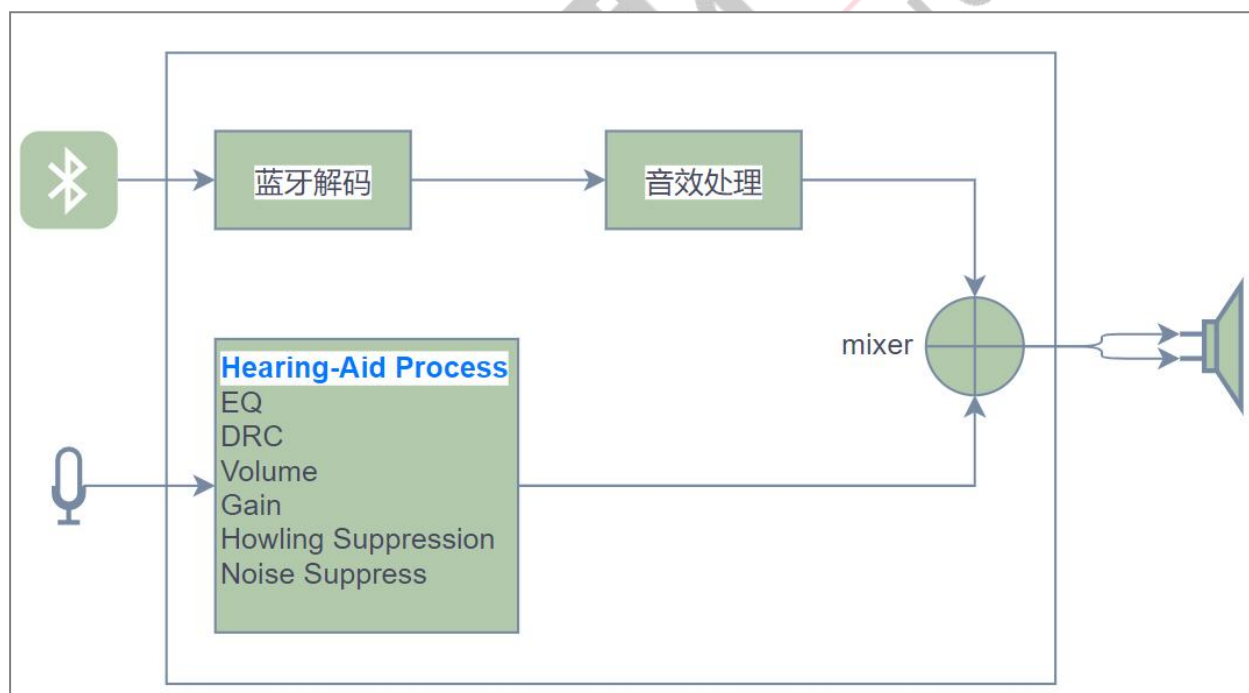
```

38. #else /*TCFG_SMART_VOICE_ENABLE == 0*/
39. const int CONFIG_KWS_WAKE_WORD_MODEL_ENABLE = 0;
40. const int CONFIG_KWS_MULTI_CMD_MODEL_ENABLE = 0;
41. const int CONFIG_KWS_CALL_CMD_MODEL_ENABLE = 0;
42. const int CONFIG_KWS_ONLINE_MODEL = 0;
43. #endif /*TCFG_SMART_VOICE_ENABLE*/

```

4.11 辅听功能配置

辅听耳机一般是指听力衰退由轻、中度到重度的过渡性产品，通过智能听力补偿，帮助年长者或是有提高沟通需求的人群改善聆听体验，恢复正常的沟通与生活。辅听耳机通过耳机自带的麦克风，采集外界信号，并通过相应的算法处理，最后输出到喇叭端，辅助佩戴者对外界信息的获取。辅听处理流程大体如下：



4.11.2 使能配置

(1) 在板级配置文件里面打开辅听使能

```

1. //*****//
2. //          Digital Hearing Aid(DHA)辅听耳机配置          //
3. //*****//

```

```

4.  /*辅听功能使能*/
5.  #define TCFG_AUDIO_HEARING_AID_ENABLE      ENABLE_THIS_MOUDLE
6.  /*听力验配功能*/
7.  #define TCFG_AUDIO_DHA_FITTING_ENABLE      ENABLE
8.  /*辅听功能互斥配置*/
9.  #define TCFG_AUDIO_DHA_AND_MUSIC_MUTEX      ENABLE    //辅听功能和音乐播放互斥(默认互斥,
资源有限)
10. #define TCFG_AUDIO_DHA_AND_CALL_MUTEX      ENABLE    //辅听功能和通话互斥(默认互斥, 资源
有限)
11. #define TCFG_AUDIO_DHA_AND_TONE_MUTEX      ENABLE    //辅听功能和提示音互斥

```

(2) 在板级配置文件里面设置 MIC 通道 TCFG_AD2DA_LOW_LATENCY_MIC_CHANNEL 和 DAC 输出采样率 TCFG_AD2DA_LOW_LATENCY_SAMPLE_RATE, 采样率默认使用 32k

```

1.  /*
2.   * MIC_TO_DAC 低延时通道功能
3.   */
4.  #define TCFG_AD2DA_LOW_LATENCY_ENABLE      (0)
5.  #define TCFG_AD2DA_LOW_LATENCY_SAMPLE_RATE (32000)      // 目前混叠输出仅支持
固定 44100
6.  #define TCFG_AD2DA_LOW_LATENCY_MIC_CHANNEL (LADC_CH_MIC_L) // 目前仅支持单声道

```

(3) 在板级配置文件里面配置 MIC 的供电

建议: 为了提高抗干扰性, 我们建议麦克风使用差分输入模式, 减少一些高频电流声的产生

```

1.  /*MIC 模式配置:单端隔直电容模式/差分隔直电容模式/单端省电容模式*/
2.  #if (TCFG_AUDIO_ANC_ENABLE || TCFG_AUDIO_HEARING_AID_ENABLE)
3.  /*注意:ANC/辅听功能使能情况下, 使用差分 mic*/
4.  #define TCFG_AUDIO_MIC_MODE      AUDIO_MIC_CAP_DIFF_MODE
5.  #define TCFG_AUDIO_MIC1_MODE     AUDIO_MIC_CAP_DIFF_MODE
6.  #else
7.  #define TCFG_AUDIO_MIC_MODE      AUDIO_MIC_CAP_MODE
8.  #define TCFG_AUDIO_MIC1_MODE     AUDIO_MIC_CAP_MODE
9.  #endif/*TCFG_AUDIO_ANC_ENABLE*/
10.

```

```

11. /*
12.  *>>MIC 电源管理:根据具体方案, 选择对应的 mic 供电方式
13.  *(1) 如果是多种方式混合, 则将对的供电方式或起来即可, 比如
    (MIC_PWR_FROM_GPIO | MIC_PWR_FROM_MIC_BIAS)
14.  *(2)如果使用固定电源供电(比如 dacvdd), 则配置成 DISABLE_THIS_MOUDLE
15. */
16. #define MIC_PWR_FROM_GPIO      (1UL << 0)    //使用普通 IO 输出供电
17. #define MIC_PWR_FROM_MIC_BIAS  (1UL << 1)    //使用内部 mic_ldo 供电(有上拉电阻可配)
18. #define MIC_PWR_FROM_MIC_LDO   (1UL << 2)    //使用内部 mic_ldo 供电
19. //配置 MIC 电源
20. #if ((TCFG_AUDIO_MIC_MODE == AUDIO_MIC_CAP_DIFF_MODE) || (TCFG_AUDIO_MIC1_MODE == AU
    DIO_MIC_CAP_DIFF_MODE))
21. #define TCFG_AUDIO_MIC_PWR_CTL      MIC_PWR_FROM_MIC_LDO
22. #else
23. #define TCFG_AUDIO_MIC_PWR_CTL      MIC_PWR_FROM_MIC_BIAS
24. #endif/*TCFG_AUDIO_MIC_MODE*/
25.
26. //使用普通 IO 输出供电:不用的 port 配置成 NO_CONFIG_PORT
27. #if (TCFG_AUDIO_MIC_PWR_CTL & MIC_PWR_FROM_GPIO)
28. #define TCFG_AUDIO_MIC_PWR_PORT      IO_PORTC_02
29. #define TCFG_AUDIO_MIC1_PWR_PORT     NO_CONFIG_PORT
30. #define TCFG_AUDIO_MIC2_PWR_PORT     NO_CONFIG_PORT
31. #endif/*MIC_PWR_FROM_GPIO*/

```

(4) 在 audio_hearing_aid.c 里面设置 MIC 采样率 DHA_MIC_SAMPLE_RATE 和采样点数 DHA_MIC_SAMPLE_POINT, 默认使用 32k 采样率和 64 采样点数, 采样率最好与 DAC 输出采样率保持一致, 减少变采样的操作; 当打开降噪时, MIC 只能使用 32k 采样率 160 采样点数或者 16k 采样率 80 采样点数

```

1.  /*Microphone 采样率和采样点数配置*/
2.  #if DHA_NS_ENABLE
3.  #define HEARING_AID_CLK      (128 * 1000000L)
4.  #define DHA_MIC_SAMPLE_RATE  32000
5.  #define DHA_MIC_SAMPLE_POINT (DHA_MIC_SAMPLE_RATE / 200)

```

```

6.  #else
7.  #define HEARING_AID_CLK          (128 * 1000000L)
8.  #define DHA_MIC_SAMPLE_RATE     32000
9.  #define DHA_MIC_SAMPLE_POINT    64
10. #endif/*DHA_NS_ENABLE*/

```

(5) 调用 audio_hearing_aid_demo();函数打开或者关闭辅听功能

```

1.  #if TCFG_AUDIO_HEARING_AID_ENABLE
2.      /*辅听功能测试 demo 开关*/
3.      case KEY_HEARING_AID_TOGGLE:
4.          printf("KEY_HEARING_AID_TOGGLE\n");
5.          extern void audio_hearing_aid_demo(void);
6.          audio_hearing_aid_demo();
7.          break;
8.  #endif/*TCFG_AUDIO_HEARING_AID_ENABLE*/

```

4.11.3 功能模块说明

辅听功能在 audio_hearing_aid.c 里面有以下可选择的模块开关：MIC 声音淡入、EQ、DRC、移频啸叫抑制、陷波啸叫抑制、降噪、变采样输出和音量控制。

```

1.  /*******//
2.  //          数字信号处理配置(Digital Hearing_Aid[DHA] Process Config)          //
3.  /*******//
4.  /*mic 声音淡入使能*/
5.  #define DHA_FADEIN_ENABLE          1
6.  /*均衡器*/
7.  #define DHA_EQ_ENABLE              1
8.  /*动态范围控制*/
9.  #define DHA_DRC_ENABLE              1
10. /*[啸叫抑制-移频法]Howling Suppress - FreqShift*/
11. #define DHA_HS_FS_ENABLE            1
12. /*[啸叫抑制-陷波法]Howling Suppress - Notch*/

```

```

13. #define DHA_HS_NOTCH_ENABLE          1
14. /*噪声抑制 Noise Suppress*/
15. #define DHA_NS_ENABLE                  0
16. /*变采样模块：如果输入输出采样率一致，条件编译自动关闭使能*/
17. #define DHA_SRC_ENABLE                 1
18. /*音量控制模块*/
19. #define DHA_VOLUME_ENABLE              1

```

(1) MIC 声音淡入

步进调节参数：HEARING_AID_FADEIN_SHIFT

(2) 均衡器 EQ

默认使用 8 端的 EQ，可在 audio_mic_eq_tab[MIC_EQ_SECTION]里面调节的参数

```

1.  /*保留 EQ 段，做自定义滤波使用*/
2.  #define DHA_REV0_FREQ    100
3.  #define DHA_REV1_FREQ    16000
4.  #define MIC_EQ_SECTION    8
5.  static struct eq_seg_info audio_mic_eq_tab[MIC_EQ_SECTION] = {
6.      {0, EQ_IIR_TYPE_BAND_PASS, DHA_CH0_FREQ, 0, 0.7f},
7.      {1, EQ_IIR_TYPE_BAND_PASS, DHA_CH1_FREQ, 0, 0.7f},
8.      {2, EQ_IIR_TYPE_BAND_PASS, DHA_CH2_FREQ, 0, 0.7f},
9.      {3, EQ_IIR_TYPE_BAND_PASS, DHA_CH3_FREQ, 0, 0.7f},
10.     {4, EQ_IIR_TYPE_BAND_PASS, DHA_CH4_FREQ, 0, 0.7f},
11.     {5, EQ_IIR_TYPE_BAND_PASS, DHA_CH5_FREQ, 0, 0.7f},
12.     {6, EQ_IIR_TYPE_HIGH_PASS, DHA_REV0_FREQ, 0, 0.7f},
13.     {7, EQ_IIR_TYPE_LOW_PASS,  DHA_REV1_FREQ, 0, 0.7f},
14. };

```

(3) 动态范围控制 DRC

可在 audio_hearing_aid.c 的 hearing_aid_drc_get_filter_info()里面调节限幅的阈值和放大的倍数

th: 限幅器阈值

Gain_dB: drc 限幅后要放大的 dB 数

```

1. static int hearing_aid_drc_get_filter_info(void *drc, struct audio_drc_filter_info *
    info)

```



```

2.  {
3.      float th = -10.0f; //db -60db~0db, 限幅器阈值
4.      Gain_dB = 7.0f; /*drc 压制后要放大的 dB 数*/
5.      Gain_dB = powf(10.0, Gain_dB / 20.0);
6.
7.      int threshold = roundf(powf(10.0f, th / 20.0f) * 32768);
8.      mic_drc_p.nband = 1;
9.      mic_drc_p.type = 1; //1:限幅器
10.     mic_drc_p._p.limiter[0].attacktime = 5;
11.     mic_drc_p._p.limiter[0].releasetime = 500;
12.     mic_drc_p._p.limiter[0].threshold[0] = threshold;
13.     mic_drc_p._p.limiter[0].threshold[1] = 32768;
14.     info->R_pch = info->pch = &mic_drc_p;
15.     return 0;
16. }

```

(4) 啸叫抑制-移频法

使用移频的方法做啸叫抑制

(5) 啸叫抑制-陷波法

使用陷波的方法做啸叫抑制

(6) 低延时降噪

对 mic 声音进行降噪处理，可调节的参数如下

suppress_level: 降噪强度，越大越强

gainfloor: 最小压制

```

1.  #if DHA_NS_ENABLE
2.      llns_init(param->sample_rate, 0.05f, 1.0f);
3.  #endif /*DHA_NS_ENABLE*/

```

(7) 输出变采样

目前降噪只能处理 16k 和 32k 采样率的数据，而 DAC 输出采样率不一样时，数据推到 DAC 前需要对数据进行采样率匹配，做变采样处理。

```

1.  #if DHA_SRC_ENABLE

```

```
2. hearing_src_init(hdl, 1, param->sample_rate, TCFG_AD2DA_LOW_LATENCY_SAMPLE_RATE);
3. #endif /*DHA_SRC_ENABLE*/
```

(8) 数字音量控制

可在 audio_dvol.h 里面设置最大音量等级 HEARING_DVOL_MAX 和音量调节步进 HEARING_DVOL_FS

```
1. /*Digital Volume Fade Step*/
2. #define HEARING_DVOL_FS      2
3. /*Digital Volume Max Level*/
4. #define HEARING_DVOL_MAX    16
```

调用 hearing_volume_set()函数可以设置数字音量

```
1. /*设置音量*/
2. static void hearing_volume_set(u8 volume);
```

4.11.1 声学结构规范

对于喇叭和麦克风共腔体的结构模具，比如 TWS 耳机，为了使得整机啸叫抑制效果更加符合预期，我们建议如下：

喇叭和麦克风有独立腔体，密封性好，隔离度好，麦克风装配位要有硅胶套或隔音棉，装机后与后盖贴合，阻断喇叭端漏音与腔体内非线性回声。

4.11.4 辅听音量

影响辅听音量大小的因素有软件和硬件

(1) 增益因素

● MIC 增益

audio_hearing_aid.c

```
1. static const hearing_adda_param_t hearing_adda_param = {
2.     .mic_ch_sel      = TCFG_AD2DA_LOW_LATENCY_MIC_CHANNEL,
3.     .mic_gain        = 12,
4.     .sample_rate     = DHA_MIC_SAMPLE_RATE, //采样率
```

```
5.      .adc_irq_points    = DHA_MIC_SAMPLE_POINT, //一次处理数据的数据单元，单位点 4 对齐(要
      配合 mic 起中断点数修改)
6.      .adc_buf_num      = 3,
7.      .dac_delay        = 6, //dac 硬件混响延时，单位 ms 要大于 point_unit*2
8.  };
```

● DAC 增益

■ DAC 模拟增益

audio_config.h

```
#define MAX_ANA_VOL          (14)    // 系统最大模拟音量,范围: 0 ~ 15
```

■ DAC 大音量模式

audio_hearing_aid.h

```
#define DHA_DAC_OUTPUT_ENHANCE_ENABLE 0 //DAC 输出音量增强使能：用来提高辅听的动态范围
```

(2) 软件处理

通过 DRC 模块，提高数字音频的平均功率。如下图所示，th= -10dB, Gain_dB= 7.0f，表示先将信号压缩-10dB，再将信号整理放大 7dB。因为影响声音大小主要是平均功率，所以可以通过先将大信号进行压缩，再将信号整体放大，起到提高平均功率的目的。建议 th 和 Gain_dB

有 3dB 左右的余量。如果觉得默认配置不够大声，可以继续放大一些，比如：th= -15dB，Gain_dB = 12dB

```
1.  static int hearing_aid_drc_get_filter_info(void *drc, struct audio_drc_filter_info *
      info)
2.  {
3.      float th = -10.0f; //单位: dB 范围: -60dB~0dB, 限幅器阈值
4.      Gain_dB = 7.0f; /*drc 压制后要放大的 dB 数*/
5.      Gain_dB = powf(10.0, Gain_dB / 20.0);
6.
7.      int threshold = roundf(powf(10.0f, th / 20.0f) * 32768);
8.      mic_drc_p.nband = 1;
9.      mic_drc_p.type = 1; //1:限幅器
10.     mic_drc_p._p.limiter[0].attacktime = 5;
11.     mic_drc_p._p.limiter[0].releasetime = 500;
12.     mic_drc_p._p.limiter[0].threshold[0] = threshold;
```

```
13. mic_drc_p._p.limiter[0].threshold[1] = 32768;
14. info->R_pch = info->pch = &mic_drc_p;
15. return 0;
16. }
```

4.11.5 听力验配

听力验配是为了找到听力障碍者耳朵不同频率的听力阈值，然后使用算法对辅听耳机播放的声音进行补偿，让听力障碍者听到的声音更加均匀清晰。

1、使用杰理之家 APP 验配

(1) 在板级配置文件里面使能辅听的宏 TCFG_AUDIO_HEARING_AID_ENABLE 和听力验配功能的宏 TCFG_AUDIO_DHA_FITTING_ENABLE

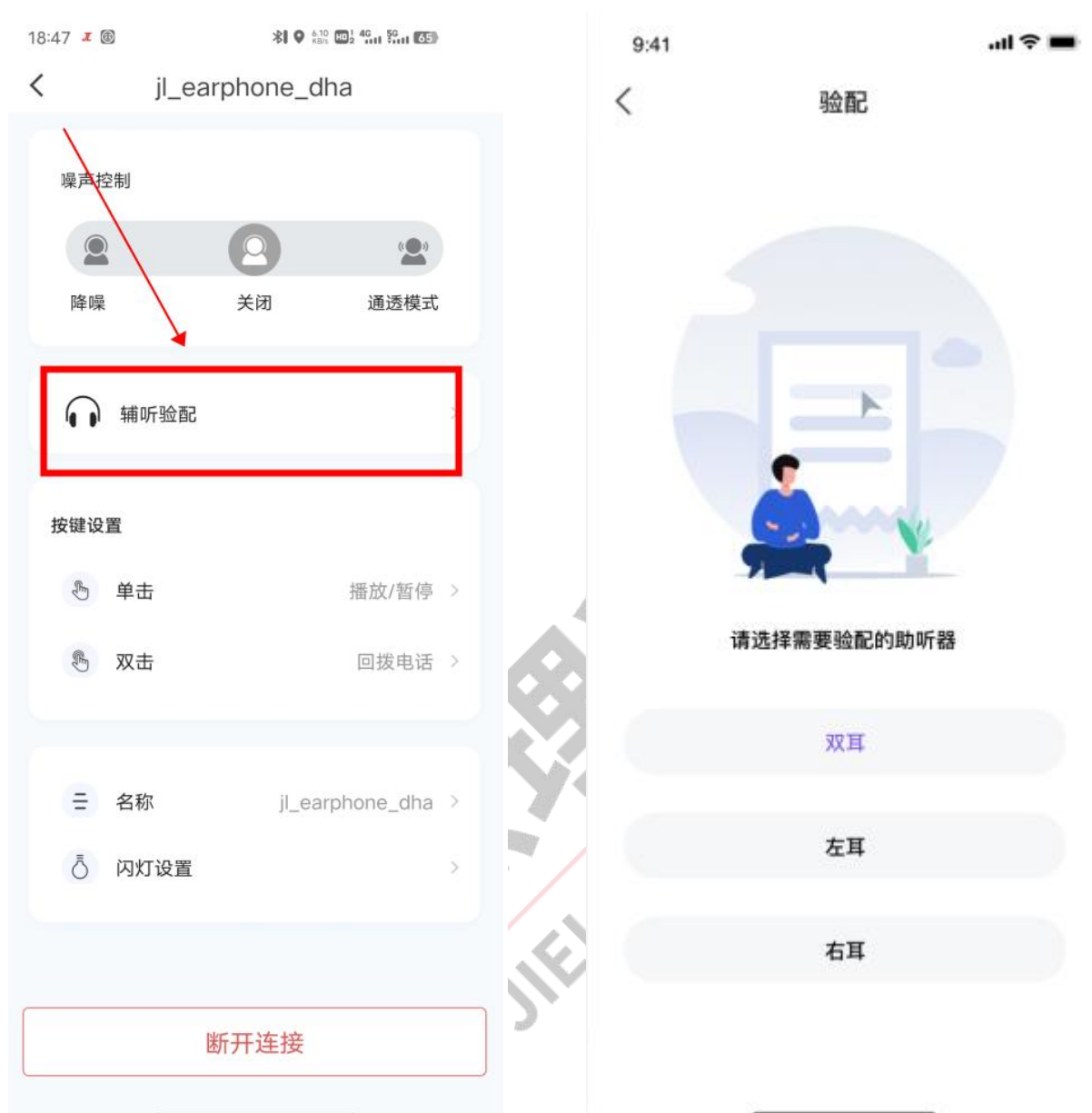
```
1. //*****//
2. // Digital Hearing Aid(DHA)辅听耳机配置 //
3. //*****//
4. /*辅听功能使能*/
5. #define TCFG_AUDIO_HEARING_AID_ENABLE ENABLE_THIS_MOUDLE
6. /*听力验配功能*/
7. #define TCFG_AUDIO_DHA_FITTING_ENABLE ENABLE
8. /*辅听功能互斥配置*/
9. #define TCFG_AUDIO_DHA_AND_MUSIC_MUTEX ENABLE //辅听功能和音乐播放互斥(默认互斥,
    资源有限)
10. #define TCFG_AUDIO_DHA_AND_CALL_MUTEX ENABLE //辅听功能和通话互斥(默认互斥, 资源
    有限)
11. #define TCFG_AUDIO_DHA_AND_TONE_MUTEX ENABLE //辅听功能和提示音互斥
```

(2) 打开 APP 功能, 在 app_config.h 文件里面打开宏 CONFIG_APP_BT_ENABLE, 使能 RSCP 协议的宏 RCSP_ADV_EN

```
1. //*****//
2. // AI 配置 //
3. //*****//
```

```
4. #define CONFIG_APP_BT_ENABLE
5.
6. #ifdef CONFIG_APP_BT_ENABLE
7. #define TRANS_DATA_EN 0
8. #define RCSP_BTmate_EN 0
9. #define RCSP_ADV_EN 1
10. #define AI_APP_PROTOCOL 0
11. #define LL_SYNC_EN 0
12. #define TUYA_DEMO_EN 0
13. #else
14. #define TRANS_DATA_EN 0
15. #define RCSP_BTmate_EN 0
16. #define RCSP_ADV_EN 0
17. #define AI_APP_PROTOCOL 0
18. #define LL_SYNC_EN 0
19. #define TUYA_DEMO_EN 0
20. #endif
```

(3) 打开杰理之家 APP 连接耳机后，会在这里显示一个辅听验配功能的入口，进去之后，可以选择只对左耳或者右耳，或者同时对双耳进行听力验配。注意：如果耳机没有打开辅听功能，这个界面是无法进去的。



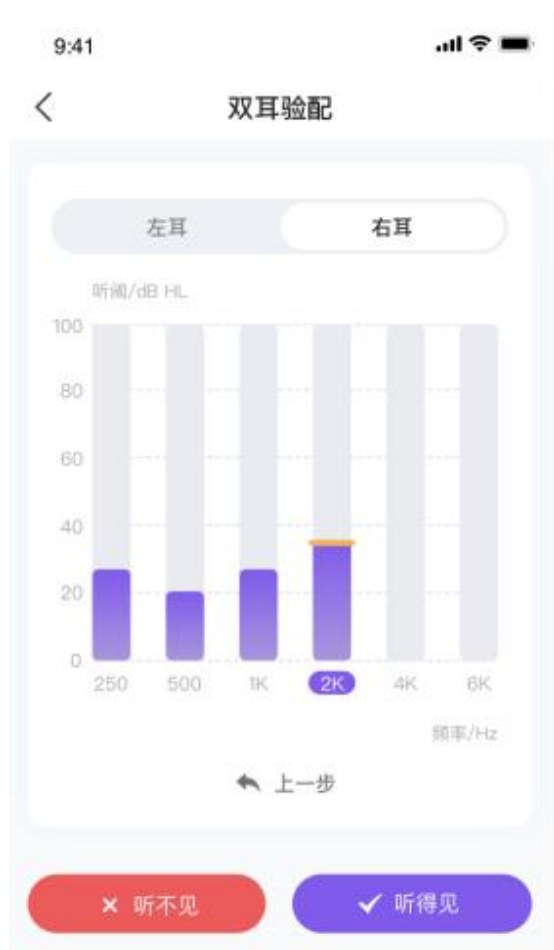
(4) 选择要验配的耳机之后，会进入到验配的界面，这个界面会下发一些声音增益和频率信息到耳机，耳机会根据接收到的信息，播放对应频率的增益大小的单频音，听力障碍者只需要根据实际是否能听到声音点击 APP 上的“听得见”或者“听不见”按钮即可；

App 端发下来的增益和频率信息如下，在 `audio_hearing_aid.c` 文件的 `int hearing_aid_fitting_parse(u8 *data, u16 len)` 函数里面进行解析。

```
1. /*
2.  * 1. Payload Format
3.  * +-----+-----+-----+
4.  * |cmd[8bit]|data_len[16bit]| data |
```



```
5.  * +-----+-----+-----+
6.  * 2.DHA Fitting Commands
7.  */
8.  /*(2)通道验配*/
9.  #define DHA_FITTING_CMD_ADJUST      0x51
10. typedef struct {
11.     u8 channel: 1;    /*左右声道标识:0=左声道 1=右声道*/
12.     u8 sine: 2;       /*开关验配单频音的标志: BIT(0) = 1 左耳播单频音, BIT(0) = 0 左耳静音
13.                                     BIT(1) = 1 右耳播单频音, BIT(1) = 0 右耳静音*/
14.     u8 reserve0: 5;    /*保留*/
15.     u8 reserve1;       /*保留*/
16.     u16 freq;          /*通道频率*/
17.     float gain;        /*通道增益*/
18. } dha_fitting_adjust_t;
```



(5) 当验配结束后，会进到听力阈值显示界面，点击“保存”按钮，APP 会把该次验配的听力阈值参数保存到手机 APP 本地，当点击“写入设备”按钮后，APP 会把听力阈值的数据发给耳机使用并保存到 vm 里面，下次打开辅听功能时，耳机会把 vm 里面听力阈值数据读出来使用；

耳机接收到的听力阈值数据格式如下，数据段第一个字节是左耳或/右耳/双耳表示，后面是对应不同频率的 float 类型的阈值增益

```
1. +-----+-----+-----+-----+-----+
2. |cmd[8bit]|len[16bit] | type(8bit)左/右/双耳|N个左耳[float]| N个右耳[float] |
3. +-----+-----+-----+-----+-----+
```

保存读取验配听力阈值参数的相关函数如下

```
1. /*
2.  * 保存验配参数
3.  * LR: 0 保存左耳, 1 保存右耳, 2 保存双耳
4.  * buf: 需要保存的参数
```

```

5.  */
6.  int fitting_param_write(u8 LR, void *buf);
7.
8.  /*读取当前耳机(左耳/右耳)验配参数*/
9.  int fitting_param_read(void *buf);
10.
11. /*清除验配参数
12.  * LR: 0 清除左耳, 1 清除右耳, 2 清除对耳
13.  */
14. int clear_fitting_patam(u8 LR);

```



2、使用第三方 APP 验配

如果要开发第三方的听力验配 APP，可以使用杰理提供的收发数据接口进行开发。

(1) 在 `adv_hearing_aid_setting.h` 函数里面使能第三方 APP 接入的宏 `ASSISTED_HEARING_CUSTOM_TRASNDATA`

(2) 使用杰理提供的数据接收函数

接收到的数据在 `adv_hearing_aid_setting.c` 的 `hearing_aid_update()`函数里面，需要替换杰理之家 APP 的验配解析函数 `hearing_aid_update_handle(g_dha_fitting_data, data_len);`

```
1. static void hearing_aid_update(void)
2. {
3.     // 跟进不同命令类型调用设置函数
4.     #if ASSISTED_HEARING_CUSTOM_TRASNDATA
5.         // 从机才触发
6.         u16 data_len = get_adv_sync_tws_len();
7.     #else
8.         u16 data_len = adv_hearing_aid_convert(g_dha_fitting_data);
9.     #endif
10.    put_buf(g_dha_fitting_data, data_len);
11.    hearing_aid_update_handle(g_dha_fitting_data, data_len); //需要替换的解析函数
12. }
```

(3) 使用杰理提供的数据发送函数

```
1. int app_third_party_hearing_aid_resp(u8 *data, u16 len);
```

注意：使用 RCSP 协议，收到的每包数据都需要回复。

4.11.6 独立辅听+低功耗模式

只要使能对应的宏，就能在开机的时候进入独立的辅听模式以及检测到 MIC 采到的能量比较小的时候进入低功耗模式,直到 mic 采到的能量到达某一个阈值的时候才退出低功耗。

(1)在 `board_config.h` 中选择独立辅听模式对应的板级以及在此板级中开启对应的宏:

```
#define CONFIG_BOARD_AC700N_HEARING_AID
```

使能独立辅听模式和 根据 mic 采到的声音能量进出低功耗的宏如下:

/*辅听功能使能*/

```
#define TCFG_AUDIO_HEARING_AID_ENABLE    ENABLE_THIS_MOUDLE
```

/*辅听 VAD 低功耗模式：仅独立辅听模式有效*/

```
#define TCFG_AUDIO_DHA_LOW_POWER_ENABLE    ENABLE
```

(2)根据 mic 采到的能量进出低功耗在 audio_hearing_aid_lp.c 文件中有如下可配置宏:

```
1. //*****//
2. //          mic 能量检测处理配置          //
3. //*****//
4. //是否开启 mic 能量检测
5. #define MIC_ENERGY_DETECT 1
6.
7. //判断多少 ms 没有能量就进入低功耗
8. //define MIC_SNIFF_TIME 2 * 60 * 1000
9. #define MIC_SNIFF_TIME 5 * 1000
10.
11. //进入低功耗后每隔多少 ms 计算一次能量
12. #define MIC_CHECK_INTER 1 * 1000
13.
14. //低功耗状态下能量检测计算多少 ms 的能量
15. #define MIC_ENERGY_TIME 20
16.
17. //进入低功耗的能量值,小于此能量值进入低功耗
18. #define enter_sniff_energy 50
19.
20. //退出低功耗的能量值,大于退出低功耗 //从低功耗出来时 adc 能量值会比较高,需要比较高的阈值
21. #define exit_sniff_energy 180
22.
23. //跳过刚上电的多少次中断数据
24. #define adc_isr_ignore_cnt 8
25.
26. //是否打开 mic 能量的调试打印
27. #define ENERGY_DEBUG_EN 0
```

4.12 混响 K 歌配置

```
1. //*****//
2. //          混响配置          //
```

```

3.  /*******//
4.  //混响使能
5.  #define TCFG_MIC_EFFECT_ENABLE          DISABLE
6.  #define TCFG_MIC_EFFECT_START_DIR      DISABLE //开机直接启动混响
7.
8.  //混响效果配置
9.  #define MIC_EFFECT_REVERB              1 //混响
10. #define MIC_EFFECT_ECHO                2 //回声
11. #define MIC_EFFECT_REVERB_ECHO        3 //混响+回声
12. #define MIC_EFFECT_MEGAPHONE          4 //扩音器/大声公
13. #define TCFG_MIC_EFFECT_SEL            MIC_EFFECT_ECHO
14.
15. //混响高低音
16. #define TCFG_MIC_TUNNING_EQ_ENABLE     DISABLE
17.
18. //混响采样率
19. #define TCFG_REVERB_SAMPLERATE_DEFUAL  (44100)
20. #define MIC_EFFECT_SAMPLERATE          (44100L)
21.
22. //混响使用数字音量，支持混响和音乐独立调节音量
23. #if TCFG_MIC_EFFECT_ENABLE
24. #undef SYS_VOL_TYPE
25. #define SYS_VOL_TYPE                    VOL_TYPE_DIGITAL
26. #endif/*TCFG_MIC_EFFECT_ENABLE*/

```

4.13 LINEIN 功能配置

做 Linein + ANC 功能时,ANC 需要 2 个 ADC 采集口给 mic 使用,linein 需要 2 个 ADC 采集口。一块芯片上 ADC 硬件资源不足,因此通过外挂一个 AC700N 系列的芯片来实现。其中一颗芯片作为 iis 从机,作用是通过采集 linein adc 信号然后通过 iis 通信协议发送给另一颗芯片,另一颗芯片也就是 iis 主机。

软件上会通过检测 linein 引脚的插入来进行对应模式的选择。当插入 linein 线时便会切到 linein 模式,当拔出 linein 线时便退出 linein 模式。在退出 linein 模式后,从机会休眠来降低功耗,直到再

次进入 linein 模式。

代码配置上，只需要配置对应的宏便可以开启 linein 功能。

1) 在 board_config.h 中选择 linein anc 模式对应的板级以及在此板级中开启对应的宏。

```
#define CONFIG_BOARD_AC700N_IIS_LINEIN
```

2) 在对应的配置文件 “board_ac700n_iis_linein_global_build_cfg.h” 里有如下的配置：

```
1.  //*****  
2.          ADC_IIS 配置  
3.  //*****  
4.  //是否开机进入 LINEIN 模式  
5.  #define TCFG_APP_LINEIN_EN          ENABLE  
6.  //LINEIN PCM 数据是否走解码  
7.  #define TCFG_LINEIN_PCM_DECODER      ENABLE  
8.  //是否使能 IIS 收发 linein 数据功能  
9.  #define TCFG_ADC_IIS_ENABLE          ENABLE  
10. #define ROLE_M          1  
11. #define ROLE_S          0  
12. //选择作为 iis 主机还是从机  
13. #define IIS_ROLE          ROLE_S  
14. //定义 linein 检测引脚  
15. #define LINEIN_DET_PORT      IO_PORT_DM  
16. #define LINEIN_DET_CH        AD_CH_DM
```

其中，只需要修改 IIS_ROLE 为 ROLE_S(从机)或者 ROLE_M(主机)，分别下载入对应的芯片即可。

3) 配置为 iis 主机时，默认已经打开 ANC 配置。在配置为 IIS 从机时，默认已经配置为省晶振的配置。

若想修改为不省晶振的配置，可以在文件 “board_ac700n_iis_linein_cfg.h” 里配置：

```
#define TCFG_CLOCK_SYS_SRC  SYS_CLOCK_INPUT_PLL_BT_OSC
```

在文件 “board_ac700n_iis_linein_global_build_cfg.h”下配置：

```
#define CONFIG_PLL_SOURCE_USING_LRC 1 //PLL 时钟源选择 1:LRC 2:OSC
```

便是配置为不省晶振的模式。

4.14 音效配置

4.14.1 虚拟低音

虚拟低音通过心理声学增强低音表现，把低频段的谐波做了增强，适合受限于腔体喇叭等因素无法重现更低频率的情况。

1、SDK 配置

```
1. #define TCFG_EQ_ONLINE_ENABLE           1 //打开在线音效调试功能
2. #define TCFG_EFFECT_TOOL_ENABLE        1 //打开音效调试工具宏
3. //选择在线调的通信方式，可选 USB、UART、SPP
4. #define TCFG_COMM_TYPE                  TCFG_USB_COMM
5. #define AUDIO_VBASS_CONFIG              1 //打开虚拟低音
```

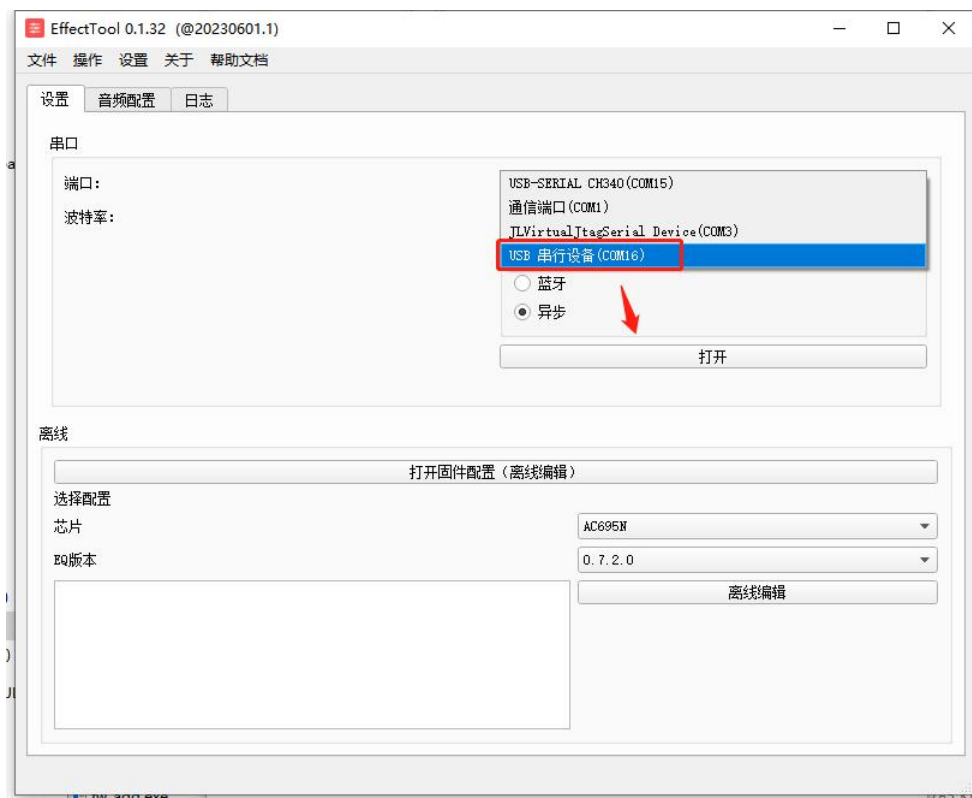
2、工具配置

下面以 USB 通讯为例，演示如何打开虚拟低音在线调试。

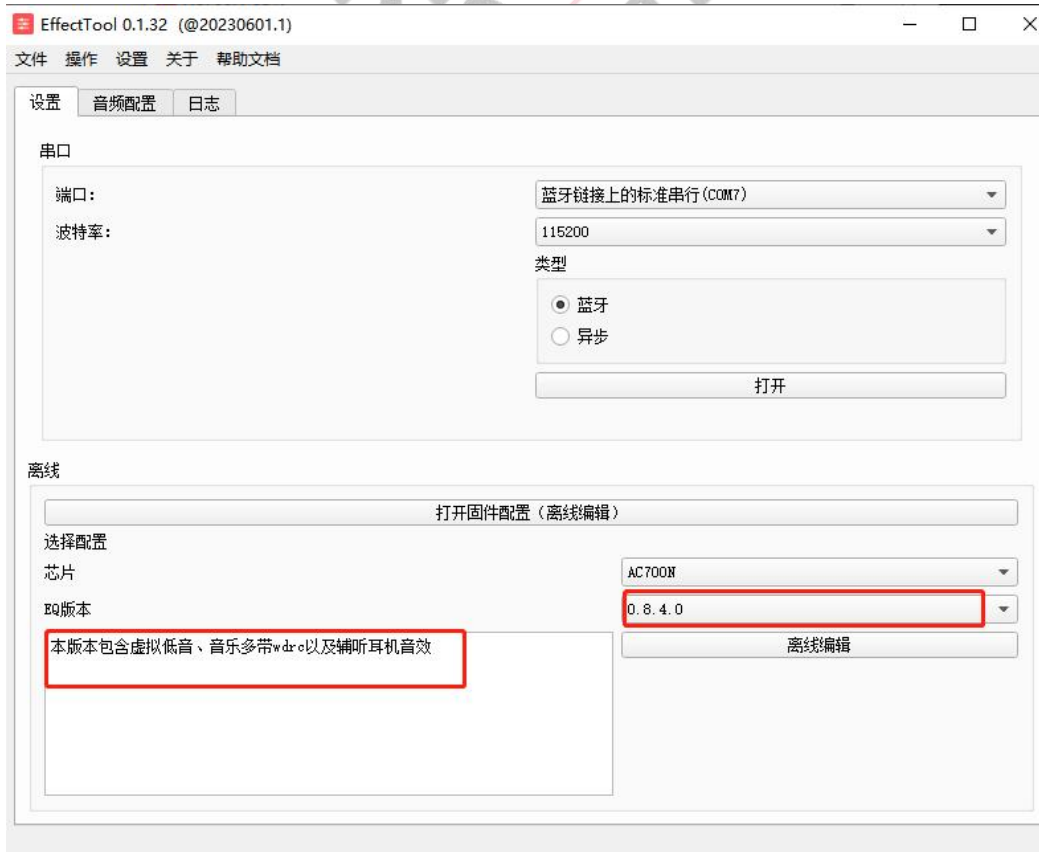
(1) 打开下载目录中的 **AC700N_配置工具入口(Config Tools Entry).jlxproj**，选择调音工具。



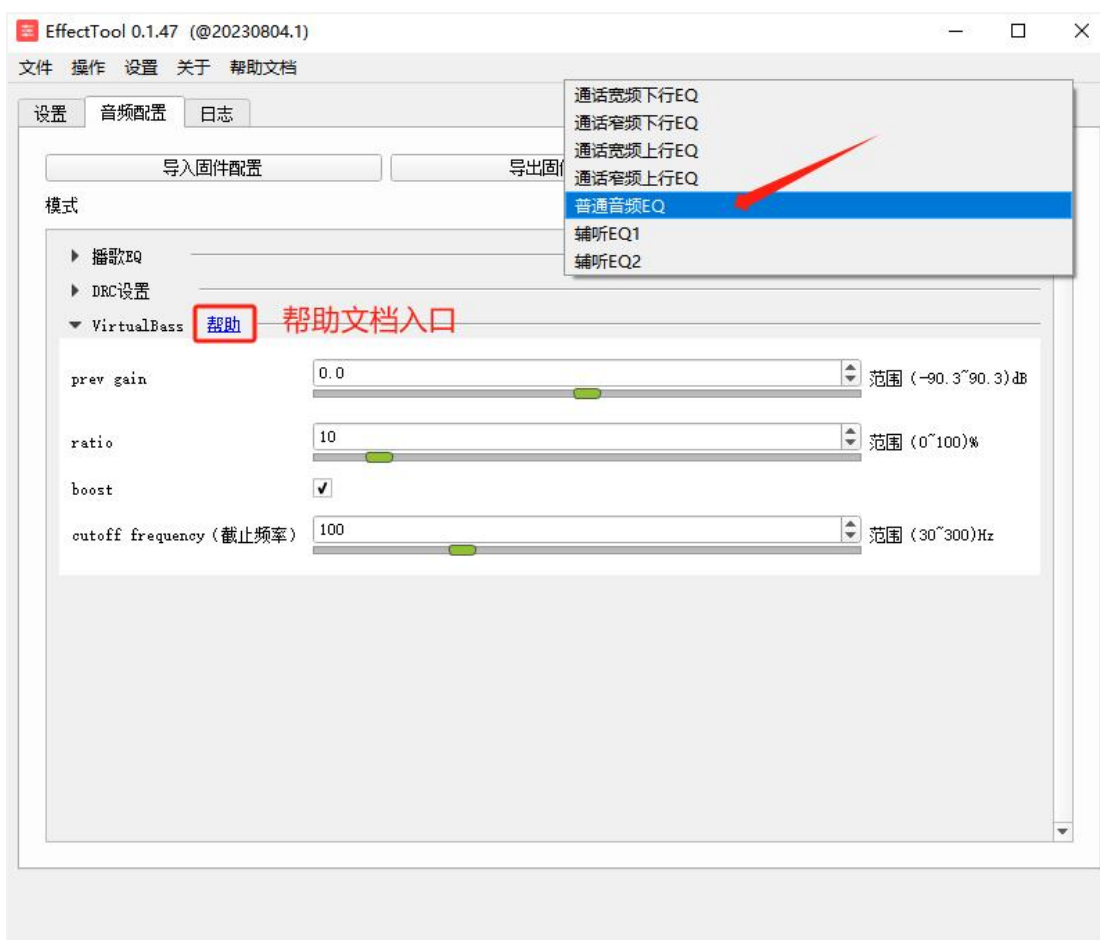
(2) 选择 **USB 串行设备** 后点击**打开**



(4) 离线编辑



(5) 选择**普通音频 EQ**，即可进入到 VirtualBass 在线调试界面



4.15 第三方音效算法接入

4.15.1 音乐播放音效算法

音乐播放，主要是蓝牙高级音频（A2DP）的音效算法接入到 SDK 流程分三步处理。

1、使能第三方音效算法开发

宏定义主要用于控制音效算法接入的代码块打开与裁剪，通常将宏定义设置在板级头文件中，例如：board_ac700n_demo_cfg.h

```
1.  /*******//
2.  //          第三方音效算法配置          //
3.  /*******//
4.  #define TCFG_EFFECT_DEVELOP_ENABLE    1
```

2、音效算法实现接口

蓝牙高级音频（A2DP）的主流程打开(初始化)、关闭以及效果处理代码位于 **cpu/br36/audio/audio_effect_develop.c** 文件中，其中 **a2dp_dec_start()**函数集成解码初始化/打开和音效模块初始化/打开，第三方音效算法同样在此配置相关的初始化与打开，示例：

(1) 算法初始化

```
1.  int a2dp_dec_start()
2.  {
3.      .....
4.      a2dp_decoder_syncts_setup(dec);
5.      a2dp_decoder_plc_setup(dec);
6.
7.      /*第三方音效算法加入与开发*/
8.      a2dp_develop_effect_setup(dec);
9.
10.     a2dp_drop_frame_stop();
11.     dec->remain = 0;
12.     a2dp_decoder_match_system_clock(dec);
13.     /* dec->state = A2DP_STREAM_START; */
14.     err = audio_decoder_start(&dec->decoder);
15.     .....
16. }
```

(2) 音效关闭

```
1.  static int __a2dp_audio_res_close(void)
2.  {
3.      .....
4.      a2dp_decoder_syncts_free(a2dp_dec);
```

```
5.     a2dp_decoder_plc_free(a2dp_dec);
6.     a2dp_develop_effect_close(a2dp_dec);
7.     .....
8. }
```

(3) 音效算法处理函数

音效算法的主流程通常在 `a2dp_dec_output_handler()` 函数中处理，数据同步进出的添加示例为（仅截取上下文部分参考）：

```
1. static int a2dp_dec_output_handler(struct audio_decoder *decoder, s16 *data,
2. int len, void *priv)
3. {
4.     .....
5.     #if TCFG_EQ_ENABLE&&TCFG_BT_MUSIC_EQ_ENABLE
6.         if (dec->eq_drc && !dec->eq_drc->async) { //同步方式 eq
7.             eq_drc_run(dec->eq_drc, data, len);
8.         }
9.     #endif //TCFG_BT_MUSIC_EQ_ENABLE
10.    #if ((defined TCFG_EFFECT_DEVELOP_ENABLE) && TCFG_EFFECT_DEVELOP_ENABLE)
11.        effect_develop_data_handler(dec, data, len);
12.    #endif
13.    a2dp_output_before_syncts_handler(dec, data, len);
14.    .....
15. }
```

(4) 接口添加示例

```
1. /*****
2.  * 音效算法开发处理函数
3.  * Input    : priv - 播放流程的主要私有句柄
4.               data - pcm 数据(16bit)
5.               len  - pcm 数据的 byte 长度
6.  * Output   : 数据处理长度
7.  * Notes    :
8.  * History   :
9.  *=====*/
10. static int effect_develop_data_handler(void *priv, void *data, int len)
11. {
12.     //TODO : 音效运行函数
13.     struct a2dp_dec_hdl *dec = (struct a2dp_dec_hdl *)priv;
14.     u8 nch = dec->ch; //声道数
15.     int sample_rate = dec->sample_rate; //采样率
16. }
```

```
17. //    putchar('#');
18.    return len;
19. }
20.
21. static void a2dp_develop_effect_setup(struct a2dp_dec_hdl *dec)
22. {
23.     #if ((defined TCFG_EFFECT_DEVELOP_ENABLE) && TCFG_EFFECT_DEVELOP_ENABLE)
24.         //TODO : 打开音效
25.     #endif
26. }
27.
28. static void a2dp_develop_effect_close(struct a2dp_dec_hdl *dec)
29. {
30.     #if ((defined TCFG_EFFECT_DEVELOP_ENABLE) && TCFG_EFFECT_DEVELOP_ENABLE)
31.
32.     #endif
33. }
```

4.15.2 算法移植 FAQ

1、资源跟踪

(1) 内存跟踪接口

```
1. mem_stats();
```

(2) CPU 运行频率跟踪接口

```
1. printf("clk_freq:%d\n", clk_get("sys"));
```

2、硬件加速模块

(1) FFT 加速器

使用范例：sdk\cpu\br36\audio\audio_demo\audio_fft_demo.c

注意：不支持浮点，仅支持定点运算

(2) 数学函数加速

引用路径：sdk\include_lib\system\math\cpu\br36\asm\math_fast_function.h

3、动态操作音效 bypass/enable 问题

使用 APP 层的按键或其他命令方式对效果进行 bypass 或 enable，由音效算法接口模块内部提供 enable/disable 接口的，无需在播歌流程中增加其他处理，防止引入歧义；由外部变量控制 enable/disable 的，仅需在 **effect_develop_data_handler()** 中处理即可，例如：

```
1. if (dec->effect_develop_enable) {
```

```
2.     effect_develop_run();
3. }
```

4、运行开销问题

当音效算法运行时，CPU 在当前时钟**满载或超载**后，音频播放通路必然会发生欠载，即数据流速达不到 DAC 播放速度，通常会报 **underrun** 错误或蓝牙 buffer **接收满** 的错误，打印信息为 uuuu (即 under run, DAC 欠载)、FFFF (即 full, 输入溢出)。

此时需要检查算法的运行效率，是否 CPU 算力满足，当检查算法消耗 mips 未超过 CPU 设计标准负荷，且 CPU 满载，则有可能为播歌+算法的运行开销处于当前系统时钟（频率）的边界值，那么需要提高系统时钟以确保正常运行。提升系统时钟可以在 **a2dp_dec_start()** 函数中的如下位置进行设置：

```
/*第三方音效算法加入与开发*/
a2dp_develop_effect_setup(dec);

a2dp_drop_frame_stop();
dec->remain = 0;
a2dp_decoder_match_system_clock(dec);
clk_set("sys", 128000000L);
/* dec->state = A2DP_STREAM_START; */
err = audio_decoder_start(&dec->decoder);
if (err) {
    goto __err3;
}
```

5、算法运行时间估算

```
1. unsigned long start_ts = jiffies_msec(); //开始时间戳
2. effcet_run(); //估算运算时间的模块
3. unsigned long end_ts = jiffies_msec(); //结束时间戳
4. //如果出现 end_ts 小于 start_ts, 表示时间 jiffies 计算溢出, 当次统计无效
5. unsigned long run_time = end_ts - start_ts;
6. printf("run_time:%d ms\n",run_time);
```

6、代码异常 (exception)

音效算法运行出现 CPU 异常，有几种情况需要注意：

1) watch_dog 异常：通常由于算法执行了 CPU 设计标志之外的 mips 开销导致 CPU 超载导致异常；

2) cpu read/write mmu 异常：通常为算法内部中对数据处理读/写越界

Chapter 5 TWS 功能使用说明

5.1 编写目的

该部分文档主要描述 TWS 功能的使用方法及开发中注意的一些问题,为用户进行二次开发提供参考,其中包括:

TWS 一些现有功能的宏定义、同步消息的做法、以及一些 TWS 相关接口函数。

5.2 TWS 的开关

```
/*
 * 连接设备个数配置 (e.g., 1拖2)
 * 支持连接1台或者2台
 */
#define TCFG_BD_NUM 1

/*
 * 配置一拖二第二台连接后自动关闭PAGE_SCAN的时间 (单位分钟)
 */
#define TCFG_AUTO_STOP_PAGE_SCAN_TIME 0

/*
 * TWS配置 (tws和1t2是互斥的)
 */
#define TCFG_USER_TWS_ENABLE 1 //tws功能使能
```

在每一个板级的头文件中,都有一个开关对耳功能的宏定义。需要注意的是,对耳和一拖二功能不能同时开启。

5.3 TWS 配对方式选择

TWS 配对和声道选择方式如下图所示：(apps/earphone/app_config.h)

```

95 // 对耳配置方式配置 //
96 // ***** //
97 #define CONFIG_TWS_CONNECT_SIBLING_TIMEOUT 4 /* 开机或超时断开后对耳互连超时时间, 单位s */
98 #define CONFIG_TWS_REMOVE_PAIR_ENABLE /* 不连手机的情况下双击按键删除配对信息 */
99 #define CONFIG_TWS_POWEROFF_SAME_TIME 1 /* 按键关机时两个耳机同时关机 */
100
101 #define ONE_KEY_CTL_DIFF_FUNC 1 /* 通过左右耳实现一个按键控制两个功能 */
102 #define CONFIG_TWS_SCO_ONLY_MASTER 0 /* 通话的时候只有主机出声音 */
103
104 /* 配对方式选择 */
105 #define CONFIG_TWS_PAIR_BY_CLICK 0 /* 按键发起配对 */
106 #define CONFIG_TWS_PAIR_BY_AUTO 1 /* 开机自动配对 */
107 #define CONFIG_TWS_PAIR_BY_FAST_CONN 2 /* 开机快速连接, 连接速度比自动配对快, 不支持取消配对操作 */
108 #define CONFIG_TWS_PAIR_MODE CONFIG_TWS_PAIR_BY_AUTO
109
110
111 /* 声道确定方式选择 */
112 #define CONFIG_TWS_MASTER_AS_LEFT 0 // 主机作为左耳
113 #define CONFIG_TWS_AS_LEFT_CHANNEL 1 // 固定左耳
114 #define CONFIG_TWS_AS_RIGHT_CHANNEL 2 // 固定右耳
115 #define CONFIG_TWS_LEFT_START_PAIR 3 // 双击发起配对的耳机做左耳
116 #define CONFIG_TWS_RIGHT_START_PAIR 4 // 双击发起配对的耳机做右耳
117 #define CONFIG_TWS_EXTERN_UP_AS_LEFT 5 // 外部有上拉电阻作为左耳
118 #define CONFIG_TWS_EXTERN_DOWN_AS_LEFT 6 // 外部有下拉电阻作为左耳
119 #define CONFIG_TWS_SECECT_BY_CHARGESTORE 7 // 充电仓决定左右耳
120 #define CONFIG_TWS_CHANNEL_SELECT CONFIG_TWS_LEFT_START_PAIR // 配对方式选择
121
122 #define CONFIG_TWS_CHANNEL_CHECK_IO IO_PORTA_07 // 上下拉电阻检测引脚
123
124
125 #if CONFIG_TWS_PAIR_MODE != CONFIG_TWS_PAIR_BY_CLICK
126 #if (CONFIG_TWS_CHANNEL_SELECT == CONFIG_TWS_LEFT_START_PAIR) || \
127     (CONFIG_TWS_CHANNEL_SELECT == CONFIG_TWS_RIGHT_START_PAIR)
128 #undef CONFIG_TWS_CHANNEL_SELECT
129 #define CONFIG_TWS_CHANNEL_SELECT CONFIG_TWS_MASTER_AS_LEFT
130 #endif
131
132 #if CONFIG_TWS_PAIR_MODE == CONFIG_TWS_PAIR_BY_AUTO
133 #define CONFIG_TWS_AUTO_PAIR_WITHOUT_UNPAIR /* 不取消配对也可以配对新的耳机 */
134 #endif

```

左右声道选择主要有以下几种情况：

(1) 智能充电仓选择

将 CONFIG_TWS_CHANNEL_SELECT 选择为 CONFIG_TWS_SECECT_BY_CHARGESTORE，耳机放入充电仓之后，通过充电仓分配左右耳。

(2) 程序固定左右耳

将 CONFIG_TWS_CHANNEL_SELECT 选择为 CONFIG_TWS_AS_LEFT_CHANNEL，则对耳连接成功时，该耳机作为左声道。固定右声道则选择为 CONFIG_TWS_AS_RIGHT_CHANNEL，要求左右声道为两套程序。

(3) 通过上下拉电阻决定左右耳

将 CONFIG_TWS_CHANNEL_SELECT 选择为 CONFIG_TWS_EXTERN_UP_AS_LEFT，并且把 CONFIG_TWS_CHANNEL_CHECK_IO 改为检测外部电阻的 io 口，则贴了外部上拉电阻的耳机作为左声道，没贴的作为右声道。

选择 CONFIG_TWS_EXTERN_DOWN_AS_LEFT 的话，外部贴了下拉电阻的为左声道，没贴的为右声道。

(4) 双击发起配对的作为左声道

将 CONFIG_TWS_CHANNEL_SELECT 选择为 CONFIG_TWS_LEFT_START_PAIR, 则通过双击发起配对的那个耳机作为左声道, 被配对上的为右声道。

选择 CONFIG_TWS_RIGHT_START_PAIR 的话, 双击发起配对的是右声道。

(5) 自动配对时, 主机作为左声道

如果没有固定左右耳或者通过上下拉区分左右耳的话, 自动配对中, 主机将作为左声道, 从机作为右声道, 不会随着主从切换而改变。

注: 自动配对可以和固定左右、上下拉电阻区分左右这两种方式结合, 在上电后自动配对, 左右耳由 CONFIG_TWS_CHANNEL_SELECT 宏决定。智能充电仓无需配自动配对的宏, 开机后将会自动配对。

TWS 配对后的 MAC 地址可选则以下两种方式: (app/earphone/app_config.h)

```
89
90 #define CONFIG_TWS_COMMON_ADDR_AUTO          0      /* 自动生成TWS配对后的MAC地址 */
91 #define CONFIG_TWS_COMMON_ADDR_USED_LEFT      1      /* 使用左耳的MAC地址作为TWS配对后的地址
92                                                         可配合烧写器MAC地址自增功能一起使用
93                                                         ||多台交叉配对会出现MAC地址相同情况 */
94 #define CONFIG_TWS_COMMON_ADDR_SELECT        CONFIG_TWS_COMMON_ADDR_AUTO
95
```

5.4 TWS 消息同步

TWS 消息同步可以用来同步播放连接提示音、来电提示音、同步 LED 灯。

消息同步可以参考提示音同步的步骤，添加消息同步的步骤如下：

(1) TWS 主从同步调用指定函数

```
int tws_api_sync_call_by_uuid(int uuid, int priv, int delay_ms);
```

TWS 消息同步需要用到 `tws_api_sync_call_by_uuid` 接口。第一个参数为唯一的 id 号，需要和系统中已存在的不重复，第二个参数是要传递的参数，第三个参数是多少毫秒后同步执行。

示例：

```
Void tws_sync_call_fun1(int priv)
{
}

TWS_SYNC_CALL_REGISTER(tws_tone_sync) = {
    .uuid = 0x12345678,
    .func = tws_sync_call_fun1,
};

Tws_pair_sync_call_by_uuid(0x12345678, 0x30, 300);
```

5.5 TWS 互发数据接口

Sdk 为用户提供了两个 tws 设备互发数据接口

发送接口：

Data:数据 buf len: 数据长度，大小不能超过 512byte

```
int tws_api_send_data_to_sibling(u8 *data, u16 len)
```

接收接口：

```
void tws_user_data_handler(u8 *data, u16 len)
```

接收数据会通过该函数回调上来,该函数在中断中调用，请勿执行任何可能导致系统睡眠的操作

5.6 siri 状态的获取

调用获取 siri 状态命令，调用函数 `user_send_cmd_prepare(USER_CTRL_HFP_GET_SIRI_STATUS, 0, NULL)`

Siri 状态会通过 `BT_STATUS_VOICE_RECOGNITION` 状态返回，在 `bt_connction_status_event_handler` 函数里面增加一个 case

Case BT_STATUS_VOICE_RECOGNITION:

```
Siri_status = bt->value;
```

```
Break;
```

获取到的 Siri_status 就是 siri 的状态

5.7 三方通话接口

三方通话提供的命令

//应答

USER_CTRL_HFP_THREE_WAY_ANSWER1, //挂断当前去听另一个(未接听或者在保留状态都可以)

USER_CTRL_HFP_THREE_WAY_ANSWER2, //保留当前去接听, 或者用于两个通话的切换

USER_CTRL_HFP_THREE_WAY_ANSWER1X,

USER_CTRL_HFP_THREE_WAY_ANSWER2X,

//拒听

USER_CTRL_HFP_THREE_WAY_REJECT, //拒绝后台来电

通过 user_send_cmd_prepare 函数发送对应的命令即可, 获取三方通话状态函数

Extern u8 get_second_call_status() 函数使用即可

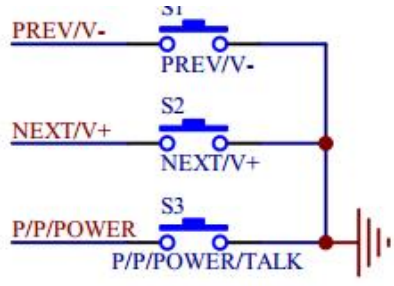
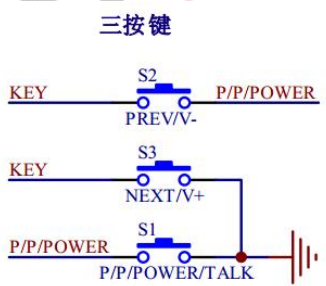
Chapter 6 常用设备使用介绍

6.1 编写目的

该文档主要描述 AC700N_SDK 一些常用设备使用以及注意的一些问题，为用户进行二次开发提供参考

6.2 IOKEY 使用

IOKEY 驱动支持两种连接方式：

1) 一个按键接一个 IO;	2) 一个按键接两个 IO (互推接法);
	

IOKEY 参数在板级配置文件中 (c 文件和 h 文件) 进行配置，如 board_ac6936a_tws.c 和 board_ac6936a_tws_cfg.h，在 h 文件中可以打开 TCFG_IOKEY_ENABLE 宏和结构配置 (IO 口和按键连接方式) 相关参数，配置结构体参数说明：

配置一个按键只需填充该结构体：

```
struct iokey_port {
    u8 connect_way;
    union key_type key_type;
    u8 key_value;
};
```

参数名	描述
connect_way	连接方式支持三种： ➤ ONE_PORT_TO_LOW: 按键一个端口接地， 另一个端口接 IO; ➤ ONE_PORT_TO_HIGH: 按键一个端口接 3.3v， 另一个端口接 IO; ➤ DOUBLE_PORT_TO_IO: 按键两个端口接 IO， 互推方式。
key_type	key_type 结构依据 connect_way 而定， 如果 connect_way 配置为 ONE_PORT_TO_LOW 和 ONE_PORT_TO_HIGH 只需要配置一个 IO；而如果 connect_way 配置为 DOUBLE_PORT_TO_IO， 需要配置两个 IO；
key_value	key_value 指的是按键值， 按键被检测到被按下将发送该按键配置的 key_value 的值。

配置示例如下：

1) 一个按键接一个 IO 配置	
H 文件 配置	<pre> //*****iokey 配置*****// //*****iokey 配置*****// //*****iokey 配置*****// #define TCFG_IOKEY_ENABLE ENABLE_THIS_MOUDLE #define TCFG_IOKEY_POWER_CONNECT_WAY ONE_PORT_TO_LOW //按键一端接低电平一端接IO #define TCFG_IOKEY_POWER_ONE_PORT IO_PORTB_01 #define TCFG_IOKEY_PREV_CONNECT_WAY ONE_PORT_TO_LOW //按键一端接低电平一端接IO #define TCFG_IOKEY_PREV_ONE_PORT IO_PORTB_00 #define TCFG_IOKEY_NEXT_CONNECT_WAY ONE_PORT_TO_LOW //按键一端接低电平一端接IO #define TCFG_IOKEY_NEXT_ONE_PORT IO_PORTB_02 </pre>
C 文件 配置	<pre> /***** IO KEY *****/ #if TCFG_IOKEY_ENABLE const struct iokey_port iokey_list[] = { { .connect_way = TCFG_IOKEY_POWER_CONNECT_WAY, .key_type.one_io.port = TCFG_IOKEY_POWER_ONE_PORT, .key_value = 0, }, { .connect_way = TCFG_IOKEY_PREV_CONNECT_WAY, .key_type.one_io.port = TCFG_IOKEY_PREV_ONE_PORT, .key_value = 1, }, { .connect_way = TCFG_IOKEY_NEXT_CONNECT_WAY, .key_type.one_io.port = TCFG_IOKEY_NEXT_ONE_PORT, .key_value = 2, }, }; const struct iokey_platform_data iokey_data = { .enable = TCFG_IOKEY_ENABLE, .num = ARRAY_SIZE(iokey_list), .port = iokey_list, }; #endif </pre>
2) 一个按键接两个 IO（互推接法）	

H 文件 配置	<pre> //***** iokey 配置 *****/ //***** iokey 配置 *****/ #define TCFG_IOKEY_ENABLE ENABLE_THIS_MOUDLE #define TCFG_IOKEY_POWER_CONNECT_WAY ONE_PORT_TO_LOW //按键一端接低电平一端接IO #define TCFG_IOKEY_POWER_ONE_PORT IO_PORTB_01 #define TCFG_IOKEY_PREV_CONNECT_WAY DOUBLE_PORT_TO_IO //按键两端接IO #define TCFG_IOKEY_PREV_IN_PORT IO_PORT_DM #define TCFG_IOKEY_PREV_OUT_PORT IO_PORTB_01 #define TCFG_IOKEY_NEXT_CONNECT_WAY ONE_PORT_TO_LOW //按键一端接低电平一端接IO #define TCFG_IOKEY_NEXT_ONE_PORT IO_PORT_DM </pre>
C 文件 配置	<pre> /***** IO KEY *****/ #if TCFG_IOKEY_ENABLE const struct iokey_port iokey_list[] = { { .connect_way = TCFG_IOKEY_POWER_CONNECT_WAY, .key_type.one_io.port = TCFG_IOKEY_POWER_ONE_PORT, .key_value = 0, }, { .connect_way = TCFG_IOKEY_PREV_CONNECT_WAY, .key_type.two_io.out_port = TCFG_IOKEY_PREV_OUT_PORT, .key_type.two_io.in_port = TCFG_IOKEY_PREV_IN_PORT, .key_value = 1, }, { .connect_way = TCFG_IOKEY_NEXT_CONNECT_WAY, .key_type.one_io.port = TCFG_IOKEY_NEXT_ONE_PORT, .key_value = 2, }, }; const struct iokey_platform_data iokey_data = { .enable = TCFG_IOKEY_ENABLE, .num = ARRAY_SIZE(iokey_list), .port = iokey_list, }; #endif </pre>

6.3 ADKEY 使用

ADKEY 配置与 IOKEY 配置方式类似，在相应 H 文件和 C 文件中配置；
配置示例如下：

H 文件 配置	<pre> // **** adkey 配置 **** // **** // **** #define TCFG_ADKEY_ENABLE ENABLE_THIS_MOUDLE #define TCFG_ADKEY_PORT IO_PORTB_01 #define TCFG_ADKEY_AD_CHANNEL AD_CH_PB1 #define TCFG_ADKEY_EXTERN_UP_ENABLE ENABLE_THIS_MOUDLE #if TCFG_ADKEY_EXTERN_UP_ENABLE //是否使用外部上拉 #define R_UP 220 //22K，外部上拉阻值在此自行设置 #else #define R_UP 100 //10K，内部上拉默认10K #endif //必须从小到大填电阻，没有则同VDDIO，填0x3ffL #define TCFG_ADKEY_AD0 (0) //0R #define TCFG_ADKEY_AD1 (0x3ffL * 30 / (30 + R_UP)) //3k #define TCFG_ADKEY_AD2 (0x3ffL * 62 / (62 + R_UP)) //6.2k #define TCFG_ADKEY_AD3 (0x3ffL * 91 / (91 + R_UP)) //9.1k #define TCFG_ADKEY_AD4 (0x3ffL * 150 / (150 + R_UP)) //15k #define TCFG_ADKEY_AD5 (0x3ffL * 240 / (240 + R_UP)) //24k #define TCFG_ADKEY_AD6 (0x3ffL * 330 / (330 + R_UP)) //33k #define TCFG_ADKEY_AD7 (0x3ffL * 510 / (510 + R_UP)) //51k #define TCFG_ADKEY_AD8 (0x3ffL * 1000 / (1000 + R_UP)) //100k #define TCFG_ADKEY_AD9 (0x3ffL * 2200 / (2200 + R_UP)) //220k #define TCFG_ADKEY_VDDIO (0x3ffL) </pre>
C 文件 配置	<pre> // **** AD KEY **** // **** #if TCFG_ADKEY_ENABLE const struct adkey_platform_data adkey_data = { .enable = TCFG_ADKEY_ENABLE, .adkey_pin = TCFG_ADKEY_PORT, .ad_channel = TCFG_ADKEY_AD_CHANNEL, .extern_up_en = TCFG_ADKEY_EXTERN_UP_ENABLE, .ad_value = { TCFG_ADKEY_VOLTAGE0, TCFG_ADKEY_VOLTAGE1, TCFG_ADKEY_VOLTAGE2, TCFG_ADKEY_VOLTAGE3, TCFG_ADKEY_VOLTAGE4, TCFG_ADKEY_VOLTAGE5, TCFG_ADKEY_VOLTAGE6, TCFG_ADKEY_VOLTAGE7, TCFG_ADKEY_VOLTAGE8, TCFG_ADKEY_VOLTAGE9, }, .key_value = { TCFG_ADKEY_VALUE0, TCFG_ADKEY_VALUE1, TCFG_ADKEY_VALUE2, TCFG_ADKEY_VALUE3, TCFG_ADKEY_VALUE4, TCFG_ADKEY_VALUE5, TCFG_ADKEY_VALUE6, TCFG_ADKEY_VALUE7, TCFG_ADKEY_VALUE8, TCFG_ADKEY_VALUE9, }, }; #endif </pre>

6.4 按键扫描配置和按键消息处理

在 IOKEY 或者 ADKEY 使能后，按键扫描代码就会注册定时器定时扫描按键是否被按下，按键扫描参数可以在文件 key_driver.c 中配置，可供配置的参数如下：

<pre>//按键驱动扫描参数列表 static struct key_driver_para scan_para = { .scan_time = 10, //按键扫描频率，单位：ms .last_key = NO_KEY, //上一次get_value按键值，初始化为NO_KEY; .filter_time = 2, //按键消抖延时; .long_time = 75, //按键判定长按数量 .hold_time = (75 + 15), //按键判定HOLD数量 .click_delay_time = 20, //按键被抬起后等待连击延时数量 };</pre>	
scan_time	按键扫描频率，单位 ms，定时器将会按照设定的时间定时扫描 IOKEY 或者 ADKEY
last_key	默认初始化为 NO_KEY
filter_time	按键消抖时间，计算方式：filter_time * scan_time (ms)
long_time	按键长按事件判定时间，计算方式：long_time * scan_time (ms)
hold_time	按键按住保持事件判定时间，计算方式：hold_time * scan_time (ms)
click_delay_time	按键等待连击操作延时时间，计算方式：hold_time * scan_time (ms)，注意该参数配置会影响按键灵敏度，同时也会影响连击操作的时间间隔，所以在调试过程中需要根据需要选择一个合适的参数值；

目前在 sdk 中实现了一些按键通用事件有：

按键事件	说明
KEY_EVENT_CLICK	单击事件，在按键被按下经过 filter_time 时间后松开并经过 click_delay_time 时间后如果没有被第二次按下，按键扫描函数会判定为按键单击事件并发布出去。
KEY_EVENT_LONG	长按事件，当按键被按下经过 filter_time 时间后并一直被按下，在经过 long_time 时间后按键扫描函数会判定为按键长按事件并发布出去。
KEY_EVENT_HOLD	按下保持事件，当按键被按下经过 filter_time 时间后并一直被按下，在经过 hold_time 时间后按键扫描函数会判定为按键按下保持事件并发布出去，发布完之后如果发现按键还被按下，会在经过 hold_time - long_time 时间后再次发布按下保持事件。
KEY_EVENT_UP	抬按事件，在发送完长按事件（KEY_EVENT_LONG）和按下保持事件（KEY_EVENT_HOLD）后，如果按键被松开，按键扫描函数会发布一个抬按事件。
KEY_EVENT_DOUBLE_CLICK	双击事件，在按键被按下经过 filter_time 时间后松开并在 click_delay_time 之前同一按键再次被按下，并经过 click_delay_time 之后按键没有再次被按下，按键扫描函数会发布一个双击事件。
KEY_EVENT_TRIPLE_CLICK	三击事件，在按键被按下经过 filter_time 时间后松开并在 click_delay_time 之前同一按键再次被按下，并经过 click_delay_time 之后按键再次被按下，如果往后操作重复上述连击操作，按键扫描函数都会判定为三击事件并发布出去。

按键发布消息后，在 app 层将会收到该消息，app 层可以根据该按键消息进行相关处理，app 层的 event_handler 收到的按键消息数据格式如下：

数据格式	说明
event->type	按键消息类型为 SYS_KEY_EVENT，标记为按键消息。
event->u.key.value	按键值，该值在配置 IOKEY 或者 ADKEY 中配置，标记某一个按键被按下。
event->u.key.event	按键事件类型，对应上表中所列举的按键事件。

用户可以根据收到的按键消息进行相关处理操作。

6.5 关于按键一些特殊需求的修改说明

在 6.4 节中提供了一些通用按键配置和消息处理方式，如果这些通用的机制还不能满足用户的需求，用户可以修改下列文件：

按键底层驱动	iokey.c	iokey 底层驱动。
	adkey.c	adkey 底层驱动。
按键上层驱动	key_driver.c	
事件处理	event.h	

目前收集到的按键特殊需求和解决方案如下：

特殊需求 1：组合键

SDK 中默认只支持单击按键，用户如果需要支持组合按键，可以通过修改 key 底层驱动文件，如 iokey.c 和 adkey.c 文件，主要修改点有：

1) 修改函数 io_get_key_value()，判断按键被按下的方式，使得支持检测哪些按键被同时按下；

2) 如果是组合键，iokey_get_key_value()函数需要返回与单按键所定义的不一样的值 (ret_value)；

3) APP 上层会收到 ret_value 值，用户可以对组合键消息进行实践处理；

目前，SDK IOKEY 按键驱动中已经实现了组合键的功能，用户可以通过如下配置实现该功能

1) board_xxx_cfg.h 文件，使能组合按键配置

```
#define MULT_KEY_ENABLE 1
```

2) board_xxx.c，配置组合按键映射表


```
//组合按键消息映射表
//配置注意事项: 单个按键按键值需要按照顺序编号, 如power:0, prev:1, next:2
//bit_value = BIT(0) | BIT(1) 指按键值为0和按键值为1的两个按键被同时按下,
//remap_value = 3指当这两个按键被同时按下后重新映射的按键值;
const struct key_remap iokey_remap_table[] = {
    {.bit_value = BIT(0) | BIT(1), .remap_value = 3},
    {.bit_value = BIT(0) | BIT(2), .remap_value = 4},
    {.bit_value = BIT(1) | BIT(2), .remap_value = 5},
};

const struct key_remap_data iokey_remap_data = {
    .remap_num = ARRAY_SIZE(iokey_remap_table),
    .table = iokey_remap_table,
};
```

特殊需求 2: 按键多击事件

SDK 中默认支持双击事件和三击事件, 用户如果需要支持更多击事件, 可以修改如下文件:

- 1) event.h 文件 50 行:

```
49 enum {
50     KEY_EVENT_CLICK,
51     KEY_EVENT_LONG,
52     KEY_EVENT_HOLD,
53     KEY_EVENT_UP,
54     KEY_EVENT_DOUBLE_CLICK,
55     KEY_EVENT_TRIPLE_CLICK,
56     KEY_EVENT_MAX,
57 };
```

在箭头位置可以添加其他多击事件;

- 2) key_driver.c 文件 106 行:

```
if (scan_para.click_delay_cnt > scan_para.click_delay_time) { //按键被抬起后延时到
//TODO: 在此可以添加任意多击事件
    if (scan_para.click_cnt >= 3) {
        key_event = KEY_EVENT_TRIPLE_CLICK; //三击
    } else if (scan_para.click_cnt >= 2) {
        key_event = KEY_EVENT_DOUBLE_CLICK; //双击
    } else {
        key_event = KEY_EVENT_CLICK; //单击
    }
    key_value = scan_para.notify_value;
    goto _notify;
} else { //按键抬起后等待下次延时时间未到
```

在图中 TODO 位置可以增加任意多击事件的判断, 然后 key_event 需要等于在 event.h 文件中增加的多击事件;

- 3) 最后在 APP 层收到该多击事件进行处理即可;

特殊需求 3: 某些按键只响应单击事件

该需求可以通过按键值的某一 bit 进行特殊处理, 由于按键值 (key_value) 目前用 1byte

来表示,可支持 0~255 个按键,但在大部分应用中用不到这么多按键,因此目前 SDK 中使用按键值 (key_value) 的第 7 位进行标记,按键扫描时对标记了 bit(7)的按键不进行多击判断处理,用户如果需要应用该需求,只需要在板级配置文件 (如 board_ac6936a_tws.c 文件) 中对按键值进行标记即可:

```
{  
    .connect_way = TCFG_IOKEY_PREV_CONNECT_WAY,  
    .key_type.one_io.port = TCFG_IOKEY_PREV_ONE_PORT,  
    .key_value = 1 | BIT(7),  
},
```

其他特殊需要: 欢迎补充!

6.6 串口使用

1. const uart_bus_t *uart_dev_open(const struct uart_platform_data_t *arg)

打开一个串口设备。根据传入的引脚选择 UART0, UART1 或 UART2, 若引脚不与 UARTx 匹配, 则自动打开一个空闲的 UARTx, 并利用 OUTPUT CHANNEL 匹配 tx_pin 或 INPUT CHANNEL 匹配 rx_pin。UART0, UART1 支持 DMA, UART2 不支持 DMA, 需要填写一个循环 buffer 的地址和长度来用作 DMA 接收, 另外还需要填写一个以 ms 为单位的超时时间参数。

参数:

const struct uart_platform_data_t *arg

uart 初始化结构体, 结构体声明如下:

```
struct uart_platform_data_t {  
    u8 tx_pin;  
    ///< 作为发送引脚的引脚号, 可从参考 gpio.h 枚举中选, 当引脚为空时, 则填 -1  
    u8 rx_pin;  
    ///< 作为接收引脚的引脚号, 可从参考 gpio.h 枚举中选, 当引脚为空时, 则填 -1  
    void *rx_cbuf;  
    ///< 如果使用中断 DMA 接收, 则写入循环 buf 的首地址, ut 中断使能; 如果不使用, 则写入 NULL, 无中断  
    u32 rx_cbuf_size;  
    ///< 循环 buf 的大小, 必须为 2 的多少几次幂, 如果不用循环 buf, 该值无效, 可写 NULL  
    u32 frame_length;  
    ///< 产生 RT 中断的字节数, 如无中断, 该值无效  
    u32 rx_timeout;  
    ///< 产生 OT 中断的时间值, 单位 ms, 如无中断, 该值无效  
    ut_isr_cbfun isr_cbfun;
```

```
///< ut 中断的回调函数句柄，不用回调函数则写入 NULL，如无中断，句柄无效
void *argv;
///< ut 中断的回调函数的一个扩展形参，可供用户设定，如无回调函数，此参数无
效
u32 is_9bit: 1;
///< ut 九位模式使能位，0：关闭；1：使能
u32 baud: 24;
///< ut 的波特率
};
```

返回值：

const uart_bus_t *类型指针变量

用作打开成功的句柄，如果打开失败则返回 NULL

2. void uart_dev_close(uart_bus_t *ut)

关闭串口设备

参数：

uart_bus_t *ut

使用 uart_dev_open() 打开串口设备返回的句柄

返回值：

NULL

应用例程：

```
void uart_isr_hook(void *arg, u32 status)
{
    const uart_bus_t *uart_bus = arg;
    ...
}
void xxx()
{
    ....
    const uart_bus_t *uart_bus;
    struct uart_platform_data_t u_arg = {0};
    u_arg.tx_pin = IO_PORTA_01;
    u_arg.rx_pin = IO_PORTA_02;
    u_arg.rx_cbuf = uart_cbuf;
    u_arg.rx_cbuf_size = 512;
    u_arg.frame_length = 64;
    u_arg.rx_timeout = 100;
    u_arg.isr_cbfun = uart_isr_hook;
    u_arg.baud = 115200;
    u_arg.is_9bit = 0;
    uart_bus = uart_dev_open(&u_arg);
}
}
```


串口打开后即可正常读写数据，具体的例子请参考 `uart_test.c` 文件里的例子

6.7 IIC 接口使用

Sdk 支持软件 IIC 和硬件 IIC，具体的驱动在 `cpu/br30` 目录下的 `iic_hw.c`、`iic_soft.c` 和 `iic_eeprom_test.c` 文件里，具体的函数接口如下：

1. `int soft_iic_init(soft_iic_dev iic)`

初始化软件 IIC，通过全局结构体变量 `soft_iic_cfg` 配置 SCL，SDA，波特率等参数

参数：

`soft_iic_dev iic`

iic 句柄，类型为 typedef 的 `soft_iic_dev`

返回值：

0 成功， 非零 失败

2. `void soft_iic_uninit(soft_iic_dev iic)`

反初始化软件 IIC

参数：

`soft_iic_dev iic`

iic 句柄，类型为 typedef 的 `soft_iic_dev`

返回值：

NULL

3. `void soft_iic_suspend(soft_iic_dev iic)`

挂起软件 IIC，使引脚不输出

参数：

`soft_iic_dev iic`

iic 句柄，类型为 typedef 的 `soft_iic_dev`

返回值：

NULL

4. `void soft_iic_resume(soft_iic_dev iic)`

恢复软件 IIC，重新占用引脚

参数：

`soft_iic_dev iic`

iic 句柄，类型为 typedef 的 `soft_iic_dev`

返回值：

NULL

5. `void soft_iic_start(soft_iic_dev iic)`

发送起始位

参数：

`soft_iic_dev iic`

iic 句柄，类型为 typedef 的 `soft_iic_dev`

返回值:

NULL

6. void soft_iic_stop(soft_iic_dev iic)

发送停止位

参数:

soft_iic_dev iic

iic 句柄, 类型为 typedef 的 soft_iic_dev

返回值:

NULL

7. u8 soft_iic_tx_byte(soft_iic_dev iic, u8 byte)

发送一个字节

参数:

soft_iic_dev iic

iic 句柄, 类型为 typedef 的 soft_iic_dev

u8 byte

被发送字节

返回值:

1 IIC 从机 ACK, 0 IIC 从机 NACK

8. u8 soft_iic_rx_byte(soft_iic_dev iic, u8 ack)

接收一个字节

参数:

soft_iic_dev iic

iic 句柄, 类型为 typedef 的 soft_iic_dev

u8 ack

主机是否应答, 1 ACK, 0 NACK

返回值:

读取的字节

9. int soft_iic_write_buf(soft_iic_dev iic, const void *buf, int len)

软件 IIC 写 buffer

参数:

soft_iic_dev iic

iic 句柄, 类型为 typedef 的 soft_iic_dev

const void *buf

发送缓冲区基地址

int len

缓冲区长度

返回值:

>=0 返回实际发送长度, <0 失败

10. int soft_iic_read_buf(soft_iic_dev iic, void *buf, int len)

软件 IIC 读 buffer

参数:

soft_iic_dev iic

iic 句柄, 类型为 typedef 的 soft_iic_dev

void *buf

接收缓冲区基地址

int len

缓冲区长度

返回值:

>=0 返回实际发送长度, <0 失败

硬件 IIC

1. int hw_iic_init(hw_iic_dev iic)

初始化硬件 IIC, 通过全局结构体变量 hw_iic_cfg 配置 SCL, SDA, 波特率等参数

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

返回值:

0 成功, 非零 失败

2. void hw_iic_uninit(hw_iic_dev iic)

反初始化硬件 IIC

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

返回值:

NULL

3. void hw_iic_suspend(hw_iic_dev iic)

挂起硬件 IIC, 使引脚不输出数据

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

返回值:

NULL

4. void hw_iic_resume(hw_iic_dev iic)

恢复硬件 IIC, 重新占用引脚

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

返回值:

NULL

5. void hw_iic_start(hw_iic_dev iic)

发送起始位

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

返回值:

NULL

6. void hw_iic_stop(hw_iic_dev iic)

发送停止位

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

返回值:

NULL

7. u8 hw_iic_tx_byte(hw_iic_dev iic, u8 byte)

发送一个字节

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

u8 byte

被发送字节

返回值:

1 IIC 从机 ACK, 0 IIC 从机 NACK

8. u8 hw_iic_rx_byte(hw_iic_dev iic, u8 ack)

接收一个字节

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

u8 ack

主机是否应答, 1 ACK, 0 NACK

返回值:

接收的字节

9. int hw_iic_write_buf(hw_iic_dev iic, const void *buf, int len)

硬件 IIC 写 buffer

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

const void *buf

发送缓冲区基地址

int len

缓冲区长度

返回值:

>=0 返回实际发送长度, <0 失败

10. int hw_iic_read_buf(hw_iic_dev iic, void *buf, int len)

硬件 IIC 读 buffer

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

const void *buf

发送缓冲区基地址

int len

缓冲区长度

返回值:

>=0 返回实际发送长度, <0 失败

11. int hw_iic_set_baud(hw_iic_dev iic, u32 baud)

设置波特率

参数:

hw_iic_dev iic

iic 句柄, 类型为 typedef 的 hw_iic_dev

u32 baud

波特率

返回值:

0 成功, 非零 失败

6.8 充电仓和测试盒通信接口配置

1. 智能充电舱提供耳机配对功能, 充满后 LDO5V 电压为 0, 此时充电舱端休眠功耗为 25uA, 耳机端休眠功耗为 1.1uA。
2. 测试盒硬件带串口升级功能与芯片 LDO_IN (合绑高压 IO) 通信的接口, 可以实现自动配对, 快速连接测试, 串口升级等功能。
3. 相关宏定义解释:

```
//***** 充电仓配置 *****//
//***** 充电仓配置 *****//
//***** 充电仓配置 *****//
#define TCFG_CHARGESTORE_ENABLE    ENABLE_THIS_MOUDLE    //是否支持智能充点仓
#define TCFG_TEST_BOX_ENABLE      1
#define TCFG_CHARGESTORE_PORT      IO_PORTP_00           //耳机和充点仓通讯的IO口
#define TCFG_CHARGESTORE_UART_ID   IRQ_UART1_IDX         //通讯使用的串口号
```

TCFG_CHARGESTORE_ENABLE 宏定义为 ENABLE_THIS_MOUDLE, 则开启了智能充电舱功能;

TCFG_TEST_BOX_ENABLE 宏定义为 ENABLE_THIS_MOUDLE，则开启测试盒自动配对，快速测试及 2Mflash 版本串口升级功能；

TCFG_CHARGESTORE_PORT 该宏为和 LDOIN 绑一起的 IO，这个 IO 口是固定的不需要改动；

TCFG_CHARGESTORE_UART_ID 使用哪一组串口，只能选 UART0/1。

注意：开启充电舱功能后，需要使能拔出开机功能 TCFG_CHARGE_OFF_POWERON_NE；。

6.9 充电相关配置说明

1、充电相关宏

```
// ***** 充电参数配置 ***** //
// ***** 充电参数配置 ***** //
//是否支持芯片内置充电
#define TCFG_CHARGE_ENABLE          ENABLE_THIS_MOUDLE
//是否支持开机充电
#define TCFG_CHARGE_POWERON_ENABLE  DISABLE
//是否支持拔出充电自动开机功能
#define TCFG_CHARGE_OFF_POWERON_NE  ENABLE
//
//
//充电截止电压可选配置：
//
//CHARGE_FULL_V_3962 CHARGE_FULL_V_4002 CHARGE_FULL_V_4044 CHARGE_FULL_V_4086
//CHARGE_FULL_V_4130 CHARGE_FULL_V_4175 CHARGE_FULL_V_4222 CHARGE_FULL_V_4270
//CHARGE_FULL_V_4308 CHARGE_FULL_V_4349 CHARGE_FULL_V_4391 CHARGE_FULL_V_4434
//CHARGE_FULL_V_4472 CHARGE_FULL_V_4517 CHARGE_FULL_V_4564 CHARGE_FULL_V_4611
//
//
//#define TCFG_CHARGE_FULL_V          CHARGE_FULL_V_4222
//
//
//充电截止电流可选配置：
//
//CHARGE_FULL_mA_2 CHARGE_FULL_mA_5 CHARGE_FULL_mA_7 CHARGE_FULL_mA_10
//CHARGE_FULL_mA_15 CHARGE_FULL_mA_20 CHARGE_FULL_mA_25 CHARGE_FULL_mA_30
//
//
//#define TCFG_CHARGE_FULL_MA          CHARGE_FULL_mA_10
//
//
//充电电流可选配置：
//
//CHARGE_mA_20 CHARGE_mA_30 CHARGE_mA_40 CHARGE_mA_50
//CHARGE_mA_60 CHARGE_mA_70 CHARGE_mA_80 CHARGE_mA_90
//CHARGE_mA_100 CHARGE_mA_110 CHARGE_mA_120 CHARGE_mA_140
//CHARGE_mA_160 CHARGE_mA_180 CHARGE_mA_200 CHARGE_mA_220
//
//
//#define TCFG_CHARGE_MA          CHARGE_mA_40
```

TCFG_CHARGE_ENABLE:内置充电使能

TCFG_CHARGE_POWERON_ENABLE:开机充电使能

TCFG_CHARGE_OFF_POWERON_NE:拔出开机使能

TCFG_CHARGE_FULL_V:充满电压配置

TCFG_CHARGE_FULL_MA:充电判满电流配置

TCFG_CHARGE_MA:恒流充电电流配置

注意：LDOIN 的纹波会影响充电判满电流，纹波越大实际判满电流会比设置的判满电流小，纹波大可设置较大的判满电流

2、几个关键的配置项（在 board.c）:

```
/****** CHARGE config******/
#if TCFG_CHARGE_ENABLE
CHARGE_PLATFORM_DATA_BEGIN(charge_data)
    .charge_en = TCFG_CHARGE_ENABLE, //内置充电使能
    .charge_poweron_en = TCFG_CHARGE_POWERON_ENABLE, //是否支持充电开机
    .charge_full_v = TCFG_CHARGE_FULL_V, //充电截止电压
    .charge_full_ma = TCFG_CHARGE_FULL_MA, //充电截止电流
    .charge_ma = TCFG_CHARGE_MA, //充电电流
/*ldo5v拔出过滤值，过滤时间 = (filter*2 + 20)ms,ldoin<0.6V且时间大于过滤时间才认为拔出
对于充满直接从5V掉到0V的充电仓，该值必须设置成0，对于充满由5V先掉到0V之后再升压到xV的
充电仓，需要根据实际情况设置该值大小*/
    .ldo5v_off_filter = 100,
    .ldo5v_pulldown_lvl = CHARGE_PULLDOWN_200K, //下拉电阻档位选择
#if !TCFG_CHARGESTORE_ENABLE
//1、对于自动升压充电仓，若充电仓需要更大的负载才能检测到插入时，请将该变量置1，并且根据需求配置下拉电阻档位
//2、对于按键升压，并且是通过上拉电阻去提供维持电压的仓，请将该变量设置1，并且根据仓的上拉配置下拉需要的电阻档位
//3、对于常5V的仓，可将改变量设为0，省功耗
    .ldo5v_pulldown_en = 1,
#else
    .ldo5v_pulldown_en = 0,
#endif
CHARGE_PLATFORM_DATA_END()
#endif//TCFG_CHARGE_ENABLE
```

对于自动升压的仓，配置下拉电阻目的是使得入舱能够让充电舱开启升压。

对于人为制造维持电压的仓，配置的终极思路就是使入舱后或者充满后，LDOIN 电压在 2V 左右，避免出现在仓内 LDOIN 电压大过耳机电池电压。（需要配合充电舱端设置下拉电阻大小或者不需要使能下拉电阻）

Chapter 7 功耗说明

7.1 编写目的

该章节主要描述 AC700N 系列芯片在 ldo 和 dcdc 模式下的参考功耗,为用户进行二次开发提供参考

7.2 功耗说明

Chapter 8 透传说明

8.1 编写目的

该章节主要描述 AC700N 系列芯片开发手机 APP 功能接口，为用户进行二次开发提供参
考

8.2 文件目录说明

蓝牙协议代码：apps/common/third party profile/jieli/trans data demo

8.3 功能开关

在 `earphone/include/app config.h` 里

```
#define CONFIG_APP_BT_ENABLE    (功能总开关)
```

```
#define TRANS_DATA_EN 1 (透传功能开关)
```

```
//*****  
//                               AI配置                               //  
//*****  
// #define CONFIG_APP_BT_ENABLE  
  
#ifdef CONFIG_APP_BT_ENABLE  
#define TRANS_DATA_EN          0  
#define RCSP_BTMAE_EN         0  
#define RCSP_ADV_EN           1  
#else  
#define TRANS_DATA_EN          0  
#define RCSP_BTMAE_EN         0  
#define RCSP_ADV_EN           0  
#endif
```

8.4 SPP 数据通信

1. SPP 模块初始化

```
void transport_spp_init(void)
{
    spp_state = 0;
    spp_get_operation_table(&spp_api);
    spp_api->regist_recieve_cbk(0, transport_spp_recieve_cbk);
    spp_api->regist_state_cbk(0, transport_spp_state_cbk);
    spp_api->regist_wakeup_send(NULL, transport_spp_send_wakeup);
}
```

2. SPP 连接和断开事件处理

```
static void transport_spp_state_cbk(u8 state)
{
    spp_state = state;
    switch (state) {
        case SPP_USER_ST_CONNECT:
            log_info("SPP_USER_ST_CONNECT ~~~\n");
            break;

        case SPP_USER_ST_DISCONN:
            log_info("SPP_USER_ST_DISCONN ~~~\n");
            break;

        default:
            break;
    }
}
```

3. SPP 发送数据接口，发送前先调用接口 transport_spp_send_data_check 检查

```
int transport_spp_send_data(u8 *data, u16 len)
{
    if (spp_api) {
        log_info("spp_api_tx(%d) \n", len);
        log_info_hexdump(data, len);
        return spp_api->send_data(NULL, data, len);
    }
    return SPP_USER_ERR_SEND_FAIL;
}
```

4. SPP 检查是否可以往协议栈发送数据

```
int transport_spp_send_data_check(u16 len)
{
    if (spp_api) {
        if (spp_api->busy_state()) {
            return 0;
        }
    }
    return 1;
}
```

5. SPP 发送完成回调，表示可以继续往协议栈发数，用来触发继续发数

```
static void transport_spp_send_wakeup(void)
{
    putchar('W');
}
```

6. SPP 接收数据接口

```
static void transport_spp_recieve_cbk(void *priv, u8 *buf, u16 len)
{
    log_info("spp_api_rx(%d) \n", len);
    log_info_hexdump(buf, len);

    //loop send data for test
    if (transport_spp_send_data_check(len)) {
        transport_spp_send_data(buf, len);
    }
}
```

8.5 BLE 数据通信

1、Profile 生成的 profile_data 数据表放在 le_trans_data.h。用户可用工具 make_gatt_services 自定义修改,重新配置 GATT 服务和属性等。

```
static const uint8_t profile_data[] = {
    //////////////////////////////////////
    //
    // 0x0001 PRIMARY_SERVICE 1800
    //
    //////////////////////////////////////
    0x0a, 0x00, 0x02, 0x00, 0x01, 0x00, 0x00, 0x28, 0x00, 0x18,

    /* CHARACTERISTIC, 2a00, READ | WRITE | DYNAMIC, */
    // 0x0002 CHARACTERISTIC 2a00 READ | WRITE | DYNAMIC
    0x0d, 0x00, 0x02, 0x00, 0x02, 0x00, 0x03, 0x28, 0x0a, 0x03, 0x00, 0x00, 0x2a,
    // 0x0003 VALUE 2a00 READ | WRITE | DYNAMIC
    0x08, 0x00, 0x0a, 0x01, 0x03, 0x00, 0x00, 0x2a,

    //////////////////////////////////////
    //
    // 0x0004 PRIMARY_SERVICE ae00
    //
    //////////////////////////////////////
    0x0a, 0x00, 0x02, 0x00, 0x04, 0x00, 0x00, 0x28, 0x00, 0xae,
```

2.协议栈事件回调处理，主要是连接、断开等事件

```
/* LISTING_START(packetHandler): Packet Handler */
static void cbk_packet_handler(uint8_t packet_type, uint16_t channel, uint8_t *packet, uint16_t size)
{
    int mtu;
    u32 tmp;

    switch (packet_type) {
    case HCI_EVENT_PACKET:
        switch (hci_event_packet_get_type(packet)) {

            /* case DAEMON_EVENT_HCI_PACKET_SENT: */
            /* break; */
            case ATT_EVENT_HANDLE_VALUE_INDICATION_COMPLETE:
                log_info("ATT_EVENT_HANDLE_VALUE_INDICATION_COMPLETE\n");
            case ATT_EVENT_CAN_SEND_NOW:
                can_send_now_wakeup();
                break;

            case HCI_EVENT_LE_META:
                switch (hci_event_le_meta_get_subevent_code(packet)) {
                case HCI_SUBEVENT_LE_CONNECTION_COMPLETE: {
                    set_app_connect_type(TYPE_BLE);
                    con_handle = little_endian_read_16(packet, 4);
                    log_info("HCI_SUBEVENT_LE_CONNECTION_COMPLETE: %0x\n", con_handle);
                }
            }
        }
    }
}
```

3.ATT 读事件处理

```
static uint16_t att_read_callback(hci_con_handle_t connection_handle, uint16_t att_handle, uint16_t offset,
{
    uint16_t att_value_len = 0;
    uint16_t handle = att_handle;

    log_info("read_callback, handle= 0x%04x,buffer= %08x\n", handle, (u32)buffer);

    switch (handle) {
        case ATT_CHARACTERISTIC_2a00_01_VALUE_HANDLE:
            att_value_len = gap_device_name_len;

            if ((offset >= att_value_len) || (offset + buffer_size) > att_value_len) {
                break;
            }

            if (buffer) {
```

4.ATT 写事件处理

```
static int att_write_callback(hci_con_handle_t connection_handle, uint16_t att_handle, uint16_t transaction_mode
{
    int result = 0;
    u16 tmp16;

    u16 handle = att_handle;

    log_info("write_callback, handle= 0x%04x,size = %d\n", handle, buffer_size);

    switch (handle) {
        case ATT_CHARACTERISTIC_2a00_01_VALUE_HANDLE:
            break;

        case ATT_CHARACTERISTIC_ae02_01_CLIENT_CONFIGURATION_HANDLE:

        case ATT_CHARACTERISTIC_ae04_01_CLIENT_CONFIGURATION_HANDLE:
        case ATT_CHARACTERISTIC_ae05_01_CLIENT_CONFIGURATION_HANDLE:
            set_ble_work_state(BLE_ST_NOTIFY_INDICATE);
            check_connection_update_deal();
            log_info("\n-----write ccc:0x%04x,%02x\n", handle, buffer[0]);
```

5.NOTIFY 和 INDICATE 发送接口，发送前调接口 app_send_user_data_check 检查

```
static int app_send_user_data(u16 handle, u8 *data, u16 len,u8 handle_type)
{
    u32 ret = APP_BLE_NO_ERROR;

    if (!con_handle) {
        return APP_BLE_OPERATION_ERROR;
    }

    ret = ble_user_cmd_prepare(BLE_CMD_ATT_SEND_DATA, 4, handle, data, len,handle_type);
    if (ret == BLE_BUFFER_FULL) {
        ret = APP_BLE_BUFF_FULL;
    }

    if (ret) {
        log_info("app_send_fail:%d !!!!!\n", ret);
    }
    return ret;
}
```

6.检查是否可以往协议栈发送数据

```
static int app_send_user_data_check(u16 len)
{
    u32 buf_space = get_buffer_vaild_len(0);
    if (len <= buf_space) {
        return 1;
    }
    return 0;
}
```

7.发送完成回调，表示可以继续往协议栈发数，用来触发继续发数

```
static void can_send_now_wakeup(void)
{
    putchar('E');
    if (ble_resume_send_wakeup) {
        ble_resume_send_wakeup();
    }
}
```

8.调用示例

```
app_send_user_data(ATT_CHARACTERISTIC_ae02_01_VALUE_HANDLE, buffer, buffer_size, ATT_OP_AUTO_READ_CCC);
```

```
app_send_user_data(ATT_CHARACTERISTIC_ae05_01_VALUE_HANDLE, buffer, buffer_size, ATT_OP_AUTO_READ_CCC);
```


Chapter 9 TWS+透传流程说明

9.1 编写目的

该章节主要描述 AC700N 系列芯片 TWS+透传流程说明，为用户进行二次开发提供参考

9.2 文件目录说明

参考代码：apps/earphone/include/earphone.h
apps/earphone/trans_data_demo/trans_data_demo.c

9.3 适配接口说明

在 apps/earphone/include/earphone.h 文件里定义了 tws+透传的流程要适配的函数，以透传功能为例：

```
#define EARPONE_STATE_INIT() trans_data_earphone_state_init()
#define EARPONE_STATE_SET_PAGE_SCAN_ENABLE() trans_data_earphone_state_set_page_scan_enable()
#define EARPONE_STATE_GET_CONNECT_MAC_ADDR() trans_data_earphone_state_get_connect_mac_addr()
#define EARPONE_STATE_CANCEL_PAGE_SCAN() trans_data_earphone_state_cancel_page_scan()
#define EARPONE_STATE_ENTER_SOFT_POWEROFF() trans_data_earphone_state_enter_soft_poweroff()
#define EARPONE_STATE_TWS_INIT(a) trans_data_earphone_state_tws_init(a)
#define EARPONE_STATE_TWS_CONNECTED(a, b) trans_data_earphone_state_tws_connected(a,b)
#define SYS_EVENT_HANDLER_SPECIFIC(a) trans_data_sys_event_handler_specific(a)
```

9.4 函数接口介绍

// 透传相关结构和回调函数初始化

```
int trans_data_earphone_state_init()
{
    transport_spp_init();
    spp_data_deal_handle_register(user_spp_data_handler);
    return 0;
}
```

//退出 ble 处理函数

```
int trans_data_earphone_state_enter_soft_poweroff()
{
    extern void bt_ble_exit(void);
    bt_ble_exit();
}
```



```
    return 0;
}

//TWS 连接上时的 ble 处理流程
int trans_data_earphone_state_tws_connected(int first_pair, u8 *comm_addr)
{
    if (first_pair) {
        extern void ble_module_enable(u8 en);
        extern void bt_update_mac_addr(u8 * addr);
        extern void lib_make_ble_address(u8 * ble_address, u8 * edr_address);
        /* bt_ble_adv_enable(0); */
        u8 tmp_ble_addr[6] = {0};
        lib_make_ble_address(tmp_ble_addr, comm_addr);
        le_controller_set_mac(tmp_ble_addr); //将 ble 广播地址改成公共地址
        bt_update_mac_addr(comm_addr);
        /* bt_ble_adv_enable(1); */

        /*新的连接，公共地址改变了，要重新将新的地址广播出去*/
        if (tws_api_get_role() == TWS_ROLE_MASTER) {
            printf("\nNew Connect Master!!!\n\n");
            ble_app_disconnect();
            bt_ble_adv_enable(0);
            bt_ble_adv_enable(1);
        } else {
            printf("\nConnect Slave!!!\n\n");
            /*从机 ble 关掉*/
            ble_app_disconnect();
            bt_ble_adv_enable(0);
        }
    }

    return 0;
}

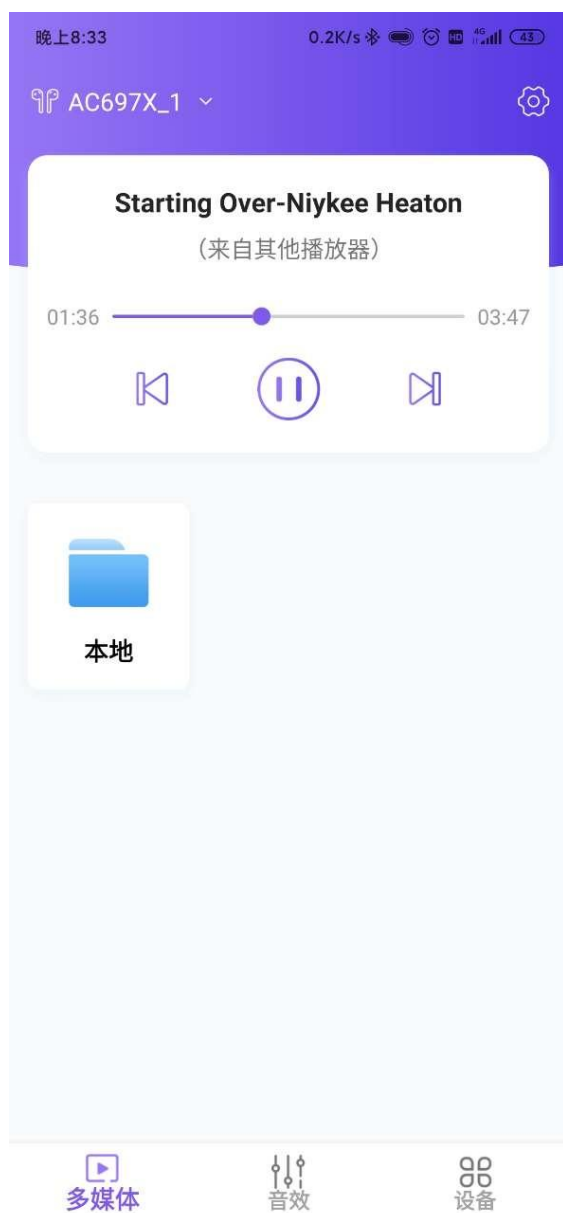
// 系统事件处理函数
int trans_data_sys_event_handler_specific(struct sys_event *event)
{
    return 0;
}
```

Chapter 10 杰理之家 APP 介绍

10.1 编写目的

该章节主要描述 AC700N 系列芯片杰理之家 APP 介绍，为用户进行二次开发提供参考

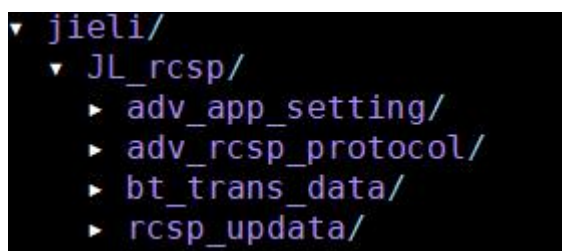
10.2 APP 介绍





10.3 文件目录说明

协议代码: apps/common/third_party_profile/jieli/JL_rcsp
apps/earphone/rcsp/



adv_app_setting //功能设置目录
adv_rcsp_protocol //rcsp 协议目录
bt_rcsp_data //透传接口目录
rcsp_updata //OTA 升级目录

10.4 功能开关、配置

在 earphone/include/app_config.h 里

#define CONFIG_APP_BT_ENABLE (功能总开关)

#define RCSP_ADV_EN 1

```
#ifndef CONFIG_APP_BT_ENABLE
#define     TRANS_DATA_EN             0
#define     RCSP_BT_MATE_EN           0
#define     RCSP_ADV_EN               1
#define     CONFIG_DOUBLE_BANK_ENABLE 0
#else
#define     TRANS_DATA_EN             0
#define     RCSP_BT_MATE_EN           0
#define     RCSP_ADV_EN               0
#endif

#if (RCSP_ADV_EN)                       //rcsp需要打开ble
#define JL_EARPHONE_APP_EN     1
// #define CONFIG_CHARGE_STORE_REMAP_ENABLE //充电仓重映射接收函数使能
#define RCSP_UPDATE_EN         1       //是否支持rcsp升级
#define OTA_TWS_SAME_TIME_ENABLE     1   //是否支持TWS同步升级
#define UPDATE_MD5_ENABLE       1       //升级是否支持MD5校验
#undef TCFG_USER_BLE_ENABLE
#define TCFG_USER_BLE_ENABLE     1       //BLE功能使能
#else
#define JL_EARPHONE_APP_EN     0
#define OTA_TWS_SAME_TIME_ENABLE     0
#define RCSP_UPDATE_EN         0
#define UPDATE_MD5_ENABLE       0       //升级是否支持MD5校验
#endif

#if RCSP_UPDATE_EN
#define APP_UPDATE_EN                 1   //需要使用APP升级的话要把该宏打开
#else
#define APP_UPDATE_EN                 0   //客户如需要开发自己的app升级协议需要把这个宏打开
#endif
```

`/rcsp功能模块使能`

```
#define RCSP_ADV_NAME_SET_ENABLE 1
#define RCSP_ADV_KEY_SET_ENABLE 1
#define RCSP_ADV_LED_SET_ENABLE 1
#define RCSP_ADV_MIC_SET_ENABLE 1
#define RCSP_ADV_WORK_SET_ENABLE 1
#if (JL_EARPHONE_APP_EN)
#define RCSP_ADV_EQ_SET_ENABLE 1
#define RCSP_ADV_MUSIC_INFO_ENABLE 1
#define RCSP_ADV_HIGH_LOW_SET 1
#else
#define RCSP_ADV_EQ_SET_ENABLE 0
#define RCSP_ADV_MUSIC_INFO_ENABLE 0
#define RCSP_ADV_HIGH_LOW_SET 0
#endif
#define RCSP_ADV_PRODUCT_MSG_ENABLE 1
```

10.5 命令、数据 API 介绍

10.5.1 已有命令

```

/*****
/*      注意：不能在这个枚举里加代码      */
/*      注意：不能在这个枚举里加代码      */
/*      注意：不能在这个枚举里加代码      */
*****/
enum {
    JL_OPCODE_DATA = 0x01,
    JL_OPCODE_GET_TARGET_FEATURE_MAP = 0x02,
    JL_OPCODE_GET_TARGET_FEATURE = 0x03,
    // JL_OPCODE_SWITCH_DEVICE = 0x04,
    JL_OPCODE_DISCONNECT_EDR = 0x06,
    JL_OPCODE_SYS_INFO_GET = 0x07,
    JL_OPCODE_SYS_INFO_SET = 0x08,
    JL_OPCODE_SYS_INFO_AUTO_UPDATE = 0x09,
    JL_OPCODE_CALL_REQUEST = 0xa,
    JL_OPCODE_SWITCH_DEVICE = 0x0b,
    JL_OPCODE_FILE_BROWSE_REQUEST_START = 0x0C,
    JL_OPCODE_FILE_BROWSE_REQUEST_STOP = 0x0D,
    JL_OPCODE_FUNCTION_CMD = 0x0E,

    JL_OPCODE_SYS_OPEN_BT_SCAN = 0x12,
    JL_OPCODE_SYS_UPDATE_BT_STATUS = 0x13,
    JL_OPCODE_SYS_STOP_BT_SCAN = 0x14,
    JL_OPCODE_SYS_BT_CONNECT_SPEC = 0x15,
    JL_OPCODE_SYS_EMITTER_BT_CONNECT_STATUS = 0x19,

    JL_OPCODE_GET_MD5 = 0xD4,
    JL_OPCODE_LOW_LATENCY_PARAM = 0xD5,

    JL_OPCODE_CUSTOMER_USER = 0xFF,
};
/*****
*****/

```

其中 JL_OPCODE_CUSTOMER_USER = 0xFF 用于自定义命令扩展

注意： 不能用其他非 0xFF 的命令号来自定义命令

10.5.2 小机发送需要回复的自定义命令

```

/// 发送需要回复的自定义命令
u32 rcsp_user_send_resp_cmd(u8 *data, u16 len)
{
    u32 ret = 0;
    rcsp_user_printf("send resp cmd\n");
    rcsp_user_printf_buf(data, len);
    ret = JL_CMD_send(JL_OPCODE_CUSTOMER_USER, data, len, 1);
    return ret;
}

```

10.5.3 收到固件发出去,APP 的回复


```
void rcsp_user_recv_resp(u8 *data, u16 len)
{
    ///收到固件发出去,APP的回复
    rcsp_user_printf("resp resp\n");
    rcsp_user_printf_buf(data,len);
}
```

10.5.4 收到 app 主动发过来的需要回复的自定义命令

```
void rcsp_user_recv_cmd_resp(u8 *data, u16 len)
{
    ///收到app主动发过来的需要回复的自定义命令
    rcsp_user_printf("user recv cmd:\n");
    rcsp_user_printf_buf(data,len);
    /* rcsp_user_send_resp_cmd(data, len); */
}
```

10.6 功能配置存储和读取

10.6.1 vm 功能配置项设置

```
#define VM_ALARM_NAME_0 (VM_ALARM_0+6)
#define VM_ALARM_NAME_1 (VM_ALARM_0+7)
#define VM_ALARM_NAME_2 (VM_ALARM_0+8)
#define VM_ALARM_NAME_3 (VM_ALARM_0+9)
#define VM_ALARM_NAME_4 (VM_ALARM_0+10)
#define VM_FM_INFO (VM_ALARM_0+11)

#define CFG_RCSP_ADV_HIGH_LOW_VOL 40
#define CFG_RCSP_ADV_EQ_MODE_SETTING 41
#define CFG_RCSP_ADV_EQ_DATA_SETTING 42
#define ADV_SEQ_RAND 43
#define CFG_RCSP_ADV_TIME_STAMP 44
#define CFG_RCSP_ADV_WORK_SETTING 45
#define CFG_RCSP_ADV_MIC_SETTING 46
#define CFG_RCSP_ADV_LED_SETTING 47
#define CFG_RCSP_ADV_KEY_SETTING 48
#define CFG_USER_DEFINE_END 49
```

eg:


```
syscfg_write(CFG_RCSP_ADV_EQ_MODE_SETTING, eq_setting, 1);
```

10.6.2 vm 功能配置项读取

```
void adv_setting_init(void)
{
    u32 time_stamp = 0;
    if (adv_read_data_from_vm(CFG_RCSP_ADV_TIME_STAMP, (u8 *)&time_stamp, sizeof(time_stamp))) {
        set_adv_time_stamp(time_stamp);
        //deal_time_stamp_setting(0, 0, 0);
    }
}
```

10.7 TWS 数据传输

10.7.1 传输大数据的参考例子，注册一个函数接口

```
static void adv_sync_tws_func_t(void *data, u16 len, bool rx)
{
    if (rx) {
        deal_sibling_setting((u8 *)data);
    }
}

REGISTER_TWS_FUNC_STUB(adv_tws_sync) = {
    .func_id = TWS_FUNC_ID_ADV_SETTING_SYNC,
    .func     = adv_sync_tws_func_t,
};
```

使用例子：

```
tws_api_send_data_to_sibling(adv_setting_to_sync, sizeof(adv_setting_to_sync), TWS_FUNC_ID_ADV_SETTING_SYNC);
```

10.7.2 TWS 之间同步操作参考例子：

```
tws_api_sync_call_by_uuid('T', SYNC_CMD_APP_RESET_LED_UI, 300);
```

Chapter 11 客户常见问题反馈解答

Q1:

Q2:

